

การออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางด้วยโมเดลภาษาขนาดใหญ่เพื่อการตรวจจับและป้องกันภัยไซเบอร์

Design and Implementation of an Endpoint Security System Using Large Language Models for Cyber Threat Detection and Mitigation

จักรกฤษ รางเงิน

Jakkrit Rangngeon

วิทยานิพนธ์นี้สำหรับการศึกษาตามหลักสูตรปริญญา
วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
มหาวิทยาลัยนครพนม

A Thesis Submitted in Fulfillment of the Requirements for the Degree of
Bachelor of Engineering in Computer Engineering
Nakhon Phanom University

2568

ลิขสิทธิ์ของมหาวิทยาลัยนครพนม



การออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางด้วยโมเดลภาษาขนาดใหญ่เพื่อการตรวจจับและป้องกันภัยไซเบอร์

Design and Implementation of an Endpoint Security System Using Large Language Models for Cyber Threat Detection and Mitigation

จักรกฤษ รางเงิน

Jakkrit Rangngeon

วิทยานิพนธ์นี้สำหรับการศึกษาตามหลักสูตรปริญญา
วิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์
มหาวิทยาลัยนครพนม

A Thesis Submitted in Fulfillment of the Requirements for the Degree of
Bachelor of Engineering in Computer Engineering
Nakhon Phanom University

2568

ลิขสิทธิ์ของมหาวิทยาลัยนครพนม

ชื่อวิทยานิพนธ์ การออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางด้วยโมเดลภาษาขนาดใหญ่เพื่อการตรวจจับและป้องกันภัยไซเบอร์

ผู้เขียน นายจักรกฤษ รางเงิน

สาขาวิชา วิศวกรรมคอมพิวเตอร์

อาจารย์ที่ปรึกษาวิทยานิพนธ์หลัก

.....

(ผู้ช่วยศาสตราจารย์ ทรงฤทธิ์ กิติศรีวรพันธุ์, ปร.ด.)

คณะกรรมการสอบ

..... ประธานกรรมการ

(ผู้ช่วยศาสตราจารย์ พัฒนพงษ์ วันจันทึก, Ph.D.)

..... กรรมการ

(อาจารย์ อภิวัตร บุญกอง, ปร.ด.)

..... กรรมการ

(ผู้ช่วยศาสตราจารย์ ชาญวิทย์ สุวรรณพงศ์, ปร.ด.)

คณะวิศวกรรมศาสตร์ มหาวิทยาลัยนครพนม อนุมัติให้นับวิทยานิพนธ์ฉบับนี้ สำหรับการศึกษา ตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์

.....

(ผู้ช่วยศาสตราจารย์ บัณฑิต บุญขาว, Ph.D.)

คณบดีคณะวิศวกรรมศาสตร์

ขอรับรองว่า โครงการนี้มาจากการศึกษาของนักศึกษาเอง และได้แสดงความขอบคุณบุคคลที่มีส่วนช่วยเหลือแล้ว

.....

(ผู้ช่วยศาสตราจารย์ ทรงฤทธิ์ กิติศรีวรพันธุ์, ปร.ด.)

อาจารย์ที่ปรึกษาวิทยานิพนธ์หลัก

.....

(นายจักรกฤษ รางเงิน)

นักศึกษา

ประกาศเจตนารมณ์การใช้ปัญญาประดิษฐ์

ข้าพเจ้าขอรับรองว่า การจัดทำเอกสารฉบับนี้มีการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ (Generative AI) เพื่อเป็นเครื่องมือสนับสนุนการทำงานภายใต้กรอบจริยธรรมทางวิชาการ โดยข้าพเจ้าตระหนักดีว่าเทคโนโลยีดังกล่าวเป็นเพียงเครื่องมือช่วยอำนวยความสะดวก ไม่ใช่ผู้สร้างสรรค์ผลงานหลัก

ข้าพเจ้าตระหนักดีว่าความถูกต้องของข้อมูลเป็นสิ่งสำคัญที่สุด และการนำเสนอข้อมูลที่เป็นเท็จจากข้อผิดพลาดของปัญญาประดิษฐ์เป็นสิ่งที่ไม่อาจยอมรับได้ในทางวิชาการ ดังนั้น ข้อมูลและเนื้อหาทั้งหมดที่ได้จากเทคโนโลยีนี้ จึงได้ผ่านกระบวนการตรวจสอบความถูกต้อง และกลั่นกรองโดยข้าพเจ้าอย่างรอบคอบถี่ถ้วนแล้ว

ข้าพเจ้าขอยืนยันว่า แนวคิดหลัก การวิเคราะห์ และบทสรุปในเอกสารฉบับนี้ เป็นผลงานทางปัญญาของข้าพเจ้าแต่เพียงผู้เดียว และข้าพเจ้าขอรับผิดชอบต่อความถูกต้องของเนื้อหาทั้งหมดที่ปรากฏในเอกสารนี้

.....

(นายจักรกฤษ รางเงิน)

นักศึกษา

Declaration of Generative AI Usage

I hereby certify that the utilization of Generative Artificial Intelligence (AI) in the preparation of this document has been conducted strictly within the framework of academic ethics. I acknowledge that AI serves solely as a supplementary tool and not as the primary author of this work.

I fully recognize that accuracy is paramount and that the presentation of false information resulting from AI errors (Hallucinations) is academically unacceptable. Therefore, all information derived from these tools has undergone rigorous verification and critical review by me to ensure its correctness.

I affirm that the core concepts, analyses, and conclusions presented herein are my original intellectual work. I assume full responsibility for the content and authenticity of this document.

.....

(Mr.Jakkrit Rangngeon)

Author

ชื่อวิทยานิพนธ์ การออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางด้วยโมเดลภาษาขนาดใหญ่เพื่อการตรวจจับและป้องกันภัยไซเบอร์
ผู้เขียน นายจักรกฤษ รางเงิน
อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์ ทรงฤทธิ์ กิติศรีวรพันธุ์, ปร.ด.
สาขาวิชา วิศวกรรมคอมพิวเตอร์
ปีการศึกษา 2568

บทคัดย่อ

ในยุคปัจจุบัน ภัยคุกคามทางไซเบอร์ได้ทวีความรุนแรงและความซับซ้อนยิ่งขึ้น โดยเฉพาะมัลแวร์ที่มีการใช้เทคนิคขั้นสูง ซึ่งทำให้ระบบตรวจจับแบบพึ่งพาฐานข้อมูลแบบดั้งเดิมไม่สามารถป้องกันได้อย่างครอบคลุม โครงงานนี้นำเสนอการออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางแบบผสมผสาน ที่ทำงานบนสถาปัตยกรรมคอนเทนเนอร์บนเซิร์ฟเวอร์จริง ระบบนี้ผสานการทำงานระหว่างการวิเคราะห์ลายเซ็นเพื่อคัดกรองภัยคุกคามที่รู้จัก และการตรวจจับความผิดปกติเพื่อค้นหาภัยคุกคามรูปแบบใหม่ โดยประยุกต์ใช้โมเดลภาษาขนาดใหญ่ ทำหน้าที่วิเคราะห์ความสัมพันธ์ของเหตุการณ์และตัดสินใจโดยใช้ความรู้จากการศึกษาพฤติกรรมผ่านโมเดลภาษาขนาดใหญ่

ผลการทดลองพบว่าระบบสามารถตรวจจับและระบุประเภทของมัลแวร์ขั้นสูง เช่น ValleyRAT ได้อย่างถูกต้องแม่นยำ และมีอัตราการแจ้งเตือนผิดพลาดต่ำจากการทดสอบกับไฟล์ปกติ อีกทั้งผลการประเมินจากผู้เชี่ยวชาญยังระบุว่า รายงานสรุปพฤติกรรมและคำแนะนำภาษาไทยที่สร้างโดยเทคโนโลยีปัญญาประดิษฐ์มีความถูกต้องเชิงบริบทอยู่ในเกณฑ์ดีมากด้วยคะแนนเฉลี่ย 4.7 จาก 5.0 คะแนน ซึ่งช่วยยกระดับความมั่นคงปลอดภัยและการตอบสนองต่อภัยคุกคามให้กับองค์กรได้อย่างมีประสิทธิภาพ

Thesis Title Design and Implementation of an Endpoint Security System Using Large Language Models for Cyber Threat Detection and Mitigation
Author Mr. Jakkrit Rangngeon
Advisor Asst.Prof. Songrit Kitisriworapan, Ph.D.
Major Program Computer Engineering
Academic Year 2025

ABSTRACT

Currently, cyber threats have escalated in severity and complexity, particularly with the emergence of evasive malware and advanced techniques such as APT Beaconing. Traditional signature-based detection systems are often insufficient to provide comprehensive protection against these threats. This project proposes the design and implementation of a hybrid endpoint security system operating on a containerized architecture (Docker) within a production server environment. The system integrates signature analysis to filter known threats with anomaly detection to identify novel and unknown threats. Large Language Models (LLMs) are applied to analyze event correlations and facilitate AI consensus decision-making. Experimental results demonstrate that the system can accurately detect and classify advanced malware, such as ValleyRAT, with a low false positive rate when tested against benign files. Furthermore, expert evaluations indicate that the summarized behavioral reports and Thai recommendations generated by the AI component achieved a highly accurate contextual level with an average score of 4.7 out of 5.0. This enhances the security posture and incident response capabilities of organizational infrastructure.

กิตติกรรมประกาศ

โครงงานฉบับนี้สำเร็จลุล่วงไปได้ด้วยดี ผู้จัดทำขอขอบพระคุณทุกท่านที่มีส่วนช่วยในการให้คำแนะนำ ให้ความช่วยเหลือ และให้การสนับสนุนมาโดยตลอด กราบขอบพระคุณเป็นอย่างสูงแด่ อาจารย์ทรงฤทธิ์ กิติศรีวรพันธุ์, ปร.ด. อาจารย์ที่ปรึกษาโครงงาน ที่ได้กรุณาให้คำปรึกษา แนะนำแนวทาง และชี้แนะข้อบกพร่องต่างๆ ด้วยความเอาใจใส่เสมอมา จนโครงงานฉบับนี้เสร็จสมบูรณ์ ขอกราบขอบพระคุณคณาจารย์ทุกท่านในสาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยนครพนม ที่ได้ประสิทธิ์ประสาทวิชาความรู้และเป็นแบบอย่างที่ดีในการศึกษา ขอขอบคุณผู้พัฒนาเครื่องมือและชุดข้อมูลสาธารณะต่างๆ ที่เป็นส่วนสำคัญในการทดลองของโครงงานนี้ เช่น VMware, REMnux, INetSim และ MalwareBazaarที่ช่วยให้การวิเคราะห์และเก็บรวบรวมข้อมูลเป็นไปได้อย่างราบรื่น สุดท้ายนี้ ขอบคุณเพื่อนๆ ในสาขาวิชาที่คอยช่วยเหลือและแลกเปลี่ยนความรู้ และขอบคุณครอบครัวที่เป็นกำลังใจและให้การสนับสนุนผู้จัดทำเสมอมา คุณค่าและประโยชน์อันใดที่พึงมีจากโครงงานฉบับนี้ ผู้จัดทำขอมอบแด่ผู้มีพระคุณทุกท่าน

จักรกฤษ รางเงิน

สารบัญ

เรื่อง หน้า

บทคัดย่อ vi
ABSTRACT vii
กิตติกรรมประกาศ viii
สารบัญ ix
รายการตาราง xi
รายการภาพประกอบ xii
บทที่ 1 บทนำ 1
1.1 ความสำคัญและที่มา 1
1.2 วัตถุประสงค์ 1
1.3 ขอบเขตของโครงงาน 1
1.4 วิธีดำเนินงานโดยสังเขป 2
1.5 ประโยชน์คาดว่าที่จะได้รับ 2
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง 3
2.1 ภูมิทัศน์ของภัยคุกคามสมัยใหม่และมาตรฐานฐานความรู้สากล 3
2.2 กระบวนทัศน์การป้องกันปลายทางและการวิเคราะห์ทรัพยากรเชิงลึก 3
2.3 ทฤษฎีการตรวจจับมัลแวร์และความผิดปกติของพฤติกรรมบีคอนนิ่ง 4
2.4 ศักยภาพของโมเดลภาษาขนาดใหญ่ในงานด้านความมั่นคงปลอดภัย 4
2.5 ระบบบริหารจัดการลำดับงานอัตโนมัติและการตอบสนองต่อภัยคุกคาม 5
2.6 ชุดคำสั่งควบคุมทรัพยากรคลาวด์แบบอินเทอร์เฟซบรรทัดคำสั่ง 5
2.7 เทคโนโลยีจำลองสภาพแวดล้อมและการประสานงานบริการย่อย 6
2.8 เครื่องมือสำหรับการพัฒนาเว็บส่วนหน้า 9
2.9 งานวิจัยที่เกี่ยวข้อง 11
บทที่ 3 วิธีดำเนินงาน 13
3.1 เครื่องมือและซอฟต์แวร์ที่ใช้ในโครงงาน 13
3.2 สถาปัตยกรรมและแนวทางการทำงานของระบบ 14
3.3 การออกแบบและพัฒนาระบบส่งไฟล์เข้าสู่ Sandbox 27
3.4 การพัฒนาระบบควบคุมและส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง 35
บทที่ 4 การทดลองและวิเคราะห์ผล 38
4.1 การตั้งค่าและขอบเขตการทดลอง 38
4.2 การจัดเตรียมเครื่องมือทดลอง 38
4.3 การจัดเตรียมซอฟต์แวร์และระบบควบคุมส่วนกลาง 39
4.4 การจำแนกประเภทและจัดเตรียมชุดข้อมูลสำหรับการทดสอบ 42
4.5 การวิเคราะห์ความแม่นยำในการตรวจจับมัลแวร์ข้ามแพลตฟอร์ม 43
4.6 การทดสอบสมรรถนะของระบบวิเคราะห์พฤติกรรมบนคลาวด์ 44
4.7 สรุปผลการทดลอง 46

สารบัญ (ต่อ)

เรื่อง หน้า

4.8 อภิปรายผลการทดลอง 47

บทที่ 5 บทสรุปและข้อเสนอแนะ 48

5.1 สรุปผลการดำเนินงาน 48

5.2 ปัญหาและอุปสรรค 48

5.3 ข้อเสนอแนะสำหรับการพัฒนาในอนาคต 49

บรรณานุกรม 50

ภาคผนวก ก การตั้งค่าระบบและซอร์สโค้ด 51

ก.1 ภาพรวมกระบวนการทำงาน 51

ก.2 การตั้งค่าสภาพแวดล้อมจำลองและการส่งข้อมูล 53

ก.3 กลไกการหน่วงเวลาและดึงข้อมูลบันทึกเหตุการณ์ 53

ก.4 การกำหนดคำสั่งและคลังความรู้สำหรับปัญญาประดิษฐ์ 54

ประวัติผู้เขียน 57

รายการตาราง

ตาราง หน้า

ตารางที่ 2.1 หน้าที่และคำอธิบายชุดคำสั่งควบคุมทรัพยากรคลาวด์แบบอินเทอร์เฟซบรรทัดคำสั่ง 6
ตารางที่ 2.2 ชุดคำสั่งพื้นฐานสำหรับการปฏิบัติงานด็อกเกอร์และด็อกเกอร์คอมโพส..... 8
ตารางที่ 2.3 ความแตกต่างและบทบาทหน้าที่ของ Streamlit และ Flask ในระบบไมโครเซอร์วิส..... 10
ตารางที่ 3.1 สรุปเครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาจำแนกตามส่วนงาน 14
ตารางที่ 3.2 รายละเอียดชุดคำสั่ง API สำหรับบริหารจัดการและควบคุมระบบจำลองผ่านชุดคำสั่ง cURL 28
ตารางที่ 3.3 รายละเอียดและตัวอย่างชุดคำสั่งสำหรับการเรียกใช้งานส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง 36
ตารางที่ 4.1 คุณสมบัติเฉพาะของเครื่องคอมพิวเตอร์เสมือน (Virtual Machine) บนระบบ Microsoft Azure 40
ตารางที่ 4.2 รายละเอียดคุณลักษณะทางเทคนิคของเครื่องคอมพิวเตอร์แม่ข่าย (Host System)..... 40
ตารางที่ 4.3 ตัวอย่างชุดข้อมูลและผลการทดสอบการทำงานของไฟล์ในสภาพแวดล้อมจำลอง 43
ตารางที่ 4.4 ผลการตรวจจับและจำแนกประเภทความเสี่ยงของไฟล์จำแนกตามระบบปฏิบัติการ..... 43

รายการภาพประกอบ

รูป หน้า

รูปที่ 2.1 แผนภาพแสดงสถาปัตยกรรมแนวคิดการจำลองระดับระบบปฏิบัติการของด็อกเกอร์ 7
รูปที่ 2.2 แผนภาพแสดงแนวคิดการทำงานร่วมกันระหว่าง Streamlit และ Flask ในสถาปัตยกรรมของระบบ 10
รูปที่ 3.1 แผนภาพสถาปัตยกรรมของระบบ 15
รูปที่ 3.2 ส่วนสรุปผลการวิเคราะห์ความเสี่ยงและมาตรวัดระดับความรุนแรง 16
รูปที่ 3.3 รายละเอียดพฤติกรรมเชิงเทคนิคแยกตามหมวดหมู่และการวิเคราะห์โดยปัญญาประดิษฐ์ 17
รูปที่ 3.4 การแสดงข้อมูลดิบในรูปแบบโครงสร้างมาตรฐาน JSON จากระบบส่วนหลัง 17
รูปที่ 3.5 ข้อมูลเชิงสถิติด้านเวลาและขั้นตอนการทำงานแบบละเอียด 18
รูปที่ 3.6 ภาพรวมการกำหนดลำดับขั้นตอนการทำงานอัตโนมัติบนแพลตฟอร์ม n8n 18
รูปที่ 3.7 สภาพแวดล้อมจำลองแบบบรรทัดคำสั่งบนระบบปฏิบัติการลินุกซ์สำหรับทดสอบมัลแวร์ 19
รูปที่ 3.8 สภาพแวดล้อมจำลองแบบส่วนต่อประสานกราฟิกบนระบบปฏิบัติการวินโดวส์ที่แฝงเครื่องมือดักจับพฤติกรรมมัลแวร์ 20
รูปที่ 3.9 ลำดับกระบวนการทำงานของระบบ 23
รูปที่ 3.10 กระบวนการวิเคราะห์ไฟล์ด้วยปัญญาประดิษฐ์ 24
รูปที่ 3.11 กระบวนการทดสอบไฟล์ในสภาพแวดล้อมจำลอง 26
รูปที่ 3.12 แผนผังลำดับขั้นตอนการทำงานของโปรแกรมตัวแทนภายในสภาพแวดล้อมการทดสอบ 30
รูปที่ 3.13 กระบวนการส่วนหน้าสำหรับการรับไฟล์และควบคุมสภาพแวดล้อมจำลอง 31
รูปที่ 3.14 กระบวนการส่วนหลังสำหรับการวิเคราะห์พฤติกรรมด้วยปัญญาประดิษฐ์และการคืนสภาพระบบ 33
รูปที่ 3.15 ขั้นตอนการฟื้นฟูสภาพแวดล้อม Sandbox บน Microsoft Azure 34
รูปที่ 3.16 ผังการทำงานของส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง 35
รูปที่ 3.17 หน้าจอแสดงประวัติการจัดเก็บผลลัพธ์การวิเคราะห์มัลแวร์บนฐานข้อมูลแอร์เทเบิล 37
รูปที่ 4.1 การกำหนดค่าระบบปฏิบัติการ Windows บนระบบ Microsoft Azure 39
รูปที่ 4.2 การกำหนดค่าระบบปฏิบัติการ Linux บนระบบ Microsoft Azure 39
รูปที่ 4.3 องค์ประกอบความหน่วงเฉลี่ยแบ่งตามขั้นตอนการทำงาน 45
รูปที่ 4.4 การกระจายตัวและความเสถียรของระยะเวลาการทำงานรวม 46
รูปที่ 4.5 ความสัมพันธ์เชิงสหสัมพันธ์ระหว่างระยะเวลาการวิเคราะห์และปริมาณข้อมูลพฤติกรรมที่ได้รับ 47
รูปที่ ก.1 กระบวนการทำงานทั้งหมดบนแพลตฟอร์ม n8n 52
รูปที่ ก.2 การตั้งค่าโหนดสำหรับการควบคุมเครื่องเสมือนและส่งไฟล์เข้าสู่ระบบคลาวด์ 53
รูปที่ ก.3 การออกแบบกลไกหน่วงเวลาและสกัดข้อมูลเวลาทำงานของระบบ 54
รูปที่ ก.4 การตั้งค่าคำสั่งควบคุมและบริบทภายในโหนดประมวลผลของปัญญาประดิษฐ์ 55
รูปที่ ก.5 การตั้งค่าฐานข้อมูลเวกเตอร์เพื่อเป็นเครื่องมือคลังความรู้สำหรับปัญญาประดิษฐ์ 56

บทที่ 1

บทนำ

1.1 ความสำคัญและที่มา

ในปัจจุบันภัยคุกคามทางไซเบอร์มีความซับซ้อนและหลบเลี่ยงการตรวจจับได้ดีขึ้น ส่งผลให้การป้องกันด้วยซอฟแวร์แอนติมัลแวร์ (Malware) แบบดั้งเดิมที่อาศัย ลักษณะเฉพาะ (Signature-based) ของมัลแวร์ไม่เพียงพออีกต่อไป จึงมีการพัฒนาระบบตรวจจับและรักษาความปลอดภัยปลายทาง (Endpoint Detection and Response, EDR) ในการตรวจจับภัยคุกคามยุคใหม่โดยอาศัยการวิเคราะห์พฤติกรรมการทำงานของโปรแกรมในเชิงลึก อย่างไรก็ตาม การทดสอบและประเมินประสิทธิภาพของระบบรักษาความปลอดภัยปลายทาง จำเป็นต้องใช้ชุดข้อมูลตัวอย่างมัลแวร์ที่มีความหลากหลายและครอบคลุมเทคนิคการโจมตีที่ทันสมัยซึ่งการใช้มัลแวร์จริงในการทดสอบมีความเสี่ยงสูงและจัดการได้ยาก เพื่อแก้ไขปัญหานี้ โครงงานได้เสนอวิธีสร้างแซนด์บ็อกซ์ (Sandbox) ซึ่งช่วยให้สามารถรันและสังเกตการณ์มัลแวร์ได้อย่างปลอดภัย ขั้นตอนต่อไปคือการนำพฤติกรรมของมัลแวร์นี้ที่ได้จากระบบจำลอง และส่งต่อพฤติกรรมให้เทคโนโลยีตัวแบบภาษาขนาดใหญ่ (Large Language Models) และการสร้างข้อความโดยการใช้ข้อมูลเสริมจากการค้นคืน (Retrieval-Augmented Generation, RAG) เข้ามาช่วยตรวจสอบมัลแวร์โดยอัตโนมัติ

1.2 วัตถุประสงค์

1.2.1 เพื่อพัฒนาระบบจำลองที่สามารถรันและตรวจสอบพฤติกรรมของมัลแวร์ในสภาพแวดล้อมควบคุมได้
1.2.2 เพื่อประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ ในการวิเคราะห์และจำแนกพฤติกรรมที่เป็นภัยคุกคามจากข้อมูลที่รวบรวมได้จากระบบจำลอง
1.2.3 เพื่อทดสอบและประเมินประสิทธิภาพของระบบในการตรวจจับและวิเคราะห์มัลแวร์ที่มีพฤติกรรมแตกต่างกัน
1.2.4 เพื่อสรุปผลและจัดทำแนวทางการตรวจสอบมัลแวร์

1.3 ขอบเขตของโครงงาน

1.3.1. พัฒนาแซนด์บ็อกซ์ในสภาพแวดล้อมควบคุมสำหรับรันมัลแวร์อย่างปลอดภัยเพื่อตรวจสอบและรวบรวมข้อมูลพฤติกรรม
1.3.2. ประยุกต์ใช้เทคโนโลยีตัวแบบภาษาขนาดใหญ่ ร่วมกับการสร้างข้อความโดยการสร้างข้อความโดยการใช้ข้อมูลเสริมจากการค้นคืน เพื่อสร้างระบบที่สามารถวิเคราะห์และจำแนกพฤติกรรมที่เป็นอันตรายได้
1.3.3. ทำการทดสอบและประเมินประสิทธิภาพของระบบจำลองที่พัฒนาขึ้นโดยใช้มัลแวร์ตัวอย่างที่มีพฤติกรรมแตกต่างกัน

1.4 วิธีดำเนินงานโดยสังเขป

1.4.1. พัฒนาระบบจำลองออกแบบและสร้างสภาพแวดล้อมควบคุมสำหรับใช้รันมัลแวร์ตัวอย่างได้อย่างปลอดภัย เพื่อทำการสังเกตการณ์และรวบรวมข้อมูลพฤติกรรมการทำงาน

1.4.2. รวบรวมข้อมูลและสร้างฐานความรู้นำมัลแวร์ตัวอย่างมารันในระบบจำลองเพื่อเก็บข้อมูลพฤติกรรม พร้อมกันนั้นจะสร้างฐานข้อมูลความรู้ (Knowledge Base) สำหรับระบบการสร้างข้อความโดยการใช้ข้อมูลเสริมจากการค้นคืนโดยใช้ข้อมูลจากแหล่งที่น่าเชื่อถือ เช่น MalwareBazaar abuse.ch (2020)

1.4.3. พัฒนาระบบประยุกต์ใช้เทคโนโลยีตัวแบบภาษาขนาดใหญ่ ร่วมกับเทคนิคการสร้างข้อความโดยการใช้ข้อมูลเสริมจากการค้นคืน เพื่อสร้างระบบที่สามารถวิเคราะห์ข้อมูลพฤติกรรมที่ได้รับ

1.4.4. เชื่อมต่อและประเมินผลนำระบบจำลองมาเชื่อมต่อกับระบบและทดสอบการทำงานโดยรวมด้วยมัลแวร์ตัวอย่างหลายชนิด เพื่อประเมินประสิทธิภาพและความแม่นยำในการวิเคราะห์

1.4.5. สรุปและจัดทำแนวทางสรุปผลการทดลองทั้งหมดและจัดทำเป็นแนวทางการตรวจสอบมัลแวร์โดยใช้ระบบที่พัฒนาขึ้น

1.5 ประโยชน์คาดว่าที่จะได้รับ

1.5.1 ได้รับระบบจำลองที่สามารถวิเคราห์พฤติกรรมมัลแวร์ได้

1.5.2 มีส่วนติดต่อของส่วนต่อประสานการเขียนโปรแกรมประยุกต์ (Application Programming Interface) ระบุพฤติกรรมของมัลแวร์สำหรับนำไปประยุกต์ใน งานอื่นได้

1.5.3 มีแนวทางการสร้างข้อมูลเสริมจากการค้นคืนเข้ามาช่วยตรวจสอบมัลแวร์

1.5.4 มีรายงานสรุปผลการประเมินประสิทธิภาพของระบบที่พัฒนา

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การดำเนินโครงงานวิจัยเรื่องการออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางด้วยโมเดลภาษาขนาดใหญ่เพื่อการตรวจจับและป้องกันภัยไซเบอร์ จำเป็นต้องอาศัยรากฐานทางทฤษฎีที่แข็งแกร่งและการบูรณาการเทคโนโลยีขั้นสูงเพื่อรับมือกับภัยคุกคามที่มีความซับซ้อนและมีการอำพรางตัวในระดับลึก เนื้อหาภายในบทนี้จึงนำเสนอข้อมูลครอบคลุมตั้งแต่ภูมิทัศน์ของภัยคุกคามสมัยใหม่ แนวทางการป้องกันเชิงรุกที่อุปกรณ์ปลายทาง ทฤษฎีการวิเคราะห์พฤติกรรมผิดปกติ ตลอดจนบทบาทสำคัญของโมเดลภาษาขนาดใหญ่ในฐานะกลไกหลักในการประมวลผลข้อมูลความปลอดภัย เพื่อสร้างความเข้าใจในองค์รวมที่นำไปสู่การพัฒนาระบบที่มีประสิทธิภาพ

2.1 ภูมิทัศน์ของภัยคุกคามสมัยใหม่และมาตรฐานฐานความรู้สากล

วิวัฒนาการของภัยคุกคามไซเบอร์ในทศวรรษปัจจุบันได้ก้าวข้ามขีดจำกัดของการโจมตีรูปแบบเดิมไปสู่ลักษณะที่เป็นระบบและมีความต่อเนื่องสูง โดยเฉพาะภัยคุกคามขั้นสูงแบบต่อเนื่อง ซึ่งมุ่งเน้นการแทรกซึมเข้าสู่เป้าหมายอย่างเงียบเชียบและพยายามรักษาการคงอยู่ภายในเครือข่ายให้ยาวนานที่สุดเพื่อบรรลุวัตถุประสงค์เชิงยุทธศาสตร์ ภัยคุกคามกลุ่มนี้มักใช้เทคนิคการอำพรางโค้ดและพฤติกรรมเพื่อหลบเลี่ยงการตรวจจับจากซอฟต์แวร์ป้องกันที่อาศัยลักษณะเฉพาะ เพียงอย่างเดียว Alshamrani et al. (2019)

เพื่อให้การเฝ้าระวังและการระบุตัวตนของภัยคุกคามประเภทต่างๆ เป็นไปอย่างแม่นยำ ฐานความรู้มาตรฐานสากลอย่างไมเทอร์ แอทแทค จึงเข้ามาเป็นแกนหลักในการอ้างอิงพฤติกรรมที่ตรวจพบ โดยเฟรมเวิร์กดังกล่าวได้จัดหมวดหมู่กลยุทธ์ เทคนิค และกระบวนการโจมตีของผู้บุกรุกไว้อย่างละเอียด ตั้งแต่ขั้นตอนการเข้าถึงเบื้องต้นไปจนถึงการควบคุมสั่งการและการโจรกรรมข้อมูล Strom et al. (2020) การนำฐานความรู้นี้มาบูรณาการเข้ากับระบบปัญญาประดิษฐ์ช่วยให้การวิเคราะห์พฤติกรรมของกลุ่มภัยคุกคามระดับสูง เช่น APT29 ซึ่งมีลักษณะเด่นในการใช้เครื่องมือที่เสมือนปกติของระบบ (Living Off the Land) สามารถระบุและจำแนกได้อย่างเป็นระบบผ่านการเทียบเคียงกับมาตรฐานสากล

2.2 กระบวนทัศน์การป้องกันปลายทางและการวิเคราะห์ทรัพยากรเชิงลึก

การย้ายจุดศูนย์กลางของการรักษาความปลอดภัยจากบริเวณรอบนอกเครือข่ายมาสู่การตรวจสอบที่เครื่องคอมพิวเตอร์ปลายทาง หรือ endpoint(Endpoint) โดยตรง ถือเป็นกระบวนทัศน์ที่สำคัญในการรับมือกับภัยคุกคามสมัยใหม่ เนื่องจากระบบปลายทางเป็นแหล่งกำเนิดกิจกรรมสำคัญและเป็นจุดที่มัลแวร์แสดงพฤติกรรมโจมตีอย่างแท้จริง แนวทางการตรวจจับและตอบสนองที่ปลายทาง หรือ ระบบตรวจจับและรักษาความปลอดภัยปลายทาง จึงได้รับการพัฒนาขึ้นเพื่อเก็บบันทึกและวิเคราะห์กิจกรรมบนอุปกรณ์อย่างต่อเนื่องแบบเวลาจริง Bayer et al. (2022)

การเฝ้าระวังที่มีประสิทธิภาพเพื่อตรวจจับมัลแวร์ระดับสูงจำเป็นต้องอาศัยข้อมูลพฤติกรรมเชิงลึกที่ครอบคลุมการใช้งานทรัพยากรระบบพื้นฐาน โดยเฉพาะการตรวจสอบความผิดปกติของการใช้งานหน่วยประมวลผลกลาง หรือ หน่วยประมวลผลกลาง (Central Processing Unit, CPU) และหน่วยความจำหรือ หน่วยความจำ (Memory) ที่มักมีความพึ่งพาหรือการดึงทรัพยากรเกินกว่าปกติเมื่อมัลแวร์ดำเนินการฉีดรหัสอันตรายเข้าสู่โปรเซสสำคัญ เครื่องมืออย่างโพรเซสมอนิเตอร์ Russinovich (2024) และทีชาร์ค (TShark) Wireshark Team (2024) จึงมีบทบาทในการบันทึกเหตุการณ์ต่างๆ อย่างละเอียด ข้อมูลดิบที่ได้จากการเฝ้าติดตาม ทั้งในส่วนของกิจกรรมระบบและการจราจรทางเครือข่าย จะส่งเข้าสู่กระบวนการคัดกรองและแปลงให้อยู่ในรูปแบบที่สมบูรณ์เพื่อให้โมเดลภาษาขนาดใหญ่ทำความเข้าใจบริบทได้อย่างรวดเร็ว

2.3 ทฤษฎีการตรวจจับมัลแวร์และความผิดปกติของพฤติกรรมบีคอนนิ่ง

การตรวจจับภัยคุกคามในพื้นที่จำลองหรือแซนด์บ็อกซ์ อาศัยทฤษฎีการวิเคราะห์พฤติกรรมเชิงพลวัต หรือ การวิเคราะห์เชิงพลวัต (Dynamic Analysis) เพื่อสังเกตกิจกรรมขณะที่โปรแกรมทำงานจริง ซึ่งเป็นวิธีที่ให้ผลลัพธ์แม่นยำกว่าการวิเคราะห์เชิงสถิตในกรณีที่มัลแวร์มีการเข้ารหัสหรือบีบอัดไฟล์เพื่อซ่อนรหัสคำสั่งอันตราย Sikorski and Honig (2012) นอกจากนี้ ทฤษฎีการตรวจจับความผิดปกติ หรือ การตรวจจับแบบอ้างอิงพฤติกรรม (Anomaly Detection) ยังถูกนำมาประยุกต์ใช้เพื่อค้นหารูปแบบการสื่อสารที่น่าสงสัยโดยเฉพาะพฤติกรรมบีคอนนิ่ง ซึ่งเป็นการติดต่อกลับไปยังเครื่องแม่ข่ายสั่งการด้วยความถี่ที่สม่ำเสมอ Abu Talib et al. (2022)

แม้กลุ่มผู้โจมตีจะพยายามหลบเลี่ยงการตรวจจับโดยการแทรกความหน่วงแบบสุ่มหรือการปรับเปลี่ยนช่องทางการสื่อสาร แต่การวิเคราะห์เชิงสถิติควบคู่ไปกับการตรวจสอบทิศทางการไหลของข้อมูลเครือข่ายยังคงระบุความผิดปกติที่เกิดขึ้นได้อย่างมีนัยสำคัญ ข้อมูลที่รวบรวมได้เมื่อนำมาผสานเข้ากับประวัติการเรียกใช้งานไฟล์และการเข้าถึงรีจิสทรี จะส่งเสริมให้นักวิเคราะห์เห็นภาพรวมของวงจรการโจมตีได้อย่างบริบูรณ์และสามารถยับยั้งภัยคุกคามก่อนสร้างความเสียหายร้ายแรงต่อระบบประสาทส่วนกลาง

2.4 ศักยภาพของโมเดลภาษาขนาดใหญ่ในงานด้านความมั่นคงปลอดภัย

การนำโมเดลภาษาขนาดใหญ่ หรือ ตัวแบบภาษาขนาดใหญ่ มาใช้เป็นกลไกหลักในการวิเคราะห์ความปลอดภัยถือเป็นนวัตกรรมที่ช่วยขยายขีดความสามารถในการตรวจจับภัยคุกคาม โดยแบบจำลองดังกล่าวมีความสามารถโดดเด่นในการสรุปความและหาความเชื่อมโยงในข้อมูลที่มีโครงสร้างซับซ้อน Zhao et al. (2023) โดยระบบจะนำข้อมูลที่ผ่านการสกัดเอาเฉพาะคุณลักษณะเด่นและแปลงเป็นรูปแบบซีเอสวีมาให้แบบจำลองประมวลผลเพื่อประเมินระดับความเสี่ยงและความรุนแรง อีกทั้งการใช้สถาปัตยกรรมแบบการเพิ่มความแม่นยำด้วยการค้นคืนข้อมูล หรือ การสร้างข้อความโดยการใช้ข้อมูลเสริมจากการค้นคืน ยังช่วยให้การประมวลผลมีความน่าเชื่อถือผ่านการอ้างอิงข้อมูลจากฐานความรู้ภายนอก Lewis et al. (2020)

2.5 ระบบบริหารจัดการลำดับงานอัตโนมัติและการตอบสนองต่อภัยคุกคาม

การบูรณาการระบบจัดการลำดับงาน ผ่านแพลตฟอร์ม n8n ช่วยให้กระบวนการตั้งแต่การรับข้อมูลจากอุปกรณ์ปลายทางไปจนถึงการสรุปผลการวิเคราะห์ดำเนินไปอย่างต่อเนื่องและเป็นระบบ แพลตฟอร์มดังกล่าวทำหน้าที่เป็นตัวกลางในการประสานงานระหว่างส่วนจัดการกลางและส่วนงานย่อยบนคลาวด์ เพื่อให้แน่ใจว่าการไหลของข้อมูลระหว่างเครื่องมือบันทึกกิจกรรมและโมเดลภาษาขนาดใหญ่เป็นไปอย่างถูกต้อง

การออกแบบเวิร์กโฟลว์อัตโนมัติในลักษณะนี้ไม่เพียงแต่ช่วยเพิ่มประสิทธิภาพในการทำงาน แต่ยังช่วยสร้างมาตรฐานในการรับมือกับภัยคุกคามระดับสูงที่ความเร็วและความแม่นยำเป็นปัจจัยชี้ขาด โดยระบบสามารถดำเนินการฟื้นฟูสภาพแวดล้อมจำลองให้กลับสู่สถานะที่สะอาดได้ทันทีหลังจากเสร็จสิ้นการวิเคราะห์แต่ละรอบ ซึ่งถือเป็นการปิดช่องว่างด้านความมั่นคงปลอดภัยและทำให้ระบบมีความพร้อมสำหรับการตรวจสอบภัยคุกคามในรอบถัดไปได้อย่างไร้รอยต่อ

2.6 ชุดคำสั่งควบคุมทรัพยากรคลาวด์แบบอินเทอร์เฟซบรรทัดคำสั่ง

ไมโครซอฟท์อาซัวร์ (Microsoft Azure) เป็นผู้ให้บริการคลาวด์คอมพิวติ้ง สาธารณะระดับโลกที่นำเสนอโครงสร้างและแพลตฟอร์มสำหรับรองรับการประมวลผลข้อมูลขั้นสูง สถาปัตยกรรมบริการบนระบบคลาวด์สามารถจำแนกออกเป็น 3 ระดับเบื้องต้น ได้แก่ บริการซอฟต์แวร์ (Software as a Service) ซึ่งจัดสรรแอปพลิเคชันผ่านอินเทอร์เน็ตโดยผู้ใช้ไม่ต้องติดตั้งโปรแกรม บริการแพลตฟอร์ม (Platform as a Service) ซึ่งมอบสภาพแวดล้อมเพื่อให้นักพัฒนาสร้างและบริหารจัดการแอปพลิเคชัน และบริการโครงสร้างพื้นฐาน (Infrastructure as a Service) ที่ให้ผู้ใช้งานเช่าทรัพยากรคอมพิวเตอร์ เช่น หน่วยประมวลผลและหน่วยจัดเก็บข้อมูลผ่านเครือข่าย สำหรับโครงงานนี้ การสร้างพื้นที่วิเคราะห์มัลแวร์ หรือ แซนด์บ็อกซ์ อาศัยการสร้างทรัพยากรประเภทเครื่องจำลอง หรือ เครื่องเสมือน (Virtual Machine) ซึ่งจัดอยู่ในกลุ่มบริการโครงสร้างพื้นฐาน หรือ บริการโครงสร้างพื้นฐาน เนื่องจากเทคโนโลยีดังกล่าวเปิดสิทธิ์ให้นักวิจัยสามารถปรับแต่งระบบปฏิบัติการและแกนหลักของซอฟต์แวร์ได้อย่างอิสระเสมือนการเข้าควบคุมเครื่องคอมพิวเตอร์ทางกายภาพ

เพื่อรองรับการตรวจจับมัลแวร์แบบรอบทิศทางและลดภาระการแทรกแซงจากมนุษย์ โครงงานนี้จึงเชื่อมโยงความสามารถในการบริหารจัดการวงจรชีวิตของเครื่องจำลองเข้ากับระบบจัดการกระบวนการทำงานอัตโนมัติ ผ่านแพลตฟอร์ม n8n เนื่องจากรูปแบบการควบคุมการเปิดหรือปิดเครื่องจำลองผ่านหน้าจอผู้ใช้งานแบบกราฟิก หรือ กูอี้ (gui) ไม่สอดคล้องกับเงื่อนไขการทำงานในระบบอัตโนมัติแบบปิด การออกแบบสถาปัตยกรรมจึงมุ่งเน้นไปที่การใช้แพลตฟอร์ม n8n สั่งการระบบคลาวด์ผ่านการปฏิสัมพันธ์ด้วยอินเทอร์เฟซบรรทัดคำสั่ง ซึ่งพฤติกรรมนี้ช่วยให้ระบบกลางสามารถยิงสคริปต์ตอบสนองเพื่อฟื้นฟูสภาพแวดล้อมให้กลับมาเป็นปกติได้ทันทีหลังสิ้นสุดรอบการวิเคราะห์

ในการบริหารจัดการทรัพยากรบนไมโครซอฟท์อาซัวร์สำหรับระบบวิเคราะห์มัลแวร์นี้นั้น แม้ว่าแกนหลักของระบบจะเรียกใช้งานผ่านโปรโตคอลการสื่อสารแบบเรสต์(Representational State Transfer) โดยใช้คำสั่งพื้นฐานอย่างซียูอาร์แอล (client URL, cURL) เพื่อควบคุมความรวดเร็วในการทำงานและลดภาระ

การติดตั้งซอฟต์แวร์ก็ตาม แต่กลไกคำสั่งปฏิบัติการทั้งหมดมีความสอดคล้องกับชุดคำสั่งมาตรฐานของเครื่องมือสั่งการบนระบบคลาวด์อย่างเคร่งครัด โดยคำสั่งสำคัญที่เกี่ยวข้องกับการระงับและเริ่มต้นการทำงานของเครื่องจำลอง รวมถึงการประยุกต์ใช้คำสั่งอเนกประสงค์เพื่อติดต่อกับอินเทอร์เฟซโดยตรง ได้ถูกรวบรวมและอธิบายหน้าที่การปฏิบัติงานดังแสดงใน **ตารางที่** 2.1

สำหรับกระบวนการที่เกี่ยวข้องกับการจัดการสถานะความปลอดภัยและการคืนสภาพแวดล้อมจำลองนั้น ชุดคำสั่งในตารางจะครอบคลุมถึงการคัดลอกข้อมูลดั้งเดิมขึ้นมาใหม่ การเปลี่ยนแปลงการตั้งค่าของเครื่องจำลอง ตลอดจนกระบวนการทำลายและลบข้อมูลที่ปนเปื้อน การทำงานที่สอดประสานกันผ่านชุดคำสั่งที่กล่าวมาช่วยยืนยันการออกแบบกลไกให้มีลำดับขั้นตอนปฏิบัติที่ปลอดภัย ควบคุมเป้าหมายได้อย่างแม่นยำ และต่อยอดบริการคลาวด์ให้เกิดประโยชน์สูงสุดเหนือสถาปัตยกรรมเดิม

ตารางที่ 2.1: หน้าที่และคำอธิบายชุดคำสั่งควบคุมทรัพยากรคลาวด์แบบอินเทอร์เฟซบรรทัดคำสั่ง

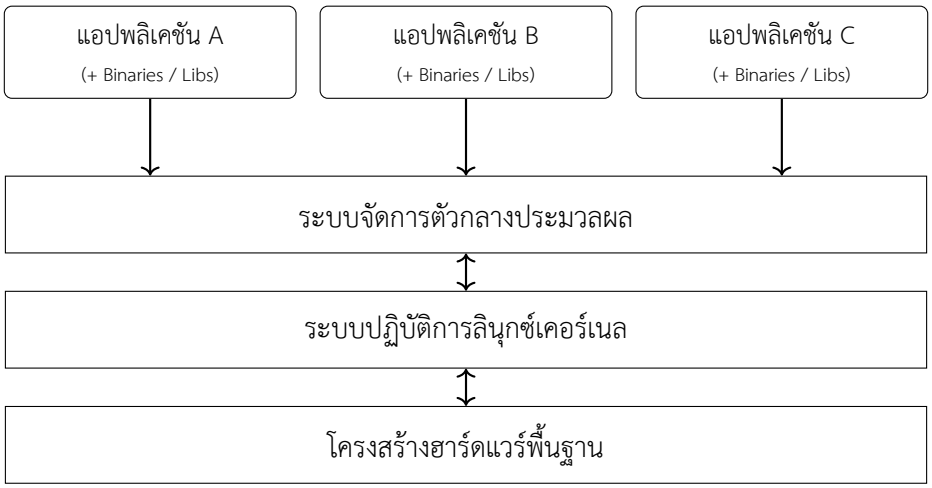
รูปแบบคำสั่ง	**คำอธิบายหน้าที่การทำงาน**
az vm deallocate	ระงับการทำงานของเครื่องจำลองและคลายการจองทรัพยากรเพื่อลดภาระค่าใช้จ่าย
az vm start	เรียกคืนทรัพยากรส่วนกลางและสั่งให้เครื่องจำลองกลับมาเริ่มต้นประมวลผลรอบถัดไป
az rest	ติดต่อกับอินเทอร์เฟซแบบเรสต์โดยตรงเพื่อดึงข้อมูลสถานะหรือส่งคำขอแบบปรับแต่งเอง
az disk create	สร้างหน่วยจัดเก็บข้อมูลก้อนใหม่โดยทำสำเนาจากภาพบันทึกรหัสที่มีความปลอดภัย
az vm update	ปรับปรุงการตั้งค่าของเครื่องจำลองเพื่อสับเปลี่ยนไปเชื่อมต่อกับหน่วยจัดเก็บข้อมูลก้อนใหม่
az disk delete	ทำลายและลบหน่วยจัดเก็บข้อมูลก้อนเดิมที่ผ่านการทดสอบมัลแวร์ออกอย่างถาวร

2.7 เทคโนโลยีจำลองสภาพแวดล้อมและการประสานงานบริการย่อย

เทคโนโลยีการจำลองสภาพแวดล้อมระดับระบบปฏิบัติการ อย่างด็อกเกอร์ Merkel (2014) ได้ก้าวเข้ามาเป็นเครื่องมือหลักในการพัฒนาและนำส่งซอฟต์แวร์สมัยใหม่ที่ต้องการความคล่องตัวสูง เทคโนโลยีดังกล่าวทำหน้าที่บรรจุแอปพลิเคชันพร้อมทั้งชุดโปรแกรมสนับสนุน คลังคำสั่ง และทรัพยากรระบบที่จำเป็นทั้งหมดลงในหน่วยประมวลผลย่อยที่ถูกปิดล้อมอย่างอิสระซึ่งเรียกว่าคอนเทนเนอร์ จุดเด่นที่สำคัญของด็อกเกอร์คือการทำงานผสานร่วมกับอวัยวะส่วนกลางหรือเคอร์เนลของระบบปฏิบัติการลินุกซ์ โดยตรง การเรียกใช้งานคุณสมบัติเฉพาะอย่างเนมสเปซ รับประกันการแยกเขตแดนของทรัพยากรประมวลผล เพื่อไม่ให้ก้าวก่ายระหว่างโปรเซส ขณะที่การใช้คอนโทรลกรุ๊ป รับหน้าที่จำกัดปริมาณการใช้ทรัพยากรฮาร์ดแวร์ให้เป็นไปตามโควตาที่กำหนด ส่งผลให้แต่ละคอนเทนเนอร์สามารถทำงานได้อย่างคู่ขนานกันและมีความปลอดภัยสูง โดยไม่จำเป็นต้องสร้างระบบปฏิบัติการจำลองใหม่ขึ้นมาครอบทับฮาร์ดแวร์ทั้งระบบเหมือนเทคโนโลยีเครื่องจำลอง

แบบดั้งเดิม แนวทางกระบวนทัศน์นี้ช่วยลดภาระน้ำหนักของสภาพแวดล้อมรวม และส่งเสริมให้แอปพลิเคชันสามารถเริ่มระบบใหม่ได้ในระดับเสี้ยววินาที

สถาปัตยกรรมระดับซอฟต์แวร์ของด็อกเกอร์ยังเป็นรากฐานสนับสนุนการประมวลผลในรูปแบบไมโครเซอร์วิส อย่างเต็มประสิทธิภาพ โดยมีเป้าหมายในการแตกโครงสร้างแอปพลิเคชันขนาดใหญ่ที่ผูกรวมกันแบบดั้งเดิมออกเป็นบริการย่อยที่มีหน้าที่ประมวลผลเฉพาะเจาะจง การแบ่งแยกองค์ประกอบต่างๆ ออกเป็นส่วนอิสระส่งเสริมผู้พัฒนาให้เข้ามาปรับปรุง แก้ไขบั๊ก หรือขยายขนาดทรัพยากรเฉพาะจุดที่มีการใช้งานหนักสะสมได้อย่างยืดหยุ่น โดยไม่ส่งผลกระทบต่อความเสถียรของบริการส่วนอื่นๆ ในภาพรวม การเลือกใช้คอนเทนเนอร์เป็นภาชนะในการห่อหุ้มบริการขั้นพื้นฐานรับประกันความเที่ยงตรงของการทำงานและแสดงผลลัพธ์ที่สม่ำเสมอไม่ว่าจะเคลื่อนย้ายไปทำงานบนเครื่องปรับแต่งส่วนบุคคล เครื่องคลาวด์องค์กร หรือบนระบบทดสอบคุณภาพก็ตาม สถาปัตยกรรมเชิงประกอบนี้เปิดทางแพลตฟอร์มให้มีความทนทานต่อข้อบกพร่องและสามารถผลักดันหน่วยบูรณาการระบบอย่างต่อเนื่องสู่วงจรการผลิตจริงตามหลักการวิศวกรรมซอฟต์แวร์ร่วมสมัย



รูปที่ 2.1: แผนภาพแสดงสถาปัตยกรรมแนวคิดการจำลองระดับระบบปฏิบัติการของด็อกเกอร์

การสร้างภาพจำลองข้อมูล เพื่อนำมาใช้เป็นฐานตั้งต้นในการสร้างคอนเทนเนอร์จำเป็นต้องอาศัยกลไกการนิยามไฟล์สั่งการที่เรียกว่าด็อกเกอร์ไฟล์ (Dockerfile) ซึ่งเอกสารประเภทดังกล่าวทำหน้าที่เสมือนคู่มือที่กำหนดพฤติกรรมและอธิบายกระบวนการนำเข้าองค์ประกอบต่างๆ ของระบบเข้าไปทีละชั้นข้อมูล เริ่มต้นจากการเลือกภาพจำลองฐานตั้งต้น การเรียกใช้ระบบจัดการแพ็กเกจเพื่อติดตั้งคลังคำสั่งย่อยในสภาวะแวดล้อมปิด รวมถึงการกำหนดตัวแปรระบบที่จำเป็นเพื่อรองรับการสั่งงานเบื้องลึก รูปแบบการนิยามโครงสร้างไฟล์ในลักษณะการตั้งโปรแกรมนี้ช่วยลดความกำกวมของการเตรียมระบบปฏิบัติการในสภาพแวดล้อมที่แตกต่างกัน ดังที่แสดงแนวคิดสถาปัตยกรรมใน **รูปที่** 2.1 ตัวอย่างเชิงประจักษ์ในระบบการวิเคราะห์มัลแวร์โครงงานนี้ ฝั่งประมวลผลกลางอาศัยภาพจำลองหลักจากระบบแพลตฟอร์มอัตโนมัติ n8n ที่ได้รับการดัดแปรสิทธิ์ให้ทำงานเป็นกลไกหลัก พร้อมการติดตั้งเครื่องมือวิเคราะห์เครือข่ายเพิ่มเติม ขณะที่ส่วนแสดงผลส่วนหน้าถูกพัฒนาบนโครงสร้างโปรแกรมเบาบางทับระบบไพทอนเพื่อเสริมโมดูลสตรีมลิต (Streamlit) และส่วนติดต่อสั่งการกับระบบบริหารทรัพยากรคลาวด์ไมโครซอฟท์อาซัวร์อย่างราบรื่น

เมื่อระบบนิเวศของแอปพลิเคชันทวีความซับซ้อนและจำเป็นต้องพึ่งพากลไกการสื่อสารข้ามคอนเทนเนอร์มากกว่าหนึ่งหน่วย เครื่องมือเสริมประสิทธิภาพอย่างด็อกเกอร์คอมโพส จึงถูกนำเข้ามาเพื่อรองรับบทบาทในการบริหารจัดการวัฏจักรของกลุ่มคอนเทนเนอร์ทั้งหมดผ่านการนิยามไฟล์ข้อมูลโครงสร้างแบบยามัล เพียงไฟล์เดียวในนาม docker-compose.yml เอกสารฉบับนี้รับหน้าที่บรรยายโครงข่ายการสื่อสารระหว่างคอนเทนเนอร์ การจัดสรรพื้นที่จัดเก็บข้อมูลถาวรเพื่อให้บันทึกรายการกิจกรรมไม่สูญหายเมื่อหยุดระบบ รวมถึงการสร้างโครงข่ายเสมือนเพื่อหลอมรวมระบบฐานข้อมูลโปสเกรสเอสคิวแอล ระบบจัดการตรรกะเบื้องหลัง และแพลตฟอร์มนำเสนอข้อมูลเข้าด้วยกัน การนำเทคนิคดังกล่าวมาประยุกต์ใช้อธิบายได้อย่างชัดเจนผ่านโครงสร้างการจัดกลุ่มในโครงงาน ซึ่งเชื่อมคอนเทนเนอร์ให้อยู่ร่วมกันในเครือข่าย malnet การร้อยเรียงระบบให้ทำงานประสานกันลักษณะนี้ช่วยก้าวข้ามความท้าทายด้านการจัดการโครงสร้าง ส่งผลให้อินสแตนซ์ทั้งหมดสามารถส่งผ่านข้อความควบคุมไปมาได้อย่างมีบูรณาการ

ในบริบทของการสั่งงานเครื่องมือระบบผ่านส่วนต่อประสานโปรแกรมเพื่อควบคุมวัฏจักรสภาพแวดล้อมคอนเทนเนอร์ ระบบอาศัยชุดคำสั่งแบบครอบคลุมวงจรชีวิตการทำงานให้เรียกใช้ได้อย่างกระชับตั้งแต่สร้างคอนเทนเนอร์ เริ่มกระบวนการ ลบส่วนเหลือทิ้ง และเฝ้าสังเกตสถานะ การสนับสนุนของเครื่องมือกลุ่มคำสั่งช่วยลดความล่าช้าสะสมของนักวิจัยตลอดช่วงอายุการทดสอบ พฤติกรรมการสร้างคำสั่งและไวยากรณ์พื้นฐานจำแนกแยกส่วนระหว่างตัวกลางหลักและตัวขยายคอมโพสถูกจัดกลุ่มรวบรวมพร้อมความหมายตามรายละเอียดใน **ตารางที่** 2.2 ความเข้าใจในขอบเขตและการบังคับใช้ไวยากรณ์จำเพาะดังกล่าวถือเป็นทักษะบังคับระดับพื้นฐานเพื่อรับประกันการจำลองสภาวะแวดล้อมเสมือนให้ราบรื่น ปลอดจากความบกพร่องที่รบกวนขบวนการส่งถ่ายทรัพยากร และรักษาความแข็งแกร่งของระบบเครือข่ายจำลองเพื่อรองรับการยิงพฤติกรรมโจมตีไซเบอร์ได้ตลอดเวลา

ตารางที่ 2.2: ชุดคำสั่งพื้นฐานสำหรับการปฏิบัติงานด็อกเกอร์และด็อกเกอร์คอมโพส

รูปแบบคำสั่ง	**คำอธิบายหน้าที่การทำงาน**
docker build -t <image> .	สร้างภาพจำลองต้นแบบจากเอกสารด็อกเกอร์ไฟล์ผ่านการกำหนดป้ายกำกับและดึงข้อมูลพื้นฐาน
docker run <image>	สั่งรันให้ภาพจำลองเปลี่ยนสถานะเป็นคอนเทนเนอร์ที่พร้อมปฏิบัติงานบนพื้นที่โปรเซสระบบ
docker ps -a	ประมวลผลและนำเสนอรายงานรายชื่อของคอนเทนเนอร์ทั้งหมดไม่ว่าจะอยู่ในสถานะใดก็ตาม
docker rm <id>	ถอดถอนและระงับสิทธิ์คอนเทนเนอร์ที่ตายแช่แข็งแล้วออกจากความจำเพื่อคืนทรัพยากรว่าง
docker rmi <image>	ลบล้างภาพต้นแบบเก่าที่ไม่มีคอนเทนเนอร์ใดอ้างอิงถึงเพื่อทำความสะอาดสื่อบันทึกข้อมูลหลัก
docker-compose up -d	ริเริ่มขั้นตอนวัฏจักรการสร้างโครงข่ายและเริ่มทุกบริการย่อยให้ทำงานในสภาวะเบื้องหลัง
docker-compose down	บังคับจำกัดสิทธิ์ยุติระบบคอมโพสและกวาดล้างทรัพยากรเครือข่ายเสมือนเพื่อเคลียร์สภาวะ
docker-compose logs -f	ติดตามปูมหลังผลประมวลผลหรือข้อความแจ้งเตือนสถานะมาแสดงออกแบบต่อเนื่องเวลาจริง

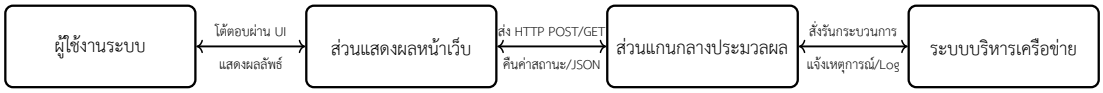
2.8 เครื่องมือสำหรับการพัฒนาเว็บส่วนหน้า

การพัฒนาซอฟต์แวร์สมัยใหม่ที่เน้นความคล่องตัวและการส่งมอบผลลัพธ์ที่รวดเร็ว มักจะพึ่งพาชุดเครื่องมือที่สามารถลดความซับซ้อนของโครงสร้างพื้นฐานและให้ความสำคัญกับการเขียนตรรกะการทำงานหลัก เป็นสำคัญ สำหรับการพัฒนาแอปพลิเคชันที่ต้องการส่วนแสดงผลข้อมูลในรูปแบบเว็บเชิงโต้ตอบ กรอบการทำงานสตรีมลิต ได้รับความนิยมอย่างแพร่หลายในแวดวงวิทยาการข้อมูลและการวิเคราะห์ความปลอดภัยทางไซเบอร์ สตรีมลิตเป็นการทำงานบนภาษาไพทอน บริสุทธิ์ที่ผู้พัฒนาไม่จำเป็นต้องมีความรู้ลึกซึ้งเกี่ยวกับการเขียนโค้ดฝั่งหน้าบ้าน อย่างเอชทีเอ็มแอล ซีเอสเอส หรือจาวาสคริปต์ แต่สามารถสถาปนาหน้าปัดควบคุม หรือระบบโต้ตอบกับผู้ใช้เพื่อนำเสนอผลการวิเคราะห์ข้อมูลขนาดใหญ่ได้อย่างรวดเร็ว กลไกเบื้องหลังของสตรีมลิตอาศัยแนวคิดรีแอคทีฟ โดยทุกครั้งที่ผู้ใช้มีการเปลี่ยนสถานะของตัวแปรนำเข้า ระบบจะทำการรันโค้ดฝั่งเซิร์ฟเวอร์ใหม่ตั้งแต่ต้นจนจบเพื่ออัปเดตผลลัพธ์บนหน้าจอให้ประสานกันทันที ข้อดีนี้ทำให้การสร้างระบบติดตามผลการทำงานของแซนด์บ็อกซ์ หรือการนำเสนอตัวชี้วัดความเสี่ยงปัญญาประดิษฐ์ทำได้อย่างไร้รอยต่อ

ในขณะที่ส่วนติดต่อผู้ใช้ถูกขับเคลื่อนด้วยสตรีมลิต การบริหารจัดการคำร้องขอ และการเชื่อมโยงระบบเพื่อพูดคุยกับบริการอื่นๆ ด้านหลังบ้านยังคงต้องการกรอบการทำงานเบาบางที่รับหน้าที่เป็นส่วนต่อประสานโปรแกรมประยุกต์แบบเรสต์ หรือ เอพีไอ (Application Programming Interface) ซึ่งฟลาสก์ (Flask) ได้เข้ามาเติมเต็มช่องว่างในส่วนนี้ ฟลาสก์เป็นกรอบการทำงานสำหรับพัฒนาเว็บแบบไมโครเฟรมเวิร์กที่ไม่มีการยัดเยียดชุดคำสั่งหรือโครงสร้างไดเรกทอรีที่ตายตัว ส่งผลให้นักพัฒนาสามารถปรับแต่งและผนวกเฉพาะโมดูลที่จำเป็นต่อการใช้งานจริงเท่านั้น ในบริบทของโครงงานนี้ ฟลาสก์ถูกใช้งานเพื่อรับข้อมูลไฟล์ต้องสงสัยหรือคำสั่งต่างๆ จากสตรีมลิต ก่อนจะทำหน้าที่กรองข้อมูล ดัดแปลงโครงสร้าง หรือประสานงานโดยตรงกับระบบการควบคุมของไมโครซอฟท์อาซัวร์ และแพลตฟอร์มอัตโนมัติ n8n การเลือกใช้ระดับชั้นตัวกลางนี้ช่วยปิดบังความซับซ้อนของระบบปฏิบัติการเบื้องลึกและรับประกันความปลอดภัยของข้อมูล ก่อนที่จะส่งผ่านสัญญานไปถึงส่วนสับประวิตช์ทรัพยากรส่วนกลางของคลาวด์

การจับคู่กันระหว่างสตรีมลิตและฟลาสก์สะท้อนให้เห็นถึงกระบวนทัศน์การออกแบบซอฟต์แวร์สไตล์ไมโครเซอร์วิส ได้อย่างทรงประสิทธิภาพ โดยหลักการของไมโครเซอร์วิสคือการหลีกเลี่ยงการสร้างซอฟต์แวร์ก้อนใหญ่เชิงเดี่ยว แต่เน้นย้ำสถาปัตยกรรมที่แบ่งภาระหน้าที่ออกไปตามแต่ละความรับผิดชอบอย่างชัดเจน ในที่นี้ สตรีมลิตจะสวมบทบาทในการดูแลแค่เพียงประสบการณ์ของผู้ใช้ และการจัดรูปแบบการนำเสนอ เท่านั้น โดยเมื่อต้องมีการเข้าถึงทรัพยากรหนัก เช่น การสั่งวิเคราะห์มัลแวร์ ระบบจะกระทำการเพียงส่งเอพีไอ (เอพีไอ) ไปยังคิวรับงานของฟลาสค์ โครงสร้างที่แยกขาดจากกัน เช่นนี้ลดความเสี่ยงที่ระเบียบวิธีประมวลผลขนาดใหญ่จะลามมาทำให้ส่วนแสดงผลหน้าเว็บเกิดการค้าง นอกจากนี้องค์ประกอบของทั้งสองเฟรมเวิร์กยังมีขนาดกะทัดรัด สามารถห่อหุ้มในด็อกเกอร์คอนเทนเนอร์ แยกส่วนจัดการกันได้อย่างยืดหยุ่น เพิ่มความคล่องตัวเมื่อต้องสเกลหรือกระจายขนาดการรองรับผู้ใช้ ดังที่แสดงแนวคิดใน **รูปที่** 2.2

เพื่อสรุปขีดความสามารถและความเหมาะสมในการเลือกใช้เครื่องมือทั้งสองในการพัฒนาซอฟต์แวร์ของโครงงาน ได้มีการจัดทำตารางเพื่อรวบรวมลักษณะเด่นที่ตอบโจทย์รูปแบบสถาปัตยกรรมบริการย่อย และข้อจำกัดที่จำเป็นต้องนำมาพิจารณาประกอบการออกแบบแสดงดัง **ตารางที่** 2.3 ความเข้าใจใน



รูปที่ 2.2: แผนภาพแสดงแนวคิดการทำงานร่วมกันระหว่าง Streamlit และ Flask ในสถาปัตยกรรมของระบบ

ขอบเขตการทำงานอย่างถ่องแท้เป็นกุญแจสำคัญที่ทำให้การเขียนโค้ดแต่ละส่วนมุ่งเน้นไปยังวัตถุประสงค์โดยไม่ซ้ำซ้อนกัน ส่งผลให้การบำรุงรักษาและการสืบค้นข้อผิดพลาดระหว่างรอยต่อทำได้อย่างมีระบบ

ตารางที่ 2.3: ความแตกต่างและบทบาทหน้าที่ของ Streamlit และ Flask ในระบบไมโครเซอร์วิส

หัวข้อเปรียบเทียบ	**สตรีมลิต**	**ฟลาสก์**
บทบาทหน้าที่หลัก	รับผิดชอบส่วนติดต่อกับผู้ใช้งาน จัดการการแสดงผล และส่งคำร้องขอ	ดำเนินการระดับตรรกะเบื้องหลัง เปิดพอร์ตเชื่อมต่อ และประสานงานเซิร์ฟเวอร์ย่อย
กลไกประมวลผล	วนรอบการทำงานโค้ดซ้ำเมื่อตรวจพบสถานะการป้อนข้อมูลเปลี่ยนแปลง	รอรับและตอบกลับข้อมูลผ่านระเบียบวิธีตามโปรโตคอลมาตรฐานเครือข่าย
ลักษณะการใช้งาน	ใช้สร้างหน้าต่างอัปโหลดไฟล์ และนำเสนอข้อมูลเกจวัดระดับความปลอดภัย	ใช้ดักจับคำสั่งวิเคราะห์ บันทึกข้อมูล และบริหารจราจรก่อนส่งไปยังฐานระบบ

ตัวอย่างรหัสคำสั่ง สำหรับการขึ้นโครงร่างพื้นฐานเพื่อริเริ่มการเชื่อมต่อไปมาระหว่างส่วนบริการรับคำร้องของฟลาสค์และส่วนควบคุมการแสดงผลของสตรีมลิต ได้รับการจัดเรียงและตัดทอนบางส่วนเพื่อแสดงพฤติกรรมหลักของการทำงาน โดยรหัสฝั่งเซิร์ฟเวอร์จะทำหน้าที่เปิดช่องรับข้อมูลคล้ายตัวอย่างดัง **รายการรหัสที่** 2.1 ส่วนฝั่งผู้สนทนาหน้าแรกอาศัยคลังคำสั่งของสตรีมลิตควบคู่กับการร้องขอเอพีไอตาม **รายการรหัสที่** 2.2 อนึ่ง การเขียนรหัสภาษาไพทอนในรายการตัวอย่างจะใช้ขีดเส้นใต้ _ เพื่อใช้แทนการเว้นวรรคตัวแปรตามข้อกำหนดสากลของภาษา

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/analyze', methods=['POST'])
def analyze_endpoint():
    if 'file' not in request.files:
        return jsonify({'error': 'No␣file␣uploaded'})

    file_obj = request.files['file']
    filename = request.args.get('filename', 'unknown')

    return jsonify({
        'Status': 'ANALYSIS_STARTED',
        'message': f'กำลังดำเนินการตรวจสอบสำหรับ␣{filename}'
    })
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001)
```

โค้ดที่ 2.1: ตัวอย่างการกำหนดบริการรับส่งข้อมูลด้วยกรอบการทำงาน Flask

```
import streamlit as st
import requests

st.title("ระบบวิเคราะห์ความปลอดภัย")
uploaded_file = st.file_uploader("อัปโหลดไฟล์")

if uploaded_file is not None:
    if st.button("ประมวลผลไฟล์"):
        files = {'file': (uploaded_file.name, uploaded_file.getvalue())}
        res = requests.post("http://localhost:5001/analyze", files=files)

        if res.status_code == 200:
            st.success(res.json()['message'])
```

โค้ดที่ 2.2: ตัวอย่างการเขียนส่วนต่อประสานผู้ใช้ด้วยกรอบการทำงาน Streamlit

2.9 งานวิจัยที่เกี่ยวข้อง

จากการสำรวจงานวิจัยที่เกี่ยวข้อง พบความพยายามในการนำเทคโนโลยีสมัยใหม่มาประยุกต์ใช้เพื่อเพิ่มประสิทธิภาพในการตรวจจับภัยคุกคาม โดย Alshamrani และคณะ Alshamrani et al. (2019) ได้นำเสนอการวิเคราะห์พฤติกรรมมัลแวร์ในกลุ่มภัยคุกคามขั้นสูงที่มุ่งเน้นการหลบเลี่ยงการตรวจจับผ่านเทคนิคที่ซับซ้อน ซึ่งสอดคล้องกับงานวิจัยของ Abu Talib และคณะ Abu Talib et al. (2022) ที่ให้ความสำคัญกับการตรวจจับสัญญาณบีคอนนิ่งผ่านการวิเคราะห์เชิงสถิติเพื่อระบุการสื่อสารระหว่างมัลแวร์และเครื่องแม่ข่ายสั่งการได้อย่างแม่นยำ ในมิติของการเฝ้าระวังที่อุปกรณ์ปลายทาง Bayer และคณะ Bayer et al. (2022) ได้ยืนยันถึงความสำคัญของระบบตรวจจับและตอบสนองที่ปลายทางในการวิเคราะห์กิจกรรมที่ผิดปกติซึ่งให้ประสิทธิภาพเหนือกว่าการใช้เพียงลายนิรหัสไฟล์แบบดั้งเดิม

นอกจากนี้ ความก้าวหน้าของปัญญาประดิษฐ์ส่งผลให้เกิดการบูรณาการโมเดลภาษาขนาดใหญ่ในงานความปลอดภัยไซเบอร์ โดย Zhao และคณะ Zhao et al. (2023) ได้สำรวจขีดความสามารถของแบบจำลองในการจัดการข้อมูลปริมาณมหาศาลและการสรุปความที่ซับซ้อน ซึ่งเชื่อมโยงกับแนวคิดของ Lewis และคณะ Lewis et al. (2020) ที่นำเสนอสถาปัตยกรรม Retrieval-Augmented Generation เพื่อเพิ่มความแม่นยำในบริบทเฉพาะทางและลดปัญหาการสร้างข้อมูลที่คลาดเคลื่อน ในขณะที่งานวิจัยของ Bhatt และคณะ Bhatt et al. (2021) ได้เน้นย้ำถึงบทบาทของระบบอัตโนมัติในการประสานงานลำดับงานความปลอดภัยเพื่อ

ลดระยะเวลาในการตอบสนองต่อภัยคุกคาม สุดท้าย งานวิจัยของ Ding และคณะ Ding et al. (2023) ได้สรุปถึงความท้าทายและโอกาสในการใช้โมเดลภาษาขนาดใหญ่เพื่อการให้เหตุผลเชิงความปลอดภัยและการอธิบายพฤติกรรมของภัยคุกคาม ซึ่งเป็นรากฐานสำคัญที่โครงงานนี้นำมาพัฒนาเพื่อเชื่อมโยงพฤติกรรมมัลแวร์เข้ากับฐานความรู้มาตรฐานสากล

บทที่ 3

วิธีดำเนินงาน

การดำเนินงานวิจัยเพื่อออกแบบและพัฒนาระบบรักษาความปลอดภัยปลายทางแบบอัตโนมัติอาศัยการกำหนดระเบียบวิธีวิจัยและขั้นตอนการปฏิบัติงานอย่างเป็นลำดับขั้นตอน โดยเนื้อหาภายในบทนี้ประกอบด้วยรายละเอียดการคัดเลือกเครื่องมือและซอฟต์แวร์สำหรับการพัฒนาในหัวข้อ 3.1 จากนั้นจะอธิบายถึงโครงสร้างสถาปัตยกรรมและกระบวนการทำงานในภาพรวมที่ระบุไว้ในหัวข้อ 3.2 ตลอดจนรายละเอียดเชิงลึกเกี่ยวกับการพัฒนาระบบนำส่งไฟล์เข้าสู่พื้นที่ทดสอบบนระบบคลาวด์ในหัวข้อ 3.3 รวมถึงการพัฒนาระบบควบคุมและส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลองตามที่ระบุในหัวข้อ 3.4 เพื่อมุ่งเน้นความสอดคล้องของการทำงานในแต่ละภาคส่วนอย่างมีประสิทธิภาพ

3.1 เครื่องมือและซอฟต์แวร์ที่ใช้ในโครงงาน

การพัฒนาระบบในครั้งนี้อาศัยการบูรณาการเทคโนโลยีและบริการหลากหลายรูปแบบเพื่อให้สามารถตอบสนองต่อการทำงานแบบอัตโนมัติได้อย่างสมบูรณ์ โดยผู้วิจัยได้กำหนดเกณฑ์การคัดเลือกเครื่องมือที่มุ่งเน้นความยืดหยุ่น ความปลอดภัยของข้อมูล และประสิทธิภาพในการประมวลผลข้อมูลขนาดใหญ่ ซึ่งความเชื่อมโยงระหว่างเครื่องมือและหน้าที่หลักภายในระบบวิเคราะห์มัลแวร์ปรากฏรายละเอียดสรุปไว้ใน **ตารางที่** 3.1

ในส่วนของกลไกการทำงานบนระบบคลาวด์นั้น แพลตฟอร์ม n8n รับบทบาทเป็นแกนกลางสำคัญในการควบคุมลำดับงานผ่านการเชื่อมต่ออินเทอร์เฟซการเขียนโปรแกรมแอปพลิเคชัน โดยสาเหตุที่เลือกใช้เครื่องมือดังกล่าวเนื่องจากเป็นแพลตฟอร์มรูปแบบรหัสต่ำ (Low-code) ที่มีความยืดหยุ่นสูงและรองรับการติดตั้งบนเซิร์ฟเวอร์ส่วนตัว (Self-hosted) ซึ่งช่วยรักษาความมั่นคงปลอดภัยของข้อมูลวิจัยภายในองค์กรได้เป็นอย่างดี นอกจากนี้ระบบยังทำงานประสานกับแบบจำลองภาษาขนาดใหญ่ Gemini ซึ่งมีคุณสมบัติเด่นด้านหน้าต่างบริบท (Context Window) ที่กว้างขวางเป็นพิเศษ ส่งผลให้สามารถนำเข้าไฟล์เหตุการณ์ขนาดใหญ่มาวิเคราะห์พฤติกรรมได้อย่างครบถ้วนโดยไม่สูญเสียรายละเอียดสำคัญ ในขณะเดียวกันระบบยังมีการบูรณาการฐานข้อมูลเวกเตอร์ในรูปแบบหน่วยความจำเพื่อทำหน้าที่เป็นฐานความรู้สำหรับตัวแทนวิเคราะห์ด้วยปัญญาประดิษฐ์ โดยมีการจัดเก็บข้อมูลพื้นฐานเชิงพฤติกรรมและข้อมูลทางเทคนิคของภัยคุกคามเพื่อนำมาใช้เปรียบเทียบและเพิ่มความแม่นยำในการระบุพฤติกรรมมัลแวร์ผ่านกลไกการค้นหาข้อมูลที่เกี่ยวข้องร่วมกับข้อมูลที่ได้รับจากการทดสอบจริง

สำหรับการบริหารจัดการข้อมูลและรายงานผลนั้น ระบบเลือกใช้งานแอร์เทเบิล (Airtable) เป็นกลไกหลักในการจัดเก็บประวัติการวิเคราะห์และแจ้งเตือนสถานะความปลอดภัยอย่างเป็นระบบ โดยเหตุผลสำคัญในการเลือกใช้เครื่องมือนี้คือความสามารถในการจัดการฐานข้อมูลที่มีความยืดหยุ่นสูง รองรับการจัดเก็บชุดข้อมูลที่มีโครงสร้างซับซ้อนและไฟล์ข้อมูลที่ผ่านการประมวลผลแล้วได้อย่างมีระเบียบ นอกจากนี้แอร์เทเบิลยังมีส่วนต่อประสานโปรแกรมประยุกต์ที่เสถียรซึ่งช่วยให้เวิร์กโฟลว์อัตโนมัติสามารถบันทึกสถานะ

การวิเคราะห์แต่ละขั้นตอนได้แบบเรียลไทม์ ส่งผลให้ผู้ดูแลระบบสามารถตรวจสอบย้อนหลังและติดตามความคืบหน้าของการวิเคราะห์ภัยคุกคามได้อย่างแม่นยำและรวดเร็ว ในส่วนของโครงสร้างสภาพแวดล้อมจำลองระบบเลือกใช้ไมโครซอฟท์อาซัวร์ (Microsoft Azure) เพื่อสร้างเครื่องจำลองเสมือนที่เป็นพื้นที่ปลอดภัยสำหรับการทดสอบไฟล์ โดยอาศัยความเสถียรของอินเทอร์เฟซการเขียนโปรแกรมแอปพลิเคชันแบบเรสต์ (RESTful API) ในการบริหารจัดการวงจรชีวิตของเครื่องจำลองอัตโนมัติ

ท้ายที่สุดในส่วนของอินเทอร์เฟซผู้ใช้งาน พัฒนาขึ้นด้วยเฟรมเวิร์กสตรีมลิต (Streamlit) เพื่อเป็นเครื่องมือหลักในการปฏิสัมพันธ์ระหว่างระบบและผู้ใช้งาน โดยรองรับการนำส่งไฟล์เพื่อตรวจสอบและการแสดงรายงานผลประเมินความเสี่ยงที่ดึงข้อมูลโดยตรงจากฐานข้อมูลส่วนหลัง ซึ่งช่วยให้กระบวนการแสดงผลข้อมูลทางเทคนิคและสรุปพฤติกรรมมัลแวร์มีความต่อเนื่องและสอดคล้องกับลำดับงานที่กำหนดไว้

ตารางที่ 3.1: สรุปเครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนาจำแนกตามส่วนงาน

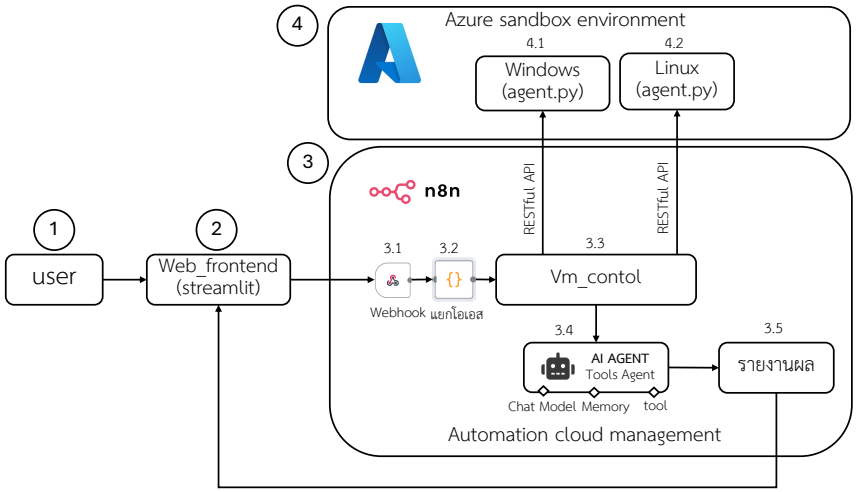
ส่วนงาน	**เครื่องมือ**	**บทบาทในระบบ**
ส่วนผู้ใช้งาน	Streamlit	อินเทอร์เฟซสำหรับการนำเข้าข้อมูลและติดตามรายงานผลประเมินความเสี่ยง
ส่วนบริการบนคลาวด์	n8n Orchestrator	ควบคุมลำดับการส่งไฟล์และการเชื่อมต่อปัญญาประดิษฐ์ (Self-hosted)
	In-Memory Vector Store	ฐานความรู้พฤติกรรมมัลแวร์สำหรับเพิ่มความแม่นยำให้ปัญญาประดิษฐ์
	Google Gemini LLM	วิเคราะห์พฤติกรรมเชิงลึกด้วยหน้าต่างบริบทขนาดใหญ่
	Airtable	ระบบจัดการฐานข้อมูลประวัติการวิเคราะห์ที่มีความยืดหยุ่นสูง
ส่วนระบบจำลอง	Python Flask API	ตัวแทนควบคุม (Agent) สำหรับสั่งการและส่งไฟล์มัลแวร์
	Bash Script	ควบคุมวงจรชีวิตและฟื้นฟูสภาพแวดล้อมผ่าน Azure API
	Microsoft Azure VM	สภาพแวดล้อมจำลองแบบปิดสำหรับวิเคราะห์พฤติกรรมไฟล์
	TShark & Strace	บันทึกกิจกรรมเครือข่ายและการทำงานของระบบปฏิบัติการ

3.2 สถาปัตยกรรมและแนวทางการทำงานของระบบ

ผู้วิจัยได้ออกแบบสถาปัตยกรรมของระบบตรวจจับและวิเคราะห์มัลแวร์โดยมีแนวคิดหลักในการพัฒนาให้ระบบสามารถทำงานได้อย่างอัตโนมัติ มีความสามารถในการวิเคราะห์พฤติกรรมของไฟล์ต้องสงสัย และมีความมั่นคงปลอดภัยต่อโครงสร้างพื้นฐานของระบบเครือข่ายที่ใช้งานจริง โดยการออกแบบสถา-

ปัตยกรรมของระบบมุ่งเน้นการแยกส่วนการทำงานออกจากกันอย่างชัดเจนเพื่อเพิ่มประสิทธิภาพในการจัดการข้อมูล ลดความเสี่ยงจากการแพร่กระจายของมัลแวร์ และเพิ่มความสามารถในการขยายระบบในอนาคว

รายละเอียดขององค์ประกอบหลักภายในระบบและความสัมพันธ์ระหว่างส่วนประกอบต่าง ๆ แสดงดังใน **รูปที่** 3.1 ซึ่งเป็นแผนภาพสถาปัตยกรรมของระบบที่แสดงการเชื่อมโยงระหว่างผู้ใช้งาน ระบบส่วนติดต่อผู้ใช้ ระบบควบคุมกลาง ระบบวิเคราะห์ด้วยปัญญาประดิษฐ์ และสภาพแวดล้อมจำลองสำหรับการทดสอบมัลแวร์



รูปที่ 3.1: แผนภาพสถาปัตยกรรมของระบบ

จากแผนภาพสถาปัตยกรรมของระบบใน **รูปที่** 3.1 สามารถจำแนกองค์ประกอบสำคัญที่ทำงานประสานกันออกเป็น 4 ส่วนหลัก เพื่อให้ครอบคลุมกระบวนการวิเคราะห์มัลแวร์แบบอัตโนมัติอย่างครบวงจร ดังนี้

3.2.1 ส่วนของผู้ใช้งาน

ส่วนของผู้ใช้งานทำหน้าที่เป็นจุดเริ่มต้นสำคัญของกระบวนการวิจัยและวิเคราะห์พฤติกรรมมัลแวร์ โดยสอดคล้องกับรายละเอียดในสัญลักษณ์ ① ใน**รูปที่** 3.1 ซึ่งผู้ใช้งานสามารถเข้าถึงระบบผ่านส่วนต่อประสานผู้ใช้งานที่จัดเตรียมไว้เพื่อส่งข้อมูลหรือไฟล์ที่ต้องการตรวจสอบเข้าสู่ระบบอย่างเป็นขั้นตอน นอกจากนี้ การทำงานในส่วนดังกล่าวยังเป็นกลไกหลักในการกระตุ้นให้ระบบเริ่มต้นกระบวนการตรวจสอบและวิเคราะห์ภัยคุกคามในรูปแบบอัตโนมัติ ช่วยลดภาระในการจัดการข้อมูลด้วยตนเองและเพิ่มความต่อเนื่องในการเชื่อมโยงข้อมูลไปยังส่วนการประมวลผลลำดับถัดไป

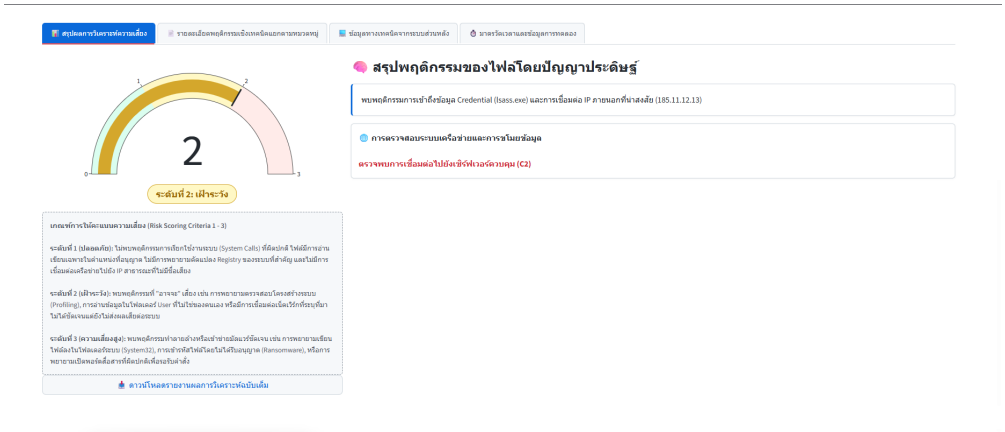
3.2.2 ส่วนติดต่อผู้ใช้งานหลัก

ส่วนติดต่อผู้ใช้งานหลักได้รับการพัฒนาขึ้นโดยอาศัยเฟรมเวิร์กสตรีมลิต (Streamlit) เพื่อเป็นเครื่องมือหลักในการปฏิสัมพันธ์ระหว่างระบบและผู้ใช้งาน ซึ่งสอดคล้องกับรายละเอียดในส่วนของสัญลักษณ์

② ในรูปที่ 3.1 โดยคุณสมบัติเด่นของเฟรมเวิร์กดังกล่าวคือความสามารถในการสร้างส่วนต่อประสานผู้ใช้งานที่ซับซ้อนด้วยภาษาไพทอนเพียงอย่างเดียว ส่งผลให้กระบวนการพัฒนาเว็บแอปพลิเคชันสำหรับงานวิทยาศาสตร์ข้อมูลและการวิเคราะห์มัลแวร์มีความรวดเร็วและมีประสิทธิภาพสูง นอกจากนี้ ระบบดังกล่าวยังได้รับการบริหารจัดการผ่านเทคโนโลยีคอนเทนเนอร์ด้วยด็อกเกอร์คอมโพส (Docker Compose) เพื่ออำนวยความสะดวกในการติดตั้งและทดสอบระบบให้มีความคล่องตัวสูง โดยส่วนติดต่อผู้ใช้งานนี้ทำหน้าที่สำคัญในการรับข้อมูลหรือไฟล์ตัวอย่างเพื่อส่งต่อไปยังกระบวนการวิเคราะห์ พร้อมทั้งรับหน้าที่แสดงผลรายงานการวิเคราะห์พฤติกรรมมัลแวร์ที่ได้รับจากระบบส่วนหลังในลักษณะการรายงานผลทันที ซึ่งการทำงานในรูปแบบที่ตอบสนองต่อการเปลี่ยนแปลงข้อมูลแบบเรียลไทม์ช่วยให้ผู้ใช้งานสามารถติดตามสถานะการตรวจสอบและประเมินผลลัพธ์ได้อย่างต่อเนื่องและแม่นยำ ดังรายละเอียดการออกแบบที่ระบุไว้ในหัวข้อ 3.2.2.1

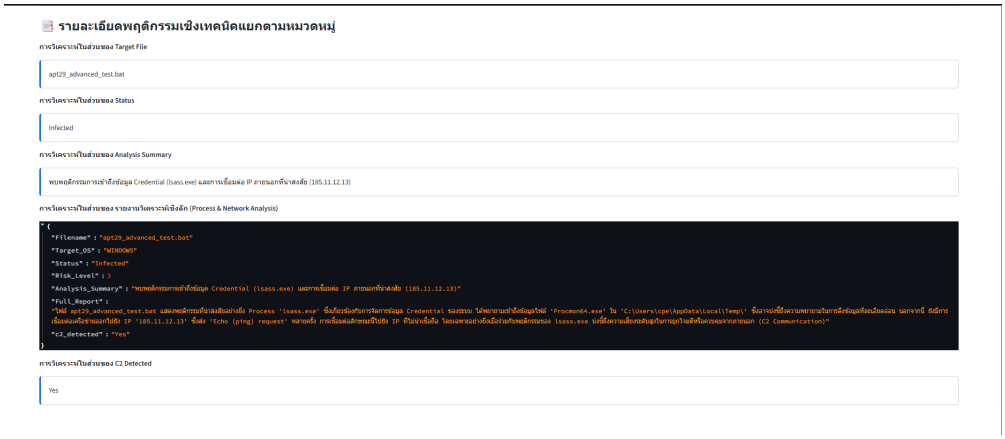
3.2.2.1 การออกแบบส่วนแสดงผลลัพธ์การวิเคราะห์

การแสดงผลลัพธ์การวิเคราะห์พฤติกรรมมัลแวร์ได้รับการออกแบบให้มีความชัดเจนและครอบคลุมทั้งข้อมูลสรุปในระดับบริหารและข้อมูลเชิงเทคนิคเชิงลึกสำหรับนักวิเคราะห์ความปลอดภัย โดยส่วนติดต่อผู้ใช้งานหลักจะแบ่งเนื้อหาออกเป็นส่วนต่าง ๆ เพื่อให้ง่ายต่อการทำความเข้าใจ ดังรายละเอียดที่ปรากฏใน **รูปที่** 3.2 จนถึง **รูปที่** 3.5



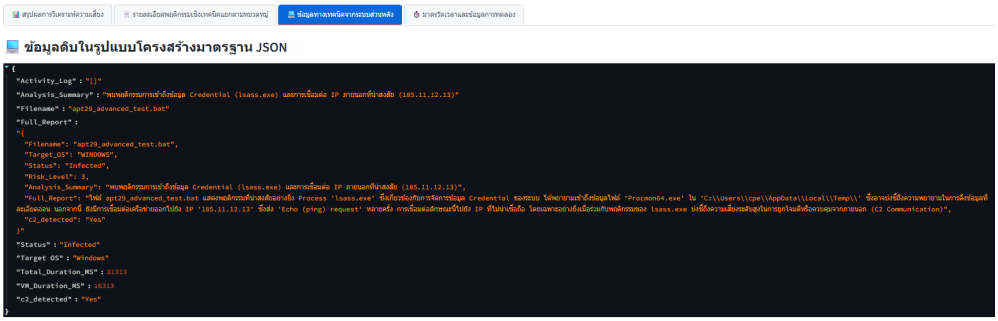
รูปที่ 3.2: ส่วนสรุปผลการวิเคราะห์ความเสี่ยงและมาตรวัดระดับความรุนแรง

ในส่วนแรกที่แสดงใน **รูปที่** 3.2 คือมาตรวัดระดับความความเสี่ยง (Risk Gauge) ที่จำแนกความรุนแรงออกเป็น 3 ระดับ ประกอบด้วยระดับที่ 1 (ปลอดภัย) ระดับที่ 2 (เฝ้าระวัง) และระดับที่ 3 (ความเสี่ยงสูง) โดยระบบจะแสดงสถานะปัจจุบันของไฟล์ที่ผ่านการตรวจสอบพร้อมคำอธิบายเกณฑ์การให้คะแนนความเสี่ยง (Risk Scoring Criteria) เพื่อให้ผู้ใช้งานทราบถึงสาเหตุเบื้องต้นของการจำแนกประเภทความปลอดภัย



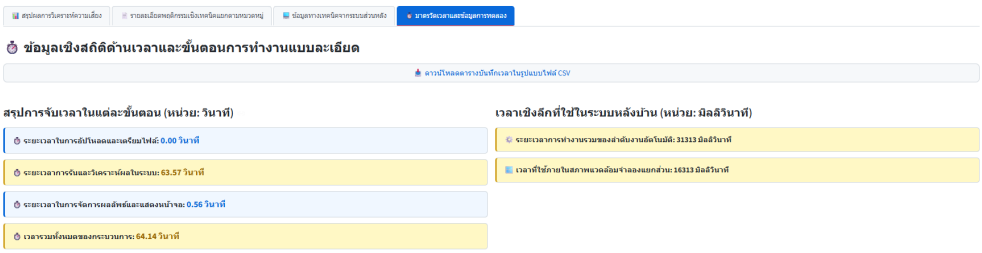
รูปที่ 3.3: รายละเอียดพฤติกรรมเชิงเทคนิคแยกตามหมวดหมู่และการวิเคราะห์โดยปัญญาประดิษฐ์

ถัดมาในส่วนของรายละเอียดเชิงลึกดังที่ปรากฏใน **รูปที่** 3.3 ระบบจะแสดงข้อมูลจำเพาะของไฟล์เป้าหมาย สถานะการตรวจพบ (Infected/Clean) และบทสรุปการวิเคราะห์ (Analysis Summary) ที่ประมวลผลโดยปัญญาประดิษฐ์ นอกจากนี้ยังมีการแสดงรายงานการวิเคราะห์เชิงลึกในรูปแบบโครงสร้าง JSON ซึ่งระบุพฤติกรรมการทำงานของโพรเซสและเครือข่ายที่มีนัยสำคัญ เช่น ความพยายามในการเข้าถึงข้อมูลสำคัญ (Credential Access) หรือการเชื่อมต่อกับเซิร์ฟเวอร์ควบคุมและสั่งการ (C2 Communication)



รูปที่ 3.4: การแสดงข้อมูลดิบในรูปแบบโครงสร้างมาตรฐาน JSON จากระบบส่วนหลัง

สำหรับในส่วนที่สามดังแผนภาพใน **รูปที่** 3.4 คือการแสดงผลข้อมูลดิบ (Raw Data) ที่ได้รับจากระบบส่วนหลังในรูปแบบ JSON ซึ่งข้อมูลชุดนี้เป็นผลลัพธ์ที่รวบรวมมาจากเครื่องจำลองและผ่านกระบวนการสกัดลักษณะเด่นมาแล้ว ข้อมูลประกอบด้วยประวัติการทำงาน (Activity Log) ระยะเวลาการประมวลผลในแต่ละภาคส่วน และรายละเอียดทางเทคนิคทั้งหมดที่ปัญญาประดิษฐ์ใช้เป็นบริบทในการตัดสินใจ ซึ่งช่วยให้นักวิเคราะห์สามารถตรวจสอบย้อนหลังและยืนยันความถูกต้องของผลการวิเคราะห์ได้

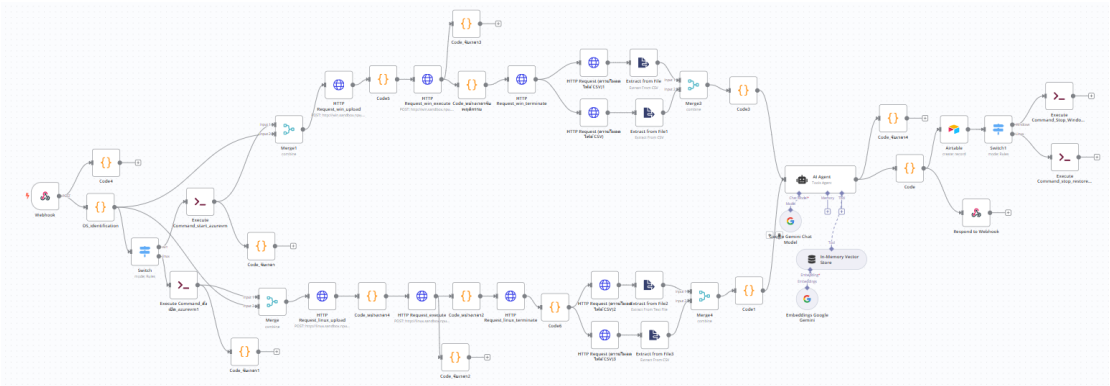


รูปที่ 3.5: ข้อมูลเชิงสถิติด้านเวลาและขั้นตอนการทำงานแบบละเอียด

ท้ายที่สุดในส่วนของการวัดประสิทธิภาพการทำงานดังแสดงใน **รูปที่** 3.5 ระบบจะแสดงมาตรวัดเวลา (Metrics) ในทุกขั้นตอนของกระบวนการทำงานอัตโนมัติ ตั้งแต่ระยะเวลาในการอัปโหลดไฟล์ การรันและวิเคราะห์ผลลัพธ์ ไปจนถึงระยะเวลาที่ใช้ภายในสภาพแวดล้อมจำลอง โดยข้อมูลเหล่านี้ช่วยให้ผู้ดูแลระบบสามารถประเมินประสิทธิภาพของความหน่วงในระบบ (Latency) และความเร็วในการตอบสนองต่อภัยคุกคามได้อย่างแม่นยำ

3.2.3 ระบบควบคุมลำดับงานอัตโนมัติ n8n

แพลตฟอร์ม n8n ทำหน้าที่เป็นแกนกลางสำคัญในการควบคุมลำดับงานและประสานการทำงานระหว่างส่วนประกอบต่าง ๆ ของระบบ ซึ่งสอดคล้องกับรายละเอียดในสัญลักษณ์ ③ ใน**รูปที่** 3.1 โดยมีรายละเอียดบทบาทหน้าที่สำคัญที่ส่งผลให้การวิเคราะห์มัลแวร์เป็นไปอย่างอัตโนมัติและแม่นยำ ซึ่งโครงสร้างเวิร์กโฟลว์การทำงานสามารถแบ่งออกเป็น 5 ส่วนหลัก ดังแสดงใน **รูปที่** 3.6



รูปที่ 3.6: ภาพรวมการกำหนดลำดับขั้นตอนการทำงานอัตโนมัติบนแพลตฟอร์ม n8n

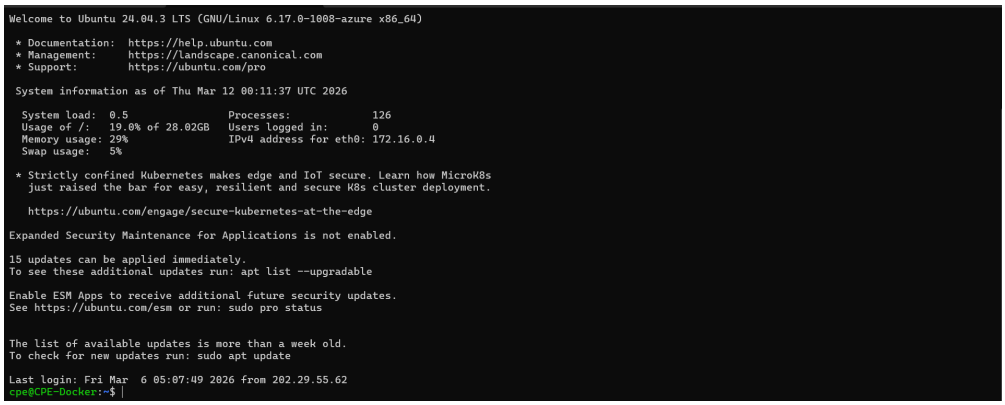
จากแผนภาพใน **รูปที่** 3.6 สามารถอธิบายรายละเอียดการทำงานแต่ละส่วนได้ดังนี้ ในส่วนที่ 3.1 เกี่ยวกับจุดรับส่งสัญญาณหรือเว็บฮุค นั้น ทำหน้าที่หลักในการเปิดช่องทางสื่อสารเพื่อรับข้อมูลคำขอและไฟล์มัลแวร์จากส่วนต่อประสานผู้ใช้งานหลักเพื่อเริ่มกระบวนการทำงานในเวิร์กโฟลว์ทันที ถัดมาในส่วนที่ 3.2 คือโหนดการคัดแยกประเภทระบบปฏิบัติการ ซึ่งอาศัยการเขียนรหัสด้วยภาษาจาวาสคริปต์เพื่อตรวจสอบค่า

เมจิกไบต์ของไฟล์ ทำให้ระบบสามารถจำแนกสถาปัตยกรรมเป้าหมายได้อย่างถูกต้องไม่ว่าจะเป็นระบบปฏิบัติการวินโดวส์หรือลินุกซ์ สำหรับในส่วนที่ 3.3 เกี่ยวกับการควบคุมเครื่องจำลอง นั้น ระบบได้บูรณาการร่วมกับส่วนการจัดการระบบคลาวด์อัตโนมัติ เพื่อสั่งการผ่านส่วนต่อประสานโปรแกรมประยุกต์แบบเรสต์ (REST) ในการบริหารจัดการวงจรชีวิตของเครื่องจำลองเสมือนบนระบบคลาวด์อาซัวร์ (Azure) อย่างเป็นระบบ

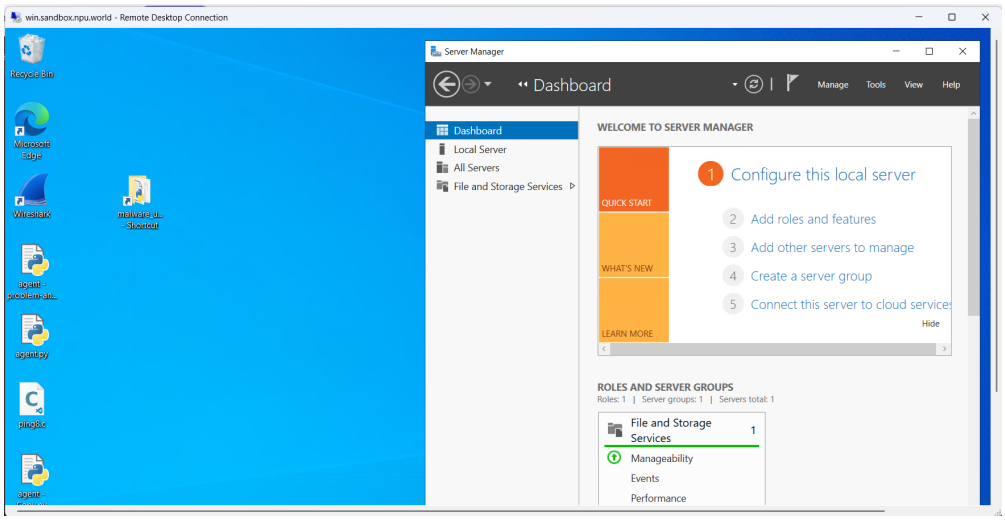
นอกจากนี้ในส่วนที่ 3.4 ซึ่งเป็นตัวแทนวิเคราะห์ด้วยปัญญาประดิษฐ์ ถือเป็นองค์ประกอบสำคัญที่ผสานการทำงานระหว่างแบบจำลองการสนทนา ร่วมกับเครื่องมือช่วยวิเคราะห์ และระบบความจำ เพื่อให้ปัญญาประดิษฐ์สามารถสวมบทบาทเป็นผู้เชี่ยวชาญด้านการวิเคราะห์มัลแวร์ระดับสูงได้อย่างสมบูรณ์ โดยมุ่งเน้นการตรวจสอบปูมพฤติกรรมเชิงลึกทั้งในส่วนของการเข้าถึงไฟล์ระบบ การเปลี่ยนแปลงค่าเรจิสทรี และพฤติกรรมการเชื่อมต่อเครือข่าย พร้อมทั้งมีการกำหนดเงื่อนไขพิเศษเพื่อตรวจจับสถาปัตยกรรมเครือข่ายของผู้โจมตี เช่น การตรวจพบเลขที่อยู่ไอพีภายนอกที่ผิดปกติซึ่งจะถูกระบุเป็นสถานะการสั่งการและควบคุมในทันที ท้ายที่สุดในส่วนที่ 3.5 คือการรายงานผลซึ่งทำหน้าที่ประมวลผลสรุปจากการวิเคราะห์ทั้งหมดเพื่อจัดส่งข้อมูลตอบกลับไปยังส่วนแสดงผลบนหน้าเว็บแอปพลิเคชัน ส่งผลให้ผู้ใช้งานได้รับทราบข้อมูลเชิงลึกและระดับความรุนแรงของภัยคุกคามได้อย่างรวดเร็ว

3.2.4 สภาพแวดล้อมจำลองอาซัวร์

สภาพแวดล้อมจำลองอาซัวร์ทำหน้าที่เป็นพื้นที่แยกอิสระที่มีความมั่นคงปลอดภัยสูงสำหรับการประมวลผลมัลแวร์เพื่อเฝ้าสังเกตพฤติกรรมจริงบนระบบคลาวด์อาซัวร์ ซึ่งสอดคล้องกับรายละเอียดในสัญลักษณ์ ④ ใน**รูปที่** 3.1 โดยโครงสร้างภายในประกอบด้วยเครื่องจำลองเสมือนที่ปรับแต่งมาเพื่อการวิเคราะห์ภัยคุกคามโดยเฉพาะ พร้อมทั้งรองรับการทดสอบบนระบบปฏิบัติการหลักสองประเภทที่มีกลไกการทำงานแตกต่างกันอย่างชัดเจน ดังแสดงตัวอย่างหน้าจอของสภาพแวดล้อมทั้งสองประเภทใน **รูปที่** 3.7 และ **รูปที่** 3.8



รูปที่ 3.7: สภาพแวดล้อมจำลองแบบบรรทัดคำสั่งบนระบบปฏิบัติการลินุกซ์สำหรับทดสอบมัลแวร์



รูปที่ 3.8: สภาพแวดล้อมจำลองแบบส่วนต่อประสานกราฟิกบนระบบปฏิบัติการวินโดวส์ที่แฝงเครื่องมือดักจับพฤติกรรมมัลแวร์

จากภาพระบบทั้งสองรูปแบบ ในส่วนที่ 4.1 เกี่ยวกับระบบปฏิบัติการวินโดวส์นั้น ดังปรากฏใน **รูปที่** 3.8 แสดงให้เห็นถึงสภาพแวดล้อมแบบส่วนต่อประสานกราฟิกของระบบปฏิบัติการวินโดวส์เซิร์ฟเวอร์ที่ได้รับการติดตั้งและตั้งค่าเครื่องมือพื้นฐานสำหรับการสืบสวนและรวบรวมข้อมูล โดยอาศัยการทำงานของโปรแกรมตัวแทนควบคุม (agent.py) ที่เขียนด้วยภาษาไพทอนเพื่อเป็นสื่อกลางรับคำสั่งจากศูนย์กลาง ควบคู่ไปกับเครื่องมือดักจับแพ็กเก็ตเครือข่ายไวร์ชาร์ก (Wireshark) ซึ่งปรากฏไอคอนอยู่บนหน้าจอเดสก์ท็อป โดยสภาพแวดล้อมนี้ถูกเจาะจงออกแบบมาเพื่อหลอกล่อให้มัลแวร์ประเภทไฟล์ปฏิบัติการแสดงพฤติกรรมออกมาอย่างเต็มที่ จากนั้นโปรแกรมตัวแทนจะทำหน้าที่บริหารจัดการกระบวนการวิเคราะห์ไฟล์และบันทึกพฤติกรรมเชิงลึกสำหรับระบบปฏิบัติการวินโดวส์โดยเฉพาะ

ในทำนองเดียวกันกับส่วนที่ 4.2 ซึ่งเป็นระบบปฏิบัติการลินุกซ์ดังที่แสดงหน้าต่างการทำงานแบบบรรทัดคำสั่งของอูบุนตูใน **รูปที่** 3.7 ระบบใช้โปรแกรมตัวแทนควบคุม (agent.py) ในรูปแบบเดียวกันเพื่อทำหน้าที่สั่งการและรวบรวมพฤติกรรมที่เกิดขึ้นจากการรันไฟล์อันตราย ซึ่งมุ่งเน้นการตรวจสอบสคริปต์ก่อกวนหรือมัลแวร์ประเภทที่เข้ากันได้กับลินุกซ์ โดยระบบจะทำการบันทึกและตรวจสอบการเรียกใช้งานระบบและการเชื่อมต่อผ่านเครือข่ายเฉพาะบนสถาปัตยกรรมลินุกซ์ ซึ่งระบบจำลองของเครื่องทั้งสองรูปแบบล้วนถูกแยกวงเครือข่ายไว้อย่างรัดกุมเพื่อรักษาความมั่นคงปลอดภัยขั้นสูงสุด

3.2.5 การจัดการความมั่นคงปลอดภัยของพื้นที่ทดสอบ (Sandbox Isolation)

หัวใจสำคัญของการออกแบบสภาพแวดล้อมจำลองคือการควบคุมความมั่นคงปลอดภัยเพื่อป้องกันไม่ให้มัลแวร์แพร่กระจายออกจากพื้นที่ทดสอบไปยังโครงสร้างพื้นฐานภายนอก ผู้วิจัยจึงได้กำหนดสถาปัตยกรรมเครือข่าย (Network Topology) แบบแยกส่วนหน้าที่อย่างชัดเจน โดยแบ่งระดับการสื่อสารออกเป็นสองระนาบหลัก คือระนาบควบคุมซึ่งทำหน้าที่บริหารจัดการระบบและรับส่งคำสั่งวิเคราะห์ และระนาบ

ข้อมูลที่เป็นพื้นที่สำหรับประมวลผลไฟล์มัลแวร์โดยเฉพาะ ดังรายละเอียดที่จะกล่าวถึงในกระบวนการฟื้นฟูระบบของหัวข้อ 3.3

ในระนาบควบคุมนั้น ระบบใช้ช่องทางการสื่อสารที่ผ่านการเข้ารหัสและมีการกำหนดสิทธิ์การเข้าถึงอย่างเคร่งครัดระหว่างศูนย์กลางจัดการกระบวนการทำงานอัตโนมัติและโปรแกรมตัวแทนภายในเครื่องจำลอง เพื่อให้มั่นใจว่าคำสั่งในการเริ่มหรือยุติการวิเคราะห์จะไม่ถูกรบกวน ในขณะเดียวกัน ระนาบข้อมูลจะถูกจำกัดให้อยู่ภายในเครือข่ายเสมือนแบบปิดบนระบบคลาวด์อาซัวร์ โดยมีการใช้กลุ่มความมั่นคงปลอดภัยเครือข่าย (Network Security Group: NSG) ในการคัดกรองการจราจรทางเครือข่ายทั้งขาเข้าและขาออกอย่างละเอียด

สำหรับการจำกัดสิทธิ์การเข้าถึงอินเทอร์เน็ตของพื้นที่ทดสอบ ระบบได้กำหนดนโยบายการเชื่อมต่อแบบจำกัดพอร์ต (Port Restriction) โดยอนุญาตเฉพาะโพรโทคอลที่จำเป็นต่อการวิเคราะห์พฤติกรรมและการส่งออกปูมเหตุการณ์เท่านั้น ในกรณีที่มัลแวร์มีความพยายามสื่อสารออกสู่ภายนอกเพื่อเชื่อมต่อกับเซิร์ฟเวอร์ควบคุมและสั่งการ ระบบจะทำการบันทึกพฤติกรรมผ่านโหนดเฝ้าระวังโดยไม่ยอมให้มีการส่งข้อมูลที่ปนเปื้อนหรือการแพร่กระจายตัวไปยังระบบเครือข่ายภายนอกอาซัวร์ได้โดยเด็ดขาด นอกจากนี้ การใช้กลไกฟื้นฟูสภาพแวดล้อมด้วยการทำลายหน่วยจัดเก็บข้อมูลที่ปนเปื้อนทิ้งทันทีตามวัฏจักรความปลอดภัยที่ระบุไว้ใน 3.15 ยังเป็นอีกหนึ่งมาตรการสำคัญที่ช่วยรักษาความบริสุทธิ์ของพื้นที่ทดสอบ

3.2.6 การสกัดลักษณะเด่นและการลดรูปข้อมูล (Feature Extraction & Data Reduction)

เพื่อให้กระบวนการวิเคราะห์พฤติกรรมมัลแวร์โดยปัญญาประดิษฐ์เป็นไปอย่างมีประสิทธิภาพและประหยัดทรัพยากรหน้าต่างบริบท (Context Window) ผู้วิจัยจึงได้พัฒนากลไกการสกัดลักษณะเด่นและการลดรูปข้อมูลก่อนส่งเข้าสู่กระบวนการประมวลผลหลัก โดยมี Logic ในการคัดกรองข้อมูลปูมเหตุการณ์ (Log Filtering) และกระบวนการแปลงข้อมูลดังนี้

ในส่วนของข้อมูลกิจกรรมระดับระบบปฏิบัติการที่ได้รับจากเครื่องมือเฝ้าสังเกตการณ์โพรเซส (Process Monitor) นั้น ระบบจะดำเนินการกรองข้อมูลดิบที่มีความซ้ำซ้อนและข้อมูลขยะที่เกิดจากกิจกรรมพื้นฐานของระบบปฏิบัติการออก โดยมุ่งเน้นการสกัดเฉพาะลักษณะเด่น (Features) ที่มีความสำคัญต่อความมั่นคงปลอดภัย ซึ่งรวมถึงการเรียกใช้งานระบบ (System Calls) ที่เกี่ยวข้องกับการสร้างโพรเซสใหม่ การแก้ไขค่ารีจิสทรีในจุดสำคัญ และการเข้าถึงไฟล์ระบบที่มีนัยสำคัญต่อการทำงานของมัลแวร์

ในทำนองเดียวกันกับข้อมูลการจราจรทางเครือข่ายที่ได้จากโปรแกรมทีชาร์ก (TShark) ระบบจะดำเนินการสกัดเอาเฉพาะฟิลด์ข้อมูลที่จำเป็น เช่น หมายเลขลำดับเฟรม เลขที่อยู่ไอพีต้นทางและปลายทาง รวมถึงรายละเอียดข้อมูลส่วนหัว (Info Column) เพื่อใช้ในการวิเคราะห์พฤติกรรมการเชื่อมต่อแบบซ็อกเก็ต (Socket Connection) และการสื่อสารกับเซิร์ฟเวอร์ภายนอก โดยตัดข้อมูลแพ็กเก็ตที่มีความซ้ำซ้อนหรือการสื่อสารภายในเครือข่ายจำลองที่ไม่มีความเกี่ยวข้องกับพฤติกรรมอันตรายออกไป

สำหรับกระบวนการแปลงข้อมูลจากข้อมูลดิบ (Raw Logs) ให้อยู่ในรูปแบบที่พร้อมใช้งานนั้น โปรแกรมตัวแทนภายในเครื่องจำลองจะดำเนินการแปลงไฟล์บันทึกพฤติกรรมซึ่งเดิมอยู่ในรูปแบบเฉพาะ เช่น ไฟล์นามสกุล .pcap สำหรับเครือข่าย หรือไฟล์นามสกุล .pml สำหรับกิจกรรมระบบ ให้กลายเป็นไฟล์

ข้อมูลที่มีโครงสร้างแบบตารางหรือซีเอสวี (CSV) ผ่านชุดคำสั่งอัตโนมัติ กระบวนการดังกล่าวเริ่มต้นจากการเปิดไฟล์บันทึกสถานะเดิมและใช้ส่วนขยายคำสั่งในการสกัดฟิลด์ข้อมูลที่กำหนดไว้ล่วงหน้าออกมาจัดเรียงใหม่ การแปลงข้อมูลเป็นรูปแบบซีเอสวีนี้มีประโยชน์สำคัญในการช่วยลดปริมาณข้อมูลขยะ (Data Noise) และทำให้ปัญญาประดิษฐ์สามารถทำความเข้าใจโครงสร้างพฤติกรรมได้ง่ายขึ้น ซึ่งส่งผลโดยตรงต่อความแม่นยำในการสรุปผลและการตอบสนองต่อภัยคุกคาม ดังความสัมพันธ์ที่ระบุไว้ในการค้นคืนข้อมูลของหัวข้อ 3.2.7

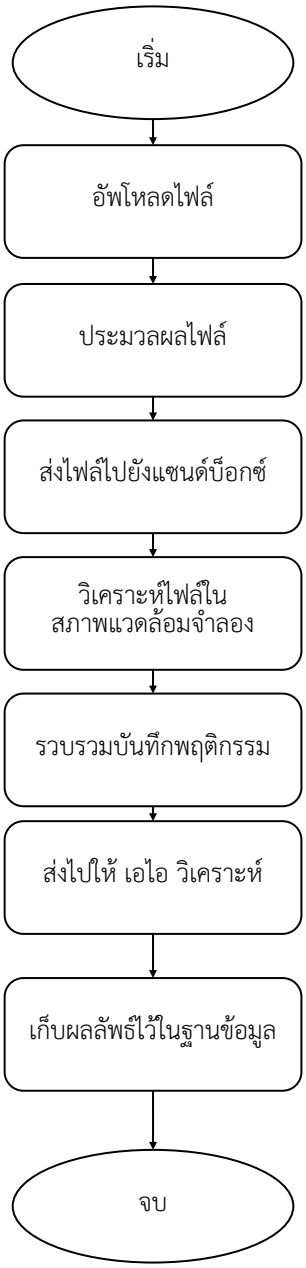
3.2.7 สถาปัตยกรรมการค้นคืนข้อมูลเสริม (RAG Architecture)

เพื่อให้ระบบวิเคราะห์มัลแวร์มีฐานความรู้ที่ทันสมัยและลดโอกาสการเกิดข้อมูลเท็จจากแบบจำลองภาษาขนาดใหญ่ ผู้วิจัยจึงประยุกต์ใช้สถาปัตยกรรมการค้นคืนข้อมูลเสริม (Retrieval-Augmented Generation) เข้ามาเป็นกลไกสนับสนุนการตัดสินใจเชิงเทคนิค โดยมีรายละเอียดการออกแบบองค์ประกอบสำคัญดังต่อไปนี้

ในส่วนของฐานข้อมูลเวกเตอร์และการจัดการคลังความรู้ นั้น ระบบเลือกใช้งานฐานข้อมูลเวกเตอร์ในรูปแบบหน่วยความจำเพื่อทำหน้าที่จัดเก็บข้อมูลทางเทคนิคในรูปแบบเวกเตอร์ ข้อมูลภายในคลังความรู้ดังกล่าวประกอบด้วยชุดข้อมูลพื้นฐานเชิงพฤติกรรมและข้อมูลความร่วมมือด้านภัยคุกคามที่สำคัญ โดยเฉพาะรายละเอียดเทคนิคการโจมตีจากกรอบแนวคิด MITRE ATT&CK ที่เกี่ยวข้องกับกลุ่มภัยคุกคาม APT29 ตลอดจนลายนิพจน์พฤติกรรมที่รวบรวมจากแหล่งข้อมูลมาตรฐาน เช่น MalwareBazaar เพื่อให้ปัญญาประดิษฐ์มีบริบทอ้างอิงที่แม่นยำในการเปรียบเทียบกับพฤติกรรมจริงที่ตรวจพบ

สำหรับขั้นตอนการค้นคืนข้อมูลเพื่อสนับสนุนการตัดสินใจของปัญญาประดิษฐ์นั้น เริ่มต้นจากโหนดประสานงานบนแพลตฟอร์ม n8n จะทำการแปลงข้อมูลปูมเหตุการณ์ที่ได้รับจากการทดสอบในเครื่องจำลองให้เป็นเวกเตอร์ข้อมูล จากนั้นระบบจะดำเนินการสืบค้นข้อมูลใน Malware Knowledge Base เพื่อดึงเอกสารหรือรายงานที่มีความหมายใกล้เคียงกับพฤติกรรมที่ตรวจพบมากที่สุด ข้อมูลที่สืบค้นได้จะถูกนำไปรวมเข้ากับคำสั่งหลัก (Prompt) เพื่อส่งมอบให้แบบจำลองภาษา Gemini ประมวลผลในขั้นตอนสุดท้าย

กระบวนการดังกล่าวส่งผลให้ปัญญาประดิษฐ์สามารถวิเคราะห์ภัยคุกคามโดยอิงจากหลักฐานทางเทคนิคที่มีอยู่จริงในคลังความรู้ แทนการใช้เพียงความรู้ภายในตัวแบบจำลองเพียงอย่างเดียว ซึ่งช่วยให้การระบุสถานะความเสี่ยงและการจัดหมวดหมู่ภัยคุกคามมีความน่าเชื่อถือและสอดคล้องกับมาตรฐานความมั่นคงปลอดภัยไซเบอร์ในระดับสากล



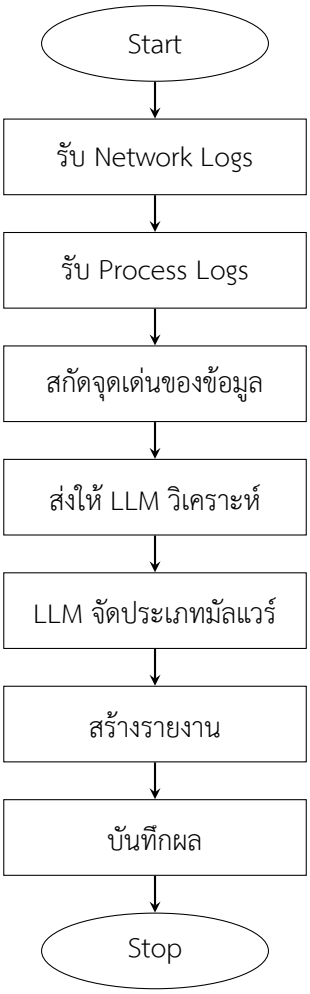
รูปที่ 3.9: ลำดับกระบวนการทำงานของระบบ

ในส่วนของลำดับขั้นตอนการทำงานของระบบใน **รูปที่** 3.9 แสดงให้เห็นสถาปัตยกรรมการทำงานเชิงปฏิสัมพันธ์ระหว่างองค์ประกอบหลักอย่างชัดเจน โดยกระบวนการเริ่มต้นจากการนำเข้าไฟล์ผ่านส่วนต่อประสานผู้ใช้งาน ซึ่งทำหน้าที่รับข้อมูลและนำส่งเข้าสู่เวิร์กโฟลว์การจัดการอัตโนมัติบนแพลตฟอร์ม n8n เพื่อเริ่มขั้นตอนการแยกแยะประเภทระบบปฏิบัติการและตรวจสอบความถูกต้องของข้อมูลเบื้องต้น ในขั้นตอนถัดมา ระบบควบคุมกลางจะสื่อสารกับบริการบนระบบคลาวด์อาซัวร์เพื่อจัดเตรียมสภาพแวดล้อม

จำลองให้มีความพร้อมและกำหนดค่าความปลอดภัยแบบแยกส่วนอย่างเข้มงวดสำหรับการทดสอบมัลแวร์โดยเฉพาะ

เมื่อชุดทดสอบเข้าสู่ระยะการทำงานจริง ระบบจะนำส่งไฟล์เป้าหมายให้แก่โปรแกรมตัวแทนภายในเครื่องจำลองเพื่อเริ่มกระบวนการปฏิบัติงานควบคู่ไปกับการเปิดใช้งานเครื่องมือเฝ้าสังเกตการณ์พฤติกรรมทั้งในมิติของการจราจรทางเครือข่ายและการเรียกใช้งานในระดับระบบปฏิบัติการผ่านเครื่องมือบันทึกเหตุการณ์ที่กำหนดไว้ ภายหลังสิ้นสุดระยะเวลาการทดสอบตามกรอบที่เหมาะสม ระบบจะรวบรวมและแปลงรูปแบบปูมเหตุการณ์ดิบให้เป็นข้อมูลที่มีโครงสร้าง ก่อนจะส่งต่อชุดข้อมูลเข้าสู่กลไกการวิเคราะห์พฤติกรรมเชิงลึกด้วยแบบจำลองภาษาขนาดใหญ่ Gemini ซึ่งอาศัยฐานความรู้จากสถาปัตยกรรมการค้นคืนข้อมูลเสริมเป็นบริบทอ้างอิงเพื่อประเมินความเสี่ยง ท้ายที่สุด ระบบจะสรุปรายงานพฤติกรรมภัยคุกคาม พร้อมทั้งจัดเก็บผลลัพธ์ลงในฐานข้อมูลกลางและอัปเดตสถานะความปลอดภัยบนหน้าจอส่วนติดต่อผู้ใช้งานหลักเพื่อให้การวิเคราะห์มัลแวร์เป็นไปอย่างครอบคลุมและมีประสิทธิภาพสูงสุด

กระบวนการวิเคราะห์โดยปัญญาประดิษฐ์แสดงรายละเอียดดังใน **รูปที่** 3.10



รูปที่ 3.10: กระบวนการวิเคราะห์ไฟล์ด้วยปัญญาประดิษฐ์

กระบวนการวิเคราะห์ไฟล์ที่แสดงใน **รูปที่** 3.10 สามารถจำแนกหน้าที่และการทำงานตามลำดับขั้นตอนได้ดังนี้:

การนำเข้าข้อมูลดิบ (Data Ingestion) ทำหน้าที่รับข้อมูลเข้าสองประเภทหลักคือ *Network Logs* ซึ่งบันทึกพฤติกรรมการเชื่อมต่อโครงข่าย และ *Process Logs* ที่บันทึกลำดับการทำงานของโปรแกรมในระบบ ข้อมูลดิบเหล่านี้จะถูกรวบรวมเพื่อเตรียมความพร้อมสำหรับการวิเคราะห์พฤติกรรมในระดับถัดไป

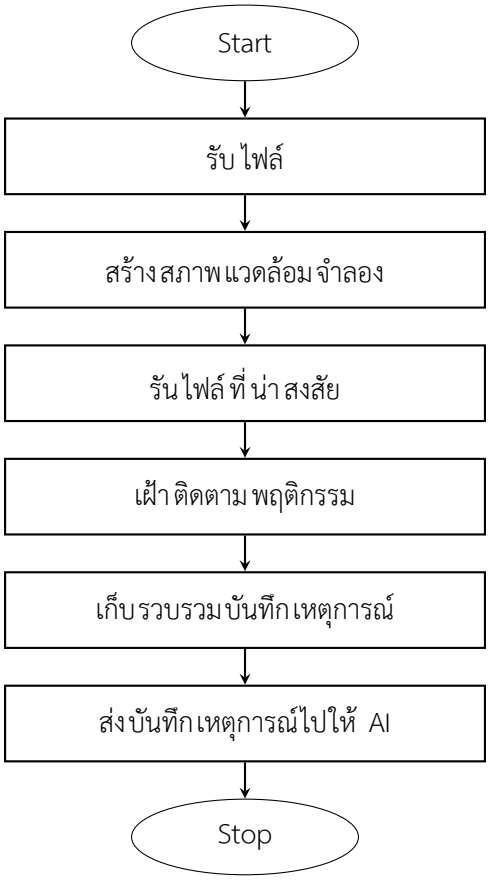
การสกัดจุดเด่นของข้อมูล เป็นขั้นตอนการแปลงข้อมูลดิบให้กลายเป็นเวกเตอร์ที่มีคุณลักษณะโดยการคัดเลือกเฉพาะพฤติกรรมที่สำคัญ เช่น การเรียกใช้ System Calls หรือการเข้าถึงหน่วยความจำที่ผิดปกติ เพื่อลดความซับซ้อนของข้อมูลและเพิ่มประสิทธิภาพในการประมวลผลของแบบจำลอง

การวิเคราะห์ด้วยแบบจำลองภาษาขนาดใหญ่ (LLM Analysis) ทำหน้าที่นำคุณลักษณะเด่นที่สกัดได้เข้าสู่กระบวนการตีความเชิงตรรกะโดย ตัวแบบภาษาขนาดใหญ่ ซึ่งแบบจำลองจะใช้ความสามารถในการทำความเข้าใจบริบท เพื่อเชื่อมโยงพฤติกรรมต่างๆ เข้ากับฐานข้อมูลพฤติกรรมมัลแวร์ที่รู้จัก

การจัดประเภทและการบันทึกผล เป็นขั้นตอนสุดท้ายที่ ตัวแบบภาษาขนาดใหญ่ จะทำการระบุประเภทของมัลแวร์จากนั้นระบบจะดำเนินการสร้างรายงานสรุปผลการวิเคราะห์ทางเทคนิค และทำการบันทึกข้อมูล ลงในฐานข้อมูลเพื่อใช้สำหรับการอ้างอิงหรือเป็นข้อมูลย้อนหลังในการเฝ้าระวังความปลอดภัย

ในขั้นตอนการประมวลผลหลัก ระบบจัดลำดับการทำงานผ่านทางโหนดประสานงานเพื่อส่งข้อมูลพฤติกรรมเข้าสู่กลไกการค้นคืนข้อมูลจากฐานข้อมูลเวกเตอร์ ข้อมูลที่สืบค้นได้จะถูกนำมาเปรียบเทียบกับฐานความรู้มาตรฐานด้านภัยคุกคามเพื่อสร้างบริบทอ้างอิงที่สอดคล้องกับพฤติกรรมจริง จากนั้นระบบรวบรวมข้อมูลทั้งหมดเข้าสู่โครงสร้างคำสั่งที่กำหนดบทบาทปัญญาประดิษฐ์เป็นผู้เชี่ยวชาญด้านการวิเคราะห์มัลแวร์ระดับสูง เพื่อให้แบบจำลอง Gemini ดำเนินการวิเคราะห์ความเชื่อมโยงของกิจกรรมต่าง ๆ และระบุเทคนิคการโจมตีตามกรอบแนวคิด MITRE ATT&CK อย่างเป็นระบบ ท้ายที่สุด ปัญญาประดิษฐ์จะทำการประเมินสถานะความเสี่ยงพร้อมกับระบุข้อเสนอแนะในการตอบโต้ภัยคุกคาม เพื่อนำไปสรุปผลและรายงานผลลัพธ์ผ่านส่วนต่อประสานผู้ใช้งานในลำดับถัดไป

หากไฟล์มีความเสี่ยงในระดับที่กำหนด ระบบจะส่งไฟล์เข้าสู่กระบวนการทดสอบในสภาพแวดล้อมจำลอง ดังแสดงใน **รูปที่** 3.11



รูปที่ 3.11: กระบวนการทดสอบไฟล์ในสภาพแวดล้อมจำลอง

กระบวนการวิเคราะห์ในรูปที่ 3.11 เริ่มต้นที่จุด Start เมื่อระบบ Endpoint ตรวจพบการนำเข้าไฟล์ ที่ไม่รู้จักหรือมีความเสี่ยง โดยข้อมูลจะถูกนำเข้าสู่ระบบผ่านโมดูลการรับไฟล์ (Receive File) เพื่อเตรียมพร้อมสำหรับการวิเคราะห์เชิงลึกโดยไม่ให้กระทบต่อทรัพยากรหลักของเครื่อง

ในลำดับต่อมา ระบบจะทำการสร้างแซนด์บ็อกซ์ ซึ่งเป็นเทคโนโลยีเครื่องเสมือน เพื่อแยกส่วนสภาพแวดล้อมการทำงานออกจากระบบปฏิบัติการจริง โดยกระบวนการนี้จะกำหนดขอบเขตของทรัพยากร เช่น CPU, Memory และ Network Interface ที่ไฟล์สามารถเข้าถึงได้

เมื่อสภาพแวดล้อมพร้อม ไฟล์จะถูกรันโปรแกรมภายใต้การควบคุมอย่างเข้มงวด ในขณะที่ไฟล์ทำงาน ระบบติดตามตรวจสอบจะทำหน้าที่ดักจับข้อมูลพฤติกรรม เช่น การเรียกใช้ส่วนต่อประสานการเขียนโปรแกรมประยุกต์ ของระบบ การพยายามแก้ไขไฟล์ในโฟลเดอร์สำคัญ หรือการสร้างการเชื่อมต่อเครือข่ายที่ผิดปกติ

ข้อมูลพฤติกรรมทั้งหมดจะถูกจัดเก็บในลักษณะของบันทึกเหตุการณ์ ซึ่งเป็นข้อมูลดิบที่มีโครงสร้างชัดเจน ข้อมูลเหล่านี้จะถูกส่งต่อไปยังตัวแบบภาษาขนาดใหญ่เพื่อใช้ในการจำแนกประเภท และประเมินคะแนนความเสี่ยง ก่อนที่จะสิ้นสุดกระบวนการที่จุด Stop

ในระหว่างการจำลองเหตุการณ์ ระบบบันทึกพฤติกรรมจะรวบรวมข้อมูลการทำงานเชิงลึก เช่น การเชื่อมต่อเครือข่ายภายนอก การแก้ไขค่ารีจิสทรี และการเข้าถึงไฟล์ระบบที่มีนัยสำคัญ เมื่อสิ้นสุดระยะเวลาที่กำหนด ระบบจะยุติการทำงานของโพรเซสและเข้าสู่ขั้นตอนการโอนย้ายข้อมูลบันทึกพฤติกรรมดิบเพื่อนำมาผ่านกระบวนการสกัดลักษณะเด่นตามกลไกในหัวข้อ 3.2.6 กระบวนการดังกล่าวส่งผลให้ระบบสามารถแปลงข้อมูลพฤติกรรมที่ซับซ้อนให้อยู่ในรูปแบบโครงสร้างที่พร้อมสำหรับการวิเคราะห์ด้วยปัญญาประดิษฐ์ในลำดับถัดไป เพื่อให้มั่นใจว่าการตรวจจับภัยคุกคามมีความครอบคลุมและอ้างอิงจากหลักฐานเชิงพฤติกรรมที่เกิดขึ้นจริงภายในสภาพแวดล้อมที่มีการควบคุมอย่างเข้มงวด

ผลลัพธ์จากการวิเคราะห์ทั้งในส่วนของปัญญาประดิษฐ์และการทดสอบในสภาพแวดล้อมจำลองจะถูกจัดเก็บไว้ในฐานข้อมูลบนคลาวด์เพื่อใช้สำหรับการตรวจสอบย้อนหลังและการวิเคราะห์ข้อมูลเชิงสถิติในอนาคต โดยโครงสร้างทั้งหมดแสดงให้เห็นถึงความสอดคล้องของระบบวิเคราะห์มัลแวร์แบบอัตโนมัติที่ผสานเทคโนโลยีคลาวด์และปัญญาประดิษฐ์เข้าด้วยกันอย่างมีประสิทธิภาพ

3.3 การออกแบบและพัฒนาระบบส่งไฟล์เข้าสู่ Sandbox

ระบบหลักของ Sandbox ใช้กระบวนการรับส่งไฟล์และการจัดเตรียมพื้นที่ทดสอบจำลองให้พร้อมสำหรับการวิเคราะห์พฤติกรรม โดยมีการกำหนดค่าการสื่อสารผ่านชุดโปรโตคอลที่รองรับการทำงานทั้งบนระบบปฏิบัติการวินโดวส์และลินุกซ์ ส่งผลให้ระบบสามารถดำเนินการตามขั้นตอนที่สำคัญได้อย่างมีประสิทธิภาพ ดังมาตรการแยกส่วนที่อธิบายในหัวข้อ 3.2.5

3.3.1 การกำหนดค่าระบบอัตโนมัติและการสื่อสารผ่าน API

ระบบเลือกใช้แพลตฟอร์ม n8n เป็นกลไกศูนย์กลางในการประสานงาน โดยเริ่มจากการสร้างจุดรับข้อมูลเพื่อรอการนำเข้าไฟล์จากผู้ใช้งาน เมื่อตรวจพบไฟล์ในระบบ เวิร์กโฟลว์จะสั่งการให้ส่งชุดข้อมูลไปยังเซิร์ฟเวอร์ควบคุมภายในพื้นที่ทดสอบผ่านโหนดเครือข่าย การสื่อสารดังกล่าวอาศัยชุดคำสั่งผ่านอินเทอร์เฟซการเขียนโปรแกรมแอปพลิเคชันที่ออกแบบมาเฉพาะสำหรับการควบคุมระบบจำลอง ซึ่งครอบคลุมตั้งแต่การนำส่งไฟล์ การสั่งประมวลผล ไปจนถึงการจัดเก็บผลลัพธ์พฤติกรรมทางระบบและเครือข่าย โดยรายละเอียดของชุดคำสั่ง cURL ที่ใช้ในการบริหารจัดการระบบจำลองปรากฏใน **ตารางที่ 3.2**

ตารางที่ 3.2: รายละเอียดชุดคำสั่ง API สำหรับบริหารจัดการและควบคุมระบบจำลองผ่านชุดคำสั่ง cURL

ลำดับ	ชื่อ API	คำสั่ง (cURL)	OS
1	Upload	curl -X POST -F ”file=@ping8.exe” [URL]/upload	Windows
		curl -X POST -F ”file=@linux.py” [URL]/upload	Linux
2	List	curl [URL]/list	Windows
		curl [URL]/list	Linux
3	Execute	curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/execute	Windows
		curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/execute	Linux
4	Monitor	curl [URL]/monitor/4512/	Windows
		curl [URL]/monitor/1234	Linux
5	Start Capture	curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/start_capture	Windows
		curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/start_capture	Linux
6	Stop Capture	curl -X POST [URL]/stop_capture	Windows
		curl -X POST [URL]/stop_capture	Linux
7	Terminate	curl -X POST [URL]/terminate/4512	Windows
		curl -X POST [URL]/terminate/1234	Linux
8	Analyze	curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/analyze	Windows
		curl -X POST -H ”Content-Type: application/json” -d ”{...}” [URL]/analyze	Linux
9	Cat	curl ”[URL]/cat?filename=trace.csv”	Windows
		curl ”[URL]/cat?filename=trace.log”	Linux
10	Download	curl -o win.csv ”[URL]/download_pcap”	Windows
		curl -o linux.pcap ”[URL]/download_pcap”	Linux

แผนผังลำดับขั้นตอนการทำงานของโปรแกรม (agent.py) ซึ่งติดตั้งภายในแซนด์บ็อกซ์ทำหน้าที่ควบคุมสภาพแวดล้อมจำลองระดับภายในแสดงรายละเอียดดังปรากฏใน **รูปที่** 3.12 สามารถอธิบายแยกตามส่วนงานได้ดังนี้

เริ่มต้นด้วยส่วนการทำงานของ **Server (Flask)** ซึ่งทำหน้าที่เป็นจุดรับคำสั่งหลัก โดยโปรแกรมจะเริ่มทำการตรวจสอบสภาพแวดล้อมเบื้องต้น ประกอบด้วยการโหลดค่าคอนฟิกูเรชันและการตรวจสอบความพร้อมของเครื่องมือวิเคราะห์ระดับระบบ ได้แก่ Tshark สำหรับการวิเคราะห์แพ็กเก็ตเครือข่าย Procmon สำหรับ Windows หรือ Strace สำหรับ Linux เพื่อให้แน่ใจว่าระบบพร้อมสำหรับการดักจับพฤติกรรม เมื่อมีคำขอผ่าน API เข้ามา ระบบจะเข้าสู่ส่วนการตัดสินใจเพื่อจำแนกประเภทของคำขอ API Request Type ซึ่งแบ่งออกเป็น 3 เส้นทางหลัก ดังนี้

กรณีการจัดการไฟล์ (Yes path) ระบบจะทำการบันทึกไฟล์มัลแวร์ที่ได้รับลงในโฟลเดอร์ UPLOAD_FOLDER จากนั้นจะเข้าสู่ขั้นตอนการปรับแต่งไฟล์ โดยการแก้ไขรูปแบบตัวอักษรขึ้นบรรทัดใหม่ (CRLF) ให้เหมาะสมกับระบบปฏิบัติการเป้าหมาย พร้อมทั้งกำหนดสิทธิ์การเข้าถึงไฟล์ (chmod 755) เพื่อให้สามารถรันโปรแกรมได้

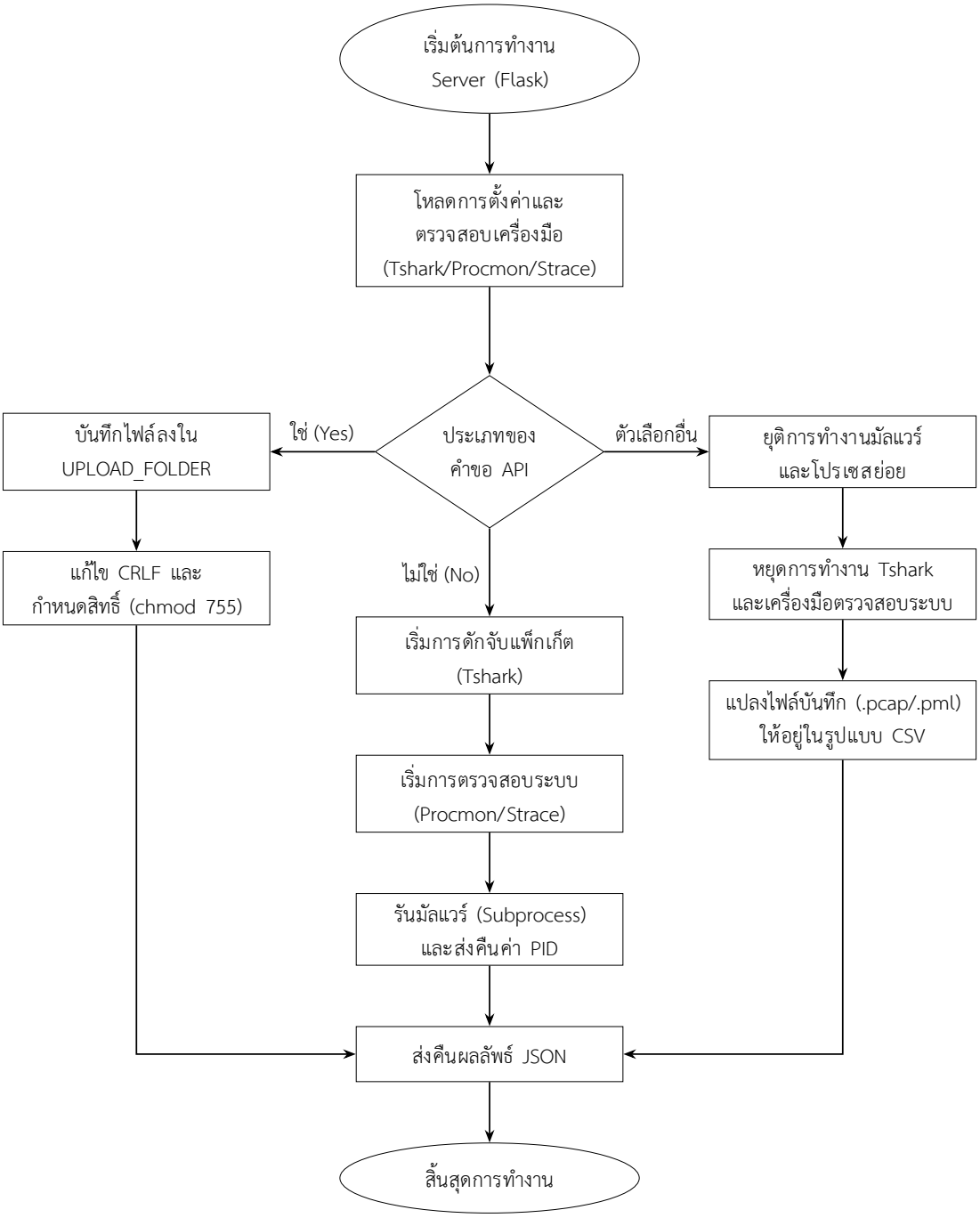
กรณีเริ่มการวิเคราะห์ (No path) โปรแกรมจะสั่งเริ่มการทำงานของเครื่องมือดักจับข้อมูล โดยเริ่มจาก **Tshark** เพื่อบันทึกการจราจรทางเครือข่าย ตามด้วยการเรียกใช้ **Procmon/Strace** เพื่อเฝ้าสังเกตการเรียกใช้งาน System Calls และกิจกรรมของระบบ จากนั้นจึงทำการรันมัลแวร์ผ่าน **Subprocess** และทำการส่งคืนค่า Process ID (PID) เพื่อใช้ในการติดตามผลในลำดับถัดไป

กรณีสิ้นสุดการวิเคราะห์ ระบบจะส่งคำสั่งเพื่อยุติการทำงาน (**Kill**) ของโปรเซสมัลแวร์และโปรเซสย่อยทั้งหมดที่เกี่ยวข้อง

จากนั้นจะสั่งหยุดการทำงานของเครื่องมือตรวจสอบระบบ Tshark/Procmon/Strace และเข้าสู่ขั้นตอนการประมวลผลข้อมูลหลังการทดลอง

โดยการแปลงไฟล์บันทึกดิบ เช่น .pcap หรือ .pml ให้อยู่ในรูปแบบ **CSV** เพื่อความสะดวกในการนำไปวิเคราะห์ในเชิงสถิติหรือใช้กับโมเดลการเรียนรู้ของเครื่อง

ในขั้นตอนสุดท้าย ไม่ว่าจะเป็นการจัดการไฟล์ การรันมัลแวร์ หรือการหยุดการทำงาน ระบบจะรวบรวมสถานะและผลลัพธ์ที่ได้จากการดำเนินการ เพื่อส่งคืนกลับไปยังผู้เรียกใช้ในรูปแบบ **JSON Response** และสิ้นสุดวงจรการทำงานของ Agent

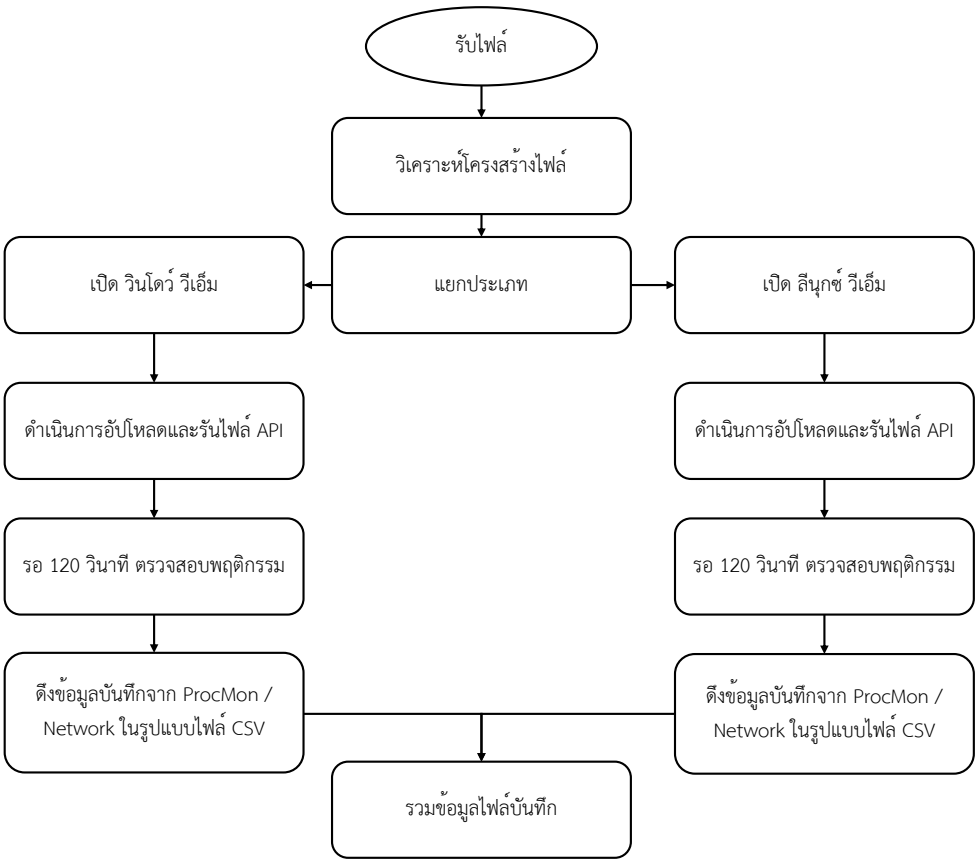


รูปที่ 3.12: แผนผังลำดับขั้นตอนการทำงานของโปรแกรมตัวแทนภายในสภาพแวดล้อมการทดสอบ

3.3.2 การจัดลำดับโหนดการทำงานอัตโนมัติบนแพลตฟอร์ม n8n

การบริหารจัดการกระบวนการวิเคราะห์อาศัยการจัดลำดับเวิร์กโฟลว์อัตโนมัติที่ครอบคลุมตั้งแต่การรับข้อมูลเข้าสู่ระบบไปจนถึงการรายงานผลสรุป โดยโครงสร้างการทำงานแบ่งออกเป็นสองช่วงหลัก ซึ่งกระบวนการช่วงแรกมุ่งเน้นการจัดเตรียมและควบคุมสภาพแวดล้อมจำลองดังแสดงใน **รูปที่** 3.13

ลำดับการทำงานเริ่มต้นจากโหนดเว็บฮุคที่ทำหน้าที่เป็นจุดรับข้อมูลไฟล์มัลแวร์และข้อมูลเมทาดาตาจากส่วนต่อประสานผู้ใช้งาน จากนั้นระบบใช้โหนดสคริปต์เพื่อวิเคราะห์ค่าสำคัญในการจำแนกประเภทระบบปฏิบัติการเป้าหมายอย่างแม่นยำ ในขั้นตอนถัดมา เวิร์กโฟลว์ดำเนินการสั่งการผ่านโหนดศูนย์กลางจัดการระบบคลาวด์เพื่อเริ่มต้นการทำงานของเครื่องจำลองเสมือนบนอาชัวร์ พร้อมทั้งมีการตรวจสอบสถานะความพร้อมของระบบปฏิบัติการก่อนดำเนินการในลำดับถัดไป เมื่อเครื่องจำลองพร้อมปฏิบัติงาน ระบบจะส่งคำสั่งผ่านโหนดการสื่อสารไปยังส่วนต่อประสานโปรแกรมประยุกต์ภายในเพื่อนำส่งไฟล์มัลแวร์และเริ่มกระบวนการบันทึกพฤติกรรมผ่านโปรแกรมตัวแทนภายในพื้นที่ปิด ซึ่งลำดับขั้นตอนเหล่านี้ช่วยให้การจัดเตรียมสภาพแวดล้อมเพื่อการวิเคราะห์เป็นไปอย่างอัตโนมัติและลดความผิดพลาดจากการดำเนินงานด้วยบุคคล ดังลำดับกระบวนการที่ปรากฏใน **รูปที่** 3.13



รูปที่ 3.13: กระบวนการส่วนหน้าสำหรับการรับไฟล์และควบคุมสภาพแวดล้อมจำลอง

สำหรับกระบวนการในส่วนหลังซึ่งมุ่งเน้นการประมวลผลวิเคราะห์พฤติกรรมเชิงลึกและการบริหารจัดการทรัพยากรภายหลังการทดสอบแสดงรายละเอียดดังปรากฏใน **รูปที่** 3.14 อธิบายได้ดังนี้

การรวมไฟล์เหตุการณ์ (Merge Data) เริ่มต้นด้วยการรวบรวมไฟล์เหตุการณ์ที่ถูกสร้างขึ้นจากการทดสอบมัลแวร์ในสภาวะแวดล้อมจำลอง โดยระบบจะทำการผนวกข้อมูลจากแหล่งต่าง ๆ เข้าด้วยกัน

เพื่อสร้างชุดข้อมูลเอกภาพที่ครอบคลุมพฤติกรรมทั้งหมดของระบบในช่วงเวลาที่กำหนด ซึ่งถือเป็นโครงสร้างพื้นฐานสำคัญสำหรับการนำไปประมวลผลในเชิงสถิติและทฤษฎีสารสนเทศต่อไป

การเตรียมและกรองข้อมูลปูมเหตุการณ์ (Data Pre-processing) ในขั้นตอนนี้ ข้อมูลดิบจะผ่านกระบวนการทำความสะอาดและลดทอนความซ้ำซ้อน โดยการคัดเลือกเฉพาะคุณลักษณะที่มีนัยสำคัญต่อการจำแนกพฤติกรรมมัลแวร์ และกำจัดสัญญาณรบกวนที่เกิดขึ้นจากกระบวนการทำงานปกติของระบบปฏิบัติการ เพื่อเพิ่มประสิทธิภาพในการประมวลผลและลดความคลาดเคลื่อนในการวิเคราะห์ของแบบจำลอง

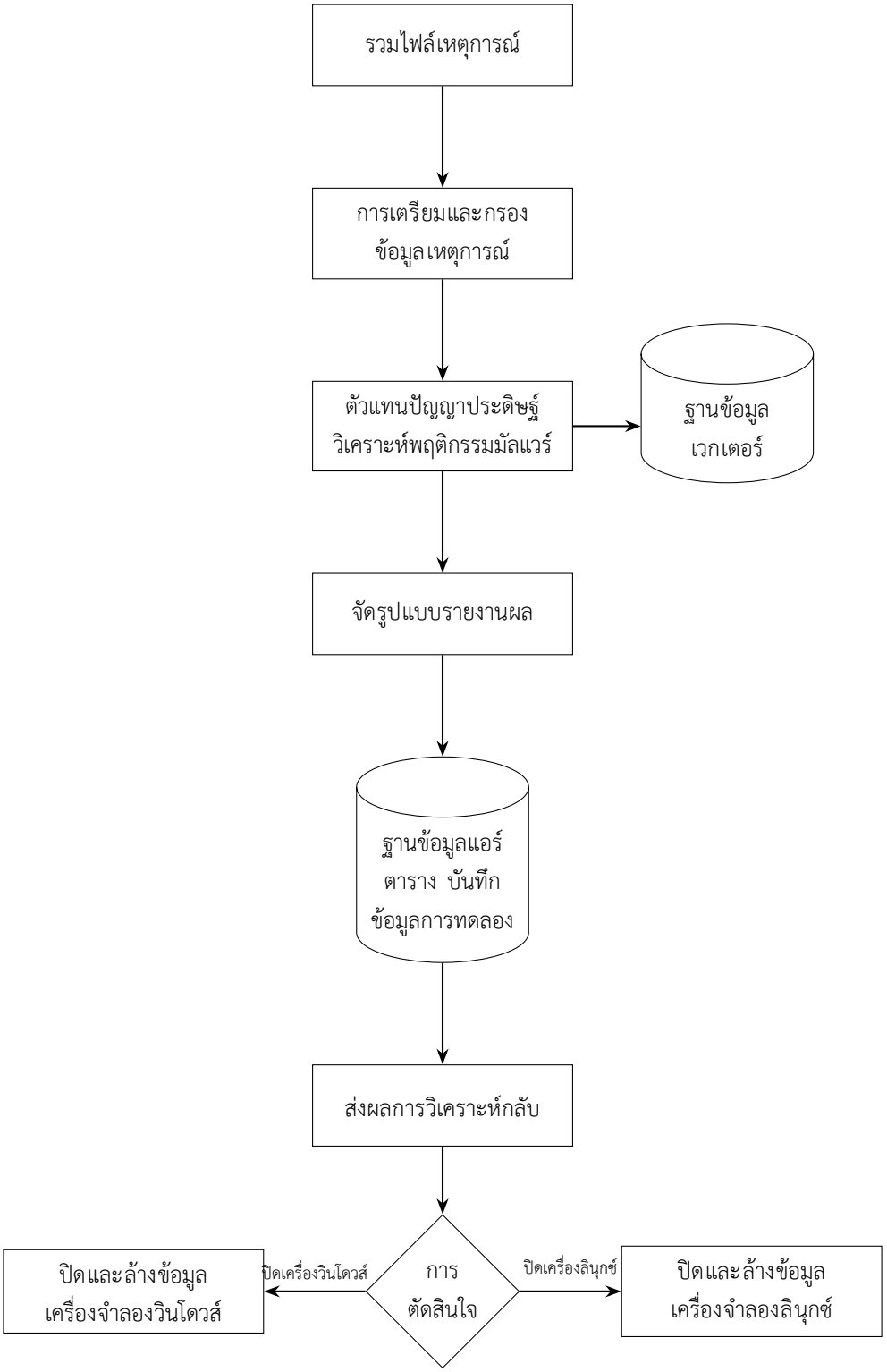
ตัวแทนปัญญาประดิษฐ์วิเคราะห์พฤติกรรมมัลแวร์ (AI Agent - Gemini) ระบบใช้แบบจำลองภาษาขนาดใหญ่ทำหน้าที่เป็นหน่วยวิเคราะห์หลักเพื่อพิจารณาลำดับของเหตุการณ์และประเมินระดับความเสี่ยง โดยจะมีการทำงานร่วมกับฐานข้อมูลเวกเตอร์ (Vector Store) เพื่อสืบค้นข้อมูลพฤติกรรมที่มีลักษณะคล้ายคลึงกันในเชิงบริบทผ่านกระบวนการดึงข้อมูลเสริมการสร้าง (Retrieval-Augmented Generation) ซึ่งช่วยให้การระบุประเภทของภัยคุกคามมีความแม่นยำและสมเหตุสมผลยิ่งขึ้น

การจัดรูปแบบรายงานผล (Parse JSON Output) ผลลัพธ์ที่ได้จากการวิเคราะห์จะถูกนำมาเข้ากระบวนการจัดลำดับชั้นข้อมูลให้อยู่ในรูปแบบเชิงโครงสร้างที่ชัดเจน โดยการแปลงข้อมูลให้อยู่ในรูปแบบเจสัน เพื่อให้ระบบกึ่งอัตโนมัติสามารถส่งต่อค่าตัวแปรและพารามิเตอร์ต่าง ๆ ไปยังโมดูลการทำงานอื่นได้อย่างเป็นระบบและรวดเร็ว

ฐานข้อมูลแอร์ตารางบันทึกข้อมูลการทดลอง (Airtable Database) ข้อมูลรายงานเชิงเทคนิคและผลการทดลองทั้งหมดจะถูกจัดเก็บลงในระบบจัดการฐานข้อมูลแบบเชิงสัมพันธ์ เพื่อสร้างคลังข้อมูลสำหรับการวิเคราะห์ย้อนหลัง และทำหน้าที่เป็นแหล่งข้อมูลหลักสำหรับส่วนต่อประสานผู้ใช้งานในการเข้าถึงผลการวิเคราะห์ได้อย่างมีประสิทธิภาพ

การส่งผลการวิเคราะห์กลับ (Webhook Response): ระบบจะทำการสื่อสารข้อมูลผ่านกลไกการส่งสัญญาณกลับไปยังหน่วยควบคุมหลัก เพื่อแจ้งสถานะการทำงานและรายงานสรุปผลการวิเคราะห์พฤติกรรมมัลแวร์ที่ตรวจพบ ซึ่งเป็นขั้นตอนสำคัญในการเชื่อมต่อระหว่างส่วนการประมวลผลข้อมูลและส่วนการจัดการโครงสร้างพื้นฐาน

การตัดสินใจและการคืนสภาพระบบ (Decision and Shutdown) ในขั้นตอนสุดท้าย ระบบจะทำการประมวลผลการตัดสินใจเพื่อสั่งการปิดเครื่องจำลองทั้งในส่วนของวินโดวส์และลินุกซ์ พร้อมทั้งดำเนินการฟื้นฟูสภาพแวดล้อมทางกายภาพและตรรกะให้กลับสู่สถานะเริ่มต้นที่บริสุทธิ์ กระบวนการนี้มีวัตถุประสงค์เพื่อทำลายทรัพยากรที่ปนเปื้อนสารอันตรายจากการทดสอบและเตรียมระบบให้พร้อมสำหรับการเริ่มวงรอบการทดสอบครั้งถัดไปภายใต้มาตรฐานความมั่นคงปลอดภัยสูงสุด



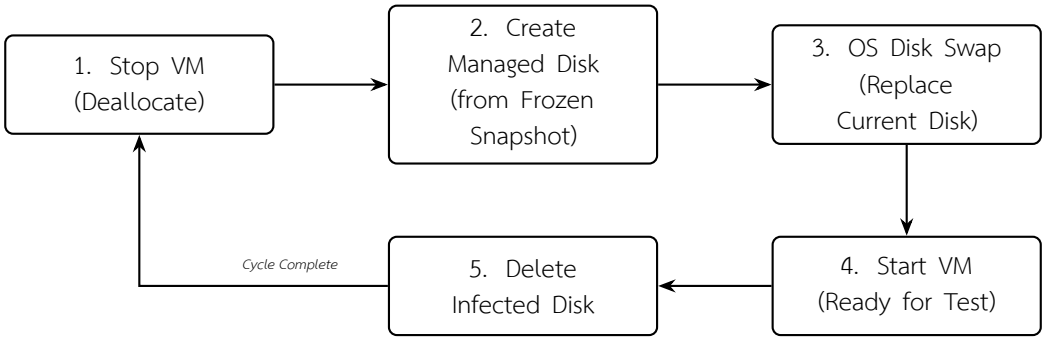
รูปที่ 3.14: กระบวนการส่วนหลังสำหรับการวิเคราะห์พฤติกรรมด้วยปัญญาประดิษฐ์และการคืนสภาพระบบ

3.3.3 การบริหารจัดการวงจรชีวิตของคลาวด์ Sandbox

เพื่อให้พื้นที่การวิเคราะห์มัลแวร์มีความบริสุทธิ์และปลอดภัยในทุกรอบการทดสอบ ผู้วิจัยได้พัฒนาชุดคำสั่งเชลล์สคริปต์เพื่อควบคุมสถานะของเครื่องจำลองผ่าน Azure API โดยตรง เชลล์สคริปต์มีกลไกทำงานเพื่อการสื่อสารผ่านส่วนต่อประสานแบบ REST หรือ API โดยเริ่มต้นจากการดึงข้อมูลโทเคนยืนยันตัวตนเพื่อเป็นสิทธิ์เข้าถึงและจัดการทรัพยากรคลาวด์ซึ่งการดำเนินงานในแต่ละขั้นตอนของสคริปต์จะทำงานผสานกับกลไกตรวจสอบสถานะแบบวนซ้ำเพื่อให้มั่นใจว่าการประกาศคำสั่งได้รับการตอบสนองสำเร็จก่อนดำเนินงานในลำดับถัดไป

กระบวนการฟื้นฟูระบบจะทำงานแบบอัตโนมัติเพื่อล้างข้อมูลสิ่งปนเปื้อนดังขั้นตอนที่ระบุใน **รูปที่** 3.15 โดยเริ่มต้นจากการระงับการทำงานของเครื่องจำลองเป้าหมายผ่านการส่งคำขอประเภท HTTP ชนิด POST ไปยังเส้นทาง /deallocate ของเครื่องจำลองเพื่อช่วยลดภาระค่าใช้จ่ายและล้างการจองทรัพยากร จากนั้นระบบจะเข้าสู่ขั้นตอนการสร้างหน่วยจัดเก็บข้อมูลใหม่ โดยส่งคำขอ HTTP ชนิด PUT เพื่อสร้างดิสก์หน่วยความจำหลักก้อนใหม่ผ่านเส้นทาง /disks พร้อมทั้งกำหนดค่าพารามิเตอร์ให้เป็นการคัดลอกจากภาพบันทึกข้อมูลตั้งต้น ซึ่งมีความบริสุทธิ์จากมัลแวร์

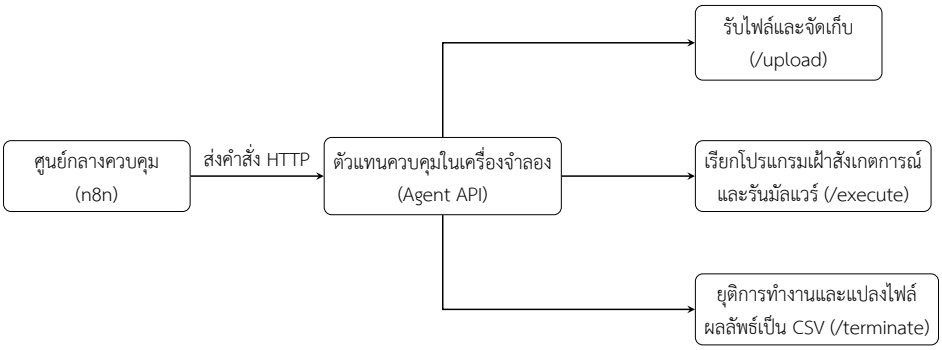
เมื่อกระบวนการสร้างดิสก์ก้อนใหม่เสร็จสมบูรณ์ ระบบจะดำเนินขั้นตอนถัดไปคือสลับหน่วยจัดเก็บข้อมูลหลักของเครื่องจำลอง โดยส่งคำขอผ่านโปรโตคอล HTTP ในรูปแบบ PATCH เพื่ออัปเดตการจัดสรรพื้นที่ให้ชี้ไปที่เส้นทางของดิสก์หน่วยความจำก้อนใหม่ที่เพิ่งเตรียมการแล้วเสร็จ ภายหลังจากการสลับดิสก์เรียบร้อยแล้ว ระบบจะสานต่อการทำงานด้วยการสั่งให้เครื่องจำลองเริ่มบูตระบบปฏิบัติการอีกครั้งผ่านเส้นทาง /start เพื่อเตรียมสภาพแวดล้อมให้พร้อมสำหรับการทดสอบรอบถัดไป ท้ายที่สุดระบบจะดำเนินการล้างข้อมูลสิ่งปนเปื้อนด้วยการทำลายดิสก์ก้อนเดิมที่เผชิญกับมัลแวร์ทิ้งทันทีผ่านคำขอ HTTP ชนิด DELETE เพื่อคืนพื้นที่กลับสู่ระบบคลาวด์และรักษาความปลอดภัยของโครงสร้างพื้นฐานอย่างสูงสุดตามวัฏจักรที่ปรากฏใน **รูปที่** 3.15



รูปที่ 3.15: ขั้นตอนการฟื้นฟูสภาพแวดล้อม Sandbox บน Microsoft Azure

3.4 การพัฒนาระบบควบคุมและส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง

เพื่อให้กระบวนการวิเคราะห์พฤติกรรมมัลแวร์สามารถดำเนินไปได้อย่างอัตโนมัติ ผู้วิจัยได้นำส่วนต่อประสานโปรแกรมประยุกต์มาประยุกต์ใช้ในการควบคุมเครื่องจำลองและสั่งการโปรแกรมภายในพื้นที่ทดสอบแบบปิด โดยส่วนต่อประสานดังกล่าวพัฒนาด้วยภาษาไพธอนและเฟรมเวิร์กฟลาสก์ (Flask) เพื่อทำหน้าที่เป็นตัวแทนควบคุมรับคำสั่งจากศูนย์กลางจัดการกระบวนการทำงานอัตโนมัติ ดังแสดงใน **รูปที่** 3.16



รูปที่ 3.16: ผังการทำงานของส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง

รายละเอียดของเส้นทางจุดเชื่อมต่อและตัวอย่างการเรียกใช้งานผ่านชุดคำสั่งสำหรับการสื่อสารระหว่างศูนย์กลางควบคุมและตัวแทนภายในเครื่องจำลองปรากฏใน **ตารางที่** 3.3 ซึ่งครอบคลุมตั้งแต่การตรวจสอบสถานะความพร้อมไปจนถึงการดาวน์โหลดผลลัพธ์กลับมายังส่วนกลางเพื่อเข้าสู่กระบวนการวิเคราะห์เชิงลึก

ตารางที่ 3.3: รายละเอียดและตัวอย่างชุดคำสั่งสำหรับการเรียกใช้งานส่วนต่อประสานโปรแกรมประยุกต์ภายในเครื่องจำลอง

เส้นทาง (Endpoint)	**รูปแบบ**	**หน้าที่และตัวอย่างการเรียกใช้งาน (cURL)**
/status	GET	**หน้าที่** ตรวจสอบสถานะการเชื่อมต่อและรายละเอียดของตัวแทนควบคุม **ตัวอย่าง** curl -X GET http://[IP]:5000/status
/upload	POST	**หน้าที่** นำส่งไฟล์มัลแวร์จากภายนอกเข้าสู่พื้นที่จัดเก็บชั่วคราวของเครื่องจำลอง **ตัวอย่าง** curl -X POST -F ”file=@malware.exe” http://[IP]:5000/upload
/execute	POST	**หน้าที่** สั่งรันไฟล์เป้าหมายพร้อมกับเปิดใช้งานเครื่องมือวิเคราะห์พฤติกรรมเบื้องหลัง **ตัวอย่าง** curl -X POST -H ”Content-Type: application/json” -d ”{\”file_path\”:\”mal.exe\”}” http://[IP]:5000/execute
/terminate/<pid>	POST	**หน้าที่** ยุติการทำงานของโพรเซสและครอบครัวโพรเซส พร้อมแปลงแฟ้มข้อมูลเป็น CSV **ตัวอย่าง** curl -X POST http://[IP]:5000/terminate/1234
/download_pcap	GET	**หน้าที่** ดาวน์โหลดแฟ้มข้อมูลพฤติกรรมหรือแพ็กเก็ตที่ผ่านการประมวลผลแล้วกลับออกมา **ตัวอย่าง** curl -o result.csv ”http://[IP]:5000/download_pcap?filename=result.csv”

3.4.1 การออกแบบคำสั่งสำหรับปัญญาประดิษฐ์

หัวใจสำคัญของการวิเคราะห์พฤติกรรมมัลแวร์เชิงลึกอาศัยการกำหนดโครงสร้างคำสั่งสำหรับปัญญาประดิษฐ์ เพื่อควบคุมทิศทางการประมวลผลของแบบจำลองภาษาขนาดใหญ่ Gemini ให้มีความแม่นยำและสอดคล้องกับหลักการวิเคราะห์ความปลอดภัยทางคอมพิวเตอร์ โดยผู้วิจัยได้กำหนดบทบาทสมมติให้ปัญญาประดิษฐ์ทำหน้าที่เป็นผู้เชี่ยวชาญด้านการวิเคราะห์มัลแวร์ระดับสูง พื่อสร้างกรอบแนวคิดในการพิจารณาข้อมูลที่มีความสับซ้อนอย่างเป็นระบบ ซึ่งโครงสร้างของคำสั่งดังกล่าวมุ่งเน้นการกำหนดลำดับความสำคัญของข้อมูลพฤติกรรมเพื่อให้ปัญญาประดิษฐ์พิจารณาจุดบ่งชี้ภัยคุกคามที่สำคัญเป็นลำดับแรก

ในการวิเคราะห์ข้อมูลปูมเหตุการณ์ที่ได้รับจากเครื่องจำลอง ระบบจะสั่งการให้ปัญญาประดิษฐ์มุ่งเน้นการตรวจจับและวิเคราะห์พฤติกรรมในมิติหลักที่มีนัยสำคัญต่อความปลอดภัย ประการแรกคือการตรวจจับความพยายามในการสื่อสารกับเลขที่อยู่ไอพีภายนอกที่ผิดปกติซึ่งสอดคล้องกับพฤติกรรมการเชื่อมต่อกับเซิร์ฟเวอร์ควบคุมและสั่งการ ประการถัดมาคือการเฝ้าระวังการเข้าถึงและการเปลี่ยนแปลงค่ารีจิสทรีที่สำคัญของระบบปฏิบัติการซึ่งมักถูกใช้เพื่อสร้างความคงทนต่อการโจมตี รวมถึงการพิจารณาลำดับขั้นตอนการทำงานที่เข้าข่ายกระบวนการฉีดรหัสอันตรายเข้าสู่โพรเซสที่ปลอดภัยของระบบ

การออกแบบคำสั่งยังครอบคลุมถึงการกำหนดรูปแบบการให้เหตุผลเชิงโครงสร้างเพื่อป้องกันการสร้างข้อมูลเท็จและช่วยให้ผู้อ่านรายงานสามารถทำความเข้าใจที่มาของผลสรุปได้อย่างชัดเจน โดยปัญญาประดิษฐ์จะทำการเปรียบเทียบข้อมูลพฤติกรรมที่ได้รับจริงกับฐานความรู้ที่จัดเก็บไว้ในฐานข้อมูลเวกเตอร์ เพื่อจัดหมวดหมู่ภัยคุกคามตามความเสี่ยงที่ตรวจพบ กระบวนการดังกล่าวส่งผลให้รายงานผลสรุปที่ได้รับจากส่วน

หลังบ้านมีความแม่นยำและมีรายละเอียดทางเทคนิคที่ครบถ้วนสำหรับการตอบสนองต่อภัยคุกคามในลำดับถัดไป

3.4.2 การจัดเก็บประวัติการวิเคราะห์ลงฐานข้อมูล

นอกจากระบบจะแสดงผลลัพธ์บนหน้าจอเว็บแล้ว ระบบยังได้ทำการส่งข้อมูลผลการประมวลผลทั้งหมดไปจัดเก็บอย่างเป็นระบบในฐานข้อมูลแอร์เทเบิล โดยอัตโนมัติ ดังภาพที่ 3.17 ซึ่งครอบคลุมรายละเอียดที่สำคัญ ได้แก่ ชื่อไฟล์ ระบบปฏิบัติการเป้าหมาย สถานะความเสี่ยง บทสรุปพฤติกรรม และสถานะการตรวจพบเซิร์ฟเวอร์ควบคุม การจัดเก็บข้อมูลในลักษณะนี้ช่วยให้ผู้ดูแลระบบสามารถตรวจสอบประวัติย้อนหลัง ค้นหาความสัมพันธ์ของภัยคุกคาม และนำข้อมูลไปประยุกต์ใช้กับการตรวจสอบและวิเคราะห์รูปแบบการโจมตีในอนาคตได้อย่างมีประสิทธิภาพ



57	linux_benign_test.sh	Linux	2026/3/8	7:39หลังเที่...	BENIGN	สคริปต์ทำงานสำเร็จและไม่มีพฤติกรรมที่น่าสงสัย	{ "Filename": "linux*benign*test.sh", "Target_OS": "Linux",...	No
58	a557d96f80d3cbe663dff79421902b556dff2cec54d7307a7f879cb20268b15e.exe	Windows	2026/3/8	7:55หลังเที่...	APT29	พบการเข้าถึง lsass.exe และการเชื่อมต่อเครือข่ายที่น่าสงสัย	{ "Filename": "a557d96f80d3cbe663dff79421902b556dff2cec54d7307...	Yes
59	0a6bdad5dc4add4b571d11608f30eca32acf96da82688f8839f13efd64e76a8f.exe	Windows	2026/3/8	8:09หลังเที่...	APT29	พบพฤติกรรมการอ่านข้อมูลจาก lsass.exe ซึ่งเกี่ยวข้องกับการเข้าถึงข้อมูลประจำตัว และมีการสื่อสารกับ IP ภายนอกที่น่า...	{ "Filename": "0a6bdad5dc4add4b571d11608f30eca32acf96da82688f...	Yes
60	benign_system_test.bat	Windows	2026/3/8	8:29หลังเที่...	BENIGN	สคริปต์ทำการตรวจสอบไฟล์ระบบปกติและไม่มีพฤติกรรมที่น่าสงสัย	{ "Filename": "benign*system*test.bat", "Target_OS": "Windows",...	No
61	ping_google_test.bat	Windows	2026/3/8	8:36หลังเที่...	BENIGN	ไฟล์สคริปต์ `.bat` นี้ทำงานสำเร็จ โดยมีการเข้าถึงไฟล์ระบบ `lsasrv.dll` และสร้างไฟล์ `VMAgentLogs.zip` ใน...	{ "Filename": "ping*google*test.bat", "Target_OS": "Windows",...	No
62	0a6bdad5dc4add4b571d11608f30eca32acf96da82688f8839f13efd64e76a8f.exe	Windows	2026/3/8	10:27หลัง...	APT29	ตรวจพบการเข้าถึงไฟล์ lsasrv.dll ซึ่งเป็นส่วนหนึ่งของ Local Security Authority Subsystem Service (LSASS) ...	{ "Filename": "0a6bdad5dc4add4b571d11608f30eca32acf96da82688f...	Yes

รูปที่ 3.17: หน้าจอแสดงประวัติการจัดเก็บผลลัพธ์การวิเคราะห์มัลแวร์บนฐานข้อมูลแอร์เทเบิล

บทที่ 4

การทดลองและวิเคราะห์ผล

การทดลองในบทนี้มีวัตถุประสงค์เพื่อประเมินประสิทธิภาพและความสมบูรณ์ของระบบรักษาความปลอดภัยปลายทางที่ได้ออกแบบไว้ในบทที่ 3 โดยมุ่งเน้นการวิเคราะห์ขีดความสามารถของระบบในการตรวจจับมัลแวร์ข้ามแพลตฟอร์ม สมรรถนะการบริหารจัดการระยะเวลาประมวลผล และคุณภาพของรายงานสรุปพฤติกรรมความเสี่ยงที่ได้รับจากปัญญาประดิษฐ์ เพื่อยืนยันว่ากลไกการทำงานอัตโนมัติที่พัฒนาขึ้นสามารถตอบสนองต่อภัยคุกคามได้อย่างแม่นยำและรวดเร็ว

4.1 การตั้งค่าและขอบเขตการทดลอง

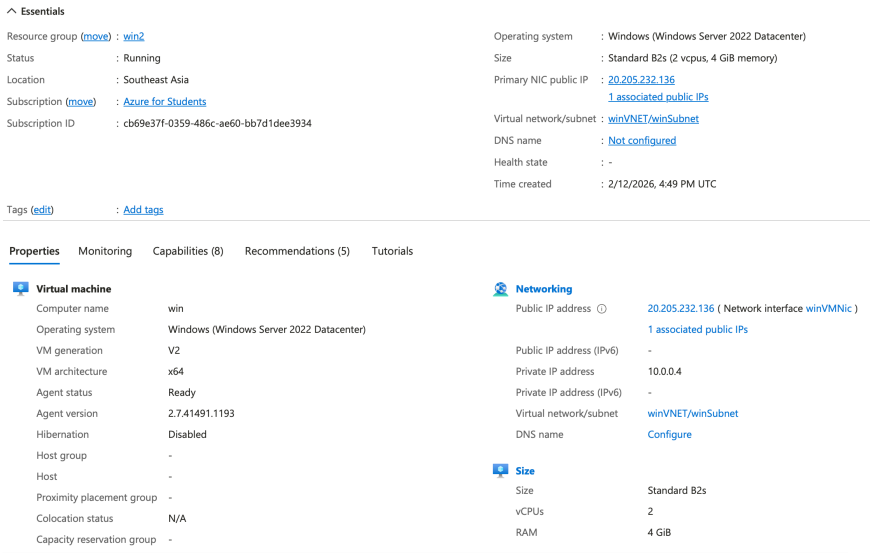
เพื่อให้ผลการทดสอบสะท้อนถึงประสิทธิภาพของระบบภายใต้สภาวะการทำงานจริง ผู้พัฒนาโครงงานได้กำหนดรายละเอียดของชุดข้อมูลโดยคัดเลือกมัลแวร์จากแหล่งข้อมูลมาตรฐานมัลแวร์บาซาร์ (MalwareBazaar) จำนวนรวมหนึ่งร้อยรายการ ประกอบด้วยไฟล์ปฏิบัติการสำหรับระบบวินโดวส์จำนวนห้าสิบรายการและไฟล์สำหรับระบบลินุกซ์จำนวนห้าสิบรายการ ซึ่งครอบคลุมตระกูลมัลแวร์ที่หลากหลาย อาทิ แรนซัมแวร์ สตีลเลอร์ และโทรจันควบคุมระยะไกล นอกจากนี้ยังจัดเตรียมชุดข้อมูลไฟล์ปกติจำนวนห้าสิบรายการเพื่อใช้ในการทดสอบอัตราการแจ้งเตือนผิดพลาด การทดสอบทั้งหมดดำเนินการภายในสภาพแวดล้อมจำลองของไมโครซอฟท์อาซัวร์ที่ติดตั้งโปรแกรมควบคุมตัวแทนตามที่ระบุไว้ในบทก่อนหน้า โดยมีตัวแปรที่ใช้ในการประเมินผลหลักประกอบด้วย ความถูกต้องในการจำแนกประเภทความเสี่ยง ระยะเวลาการประมวลผลตั้งแต่ต้นจนจบกระบวนการ และความแม่นยำของรายงานสรุปผลพฤติกรรม

4.2 การจัดเตรียมเครื่องมือทดลอง

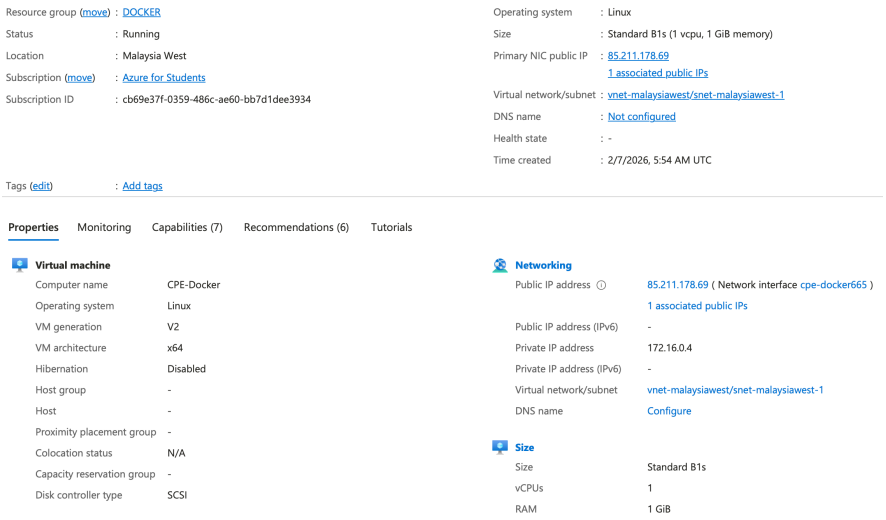
4.2.1 เครื่องคอมพิวเตอร์เสมือนสำหรับการสร้างสภาพแวดล้อมจำลอง (แซนด์บ็อกซ์)

สำหรับการสร้างสภาพแวดล้อมจำลอง (แซนด์บ็อกซ์) ในโครงงานนี้ ผู้พัฒนาโครงงานได้ประยุกต์ใช้เครื่องเสมือนบนระบบคลาวด์ Microsoft Azure เนื่องจากมีคุณสมบัติในการคืนค่าสถานะระบบกลับสู่จุดเริ่มต้นได้อย่างรวดเร็วและมีประสิทธิภาพ โดยได้จัดเตรียมระบบปฏิบัติการไว้ 2 รูปแบบ ได้แก่ Windows และ Linux พร้อมทั้งดำเนินการบันทึกสถานะเริ่มต้นในลักษณะสแนปชอต ไว้ล่วงหน้า

ในกระบวนการทดสอบ เมื่อระบบการบริหารจัดการเวิร์กโฟลว์ n8n ส่งคำสั่งเพื่อดำเนินการรันไฟล์มัลแวร์เสร็จสิ้นลง ระบบจะทำการกู้คืนสถานะกลับไปยังจุดเริ่มต้นทันที เพื่อให้ระบบปฏิบัติการอยู่ในสภาวะสะอาดปราศจากการปนเปื้อนของรหัสต้นฉบับที่เป็นอันตราย และมีความพร้อมสำหรับการทดสอบมัลแวร์ ในลำดับถัดไปได้อย่างปลอดภัย



รูปที่ 4.1: การกำหนดค่าระบบปฏิบัติการ Windows บนระบบ Microsoft Azure



รูปที่ 4.2: การกำหนดค่าระบบปฏิบัติการ Linux บนระบบ Microsoft Azure

รายละเอียดคุณลักษณะของเครื่องคอมพิวเตอร์เสมือนที่จัดตั้งขึ้นแสดงดังรูปที่ 4.1 และรูปที่ 4.2 โดยสรุปเปรียบเทียบคุณสมบัติเฉพาะของทั้งสองระบบได้ดังตารางที่ 4.1

4.3 การจัดเตรียมซอฟต์แวร์และระบบควบคุมส่วนกลาง

ในส่วนของกลไกการบริหารจัดการระบบวิเคราะห์มัลแวร์ ผู้พัฒนาโครงงานได้ดำเนินการติดตั้งซอฟต์แวร์ควบคุมส่วนกลางบนเครื่องคอมพิวเตอร์แม่ข่าย โดยอาศัยเทคโนโลยีคอนเทนเนอร์ในรูปแบบด็อก

ตารางที่ 4.1: คุณสมบัติเฉพาะของเครื่องคอมพิวเตอร์เสมือน (Virtual Machine) บนระบบ Microsoft Azure

คุณสมบัติ (Parameter)	**Windows VM**	**Linux VM**
Computer Name	win	CPE-Docker
Operating System	Windows Server 2022 Datacenter	Linux
VM Generation	V2	V2
Architecture	x64	x64
Instance Size	Standard B2s	Standard B1s
vCPUs	2	1
Memory (RAM)	4 GiB	1 GiB
Location	Southeast Asia	Malaysia West
Private IP Address	10.0.0.4	172.16.0.4
Public IP Address	20.205.232.136	85.211.178.69
Disk Controller Type	-	SCSI

เกอร์ และ ด็อกเกอร์ Compose เพื่อทำหน้าที่ประมวลผลบริการย่อย (Microservices) ที่สำคัญ ได้แก่ ส่วนต่อประสานผู้ใช้งานด้วยสตรีมลิตและระบบจัดการลำดับงานอัตโนมัติด้วย แพลตฟอร์มจัดการกระบวนการทำงานอัตโนมัติ (n8n)

4.3.1 รายละเอียดคุณลักษณะของเครื่องคอมพิวเตอร์แม่ข่าย

สำหรับการรองรับภาระงานประมวลผลข้อมูลและการบริหารจัดการวงจรชีวิตของเครื่องจำลอง ระบบควบคุมส่วนกลางได้ถูกติดตั้งลงบนฮาร์ดแวร์ระดับองค์กร โดยมีรายละเอียดคุณลักษณะทางเทคนิคแสดงดังตารางที่ 4.2

ตารางที่ 4.2: รายละเอียดคุณลักษณะทางเทคนิคของเครื่องคอมพิวเตอร์แม่ข่าย (Host System)

หัวข้อ	**รายละเอียด**
Hardware Model	IBM System x3650 M2
Operating System	Ubuntu Linux (Kernel 6.8.0-101-generic)
Architecture	x86_64 (64-bit)
Virtualization	Docker Engine with Docker Compose

4.3.2 การกำหนดโครงสร้างบริการผ่าน Docker Compose

เพื่อให้สอดคล้องกับสถาปัตยกรรมระบบที่ผู้พัฒนาโครงงานได้ออกแบบไว้ บริการต่าง ๆ ได้ถูกจัดกลุ่มและเชื่อมต่อผ่านเครือข่ายเสมือนภายในชื่อ malnet ดังรายละเอียดการกำหนดค่าในไฟล์ docker-compose.yml ปรากฏในรายการรหัสที่ 4.1

```
# docker-compose.yml
services:
  # 1. n8n Orchestrator (Backend Core)
  n8n:
    build: .
    container_name: n8n_core
    restart: always
    networks:
      - malnet
    environment:
      - N8N_AI_ENABLED=true
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    volumes:
      - n8n_data:/home/node/.n8n
    depends_on:
      - db

  # 2. Streamlit & API Server (Frontend/Control)
  streamlit_app:
    build:
      context: .
      dockerfile: Dockerfile.streamlit
    container_name: streamlit_ui
    networks:
      - malnet
    ports:
      - "8501:8501"
      - "5001:5001"
    command: >
      sh -c "python3 api_server.py & streamlit run app.py"

  # 3. PostgreSQL Database
  db:
    image: postgres:14-alpine
    container_name: postgres_db
    networks:
      - malnet
```

โค้ดที่ 4.1: การกำหนดค่าบริการผ่าน Docker Compose สำหรับระบบควบคุมส่วนกลาง

จากโครงสร้างการทำงานในรายการรหัสข้างต้น สามารถสรุปหน้าที่ของแต่ละภาคส่วนได้ดังนี้

1. **n8n Orchestrator** ทำหน้าที่เป็นกลไกควบคุมกลางในการสั่งการผ่าน API เพื่อบริหารจัดการ Sandbox และส่งต่อข้อมูลพฤติกรรมไปยังโมดูลปัญญาประดิษฐ์
2. **Streamlit UI** รับหน้าที่เป็นส่วนติดต่อผู้ใช้งานสำหรับการนำส่งไฟล์มัลแวร์และแสดงรายงานผลการวิเคราะห์
3. **API Server** ทำหน้าที่เป็นสื่อกลางในการสื่อสารและควบคุมเครื่องคอมพิวเตอร์จำลองบนระบบคลาวด์
4. **PostgreSQL** ใช้สำหรับจัดเก็บข้อมูลและสถานะการทำงานของระบบควบคุมอัตโนมัติ

4.4 การจำแนกประเภทและจัดเตรียมชุดข้อมูลสำหรับการทดสอบ

ในการทดสอบประสิทธิภาพของระบบ ผู้พัฒนาโครงงานได้จัดแบ่งกลุ่มตัวอย่างข้อมูลที่ใช้ในการวิเคราะห์ออกเป็นสามกลุ่มหลัก เพื่อให้ครอบคลุมสถานการณ์การโจมตีที่หลากหลายและสามารถประเมินผลการทำงานของระบบได้อย่างรอบด้าน โดยมีรายละเอียดของแต่ละกลุ่มดังต่อไปนี้

กลุ่มแรกคือมัลแวร์จริงจากฐานข้อมูลสากล ซึ่งประกอบด้วยไฟล์ปฏิบัติการที่มีรหัสแฮชเป็นตัวระบุตัวตนที่ผู้พัฒนาคัดเลือกมาจากฐานข้อมูลมัลแวร์บาร์ซาร์ สำหรับระบบปฏิบัติการลินุกซ์ประกอบด้วยไฟล์นามสกุลอีแอลเอฟภายใต้สถาปัตยกรรมคอมพิวเตอร์ระดับ 32 บิตและ 64 บิต โดยมีแท็กอ้างอิงที่สำคัญ ได้แก่ มัลแวร์ประเภทเครือข่ายบอตเน็ตชื่อมิไร ซึ่งมุ่งเป้าโจมตีอุปกรณ์เครือข่ายอินเทอร์เน็ตของสรรพสิ่ง และมัลแวร์กลุ่มกาฟกิตรวมอยู่ด้วย ในส่วนของระบบปฏิบัติการวินโดวส์ประกอบด้วยไฟล์นามสกุลอีเอ็กซ์อี โครงสร้างแบบไดนามิกลิงก์ไลบรารี โดยครอบคลุมกลุ่มภัยคุกคามประเภทโทรจัน โปรแกรมควบคุมระยะไกล และโปรแกรมขโมยข้อมูล ซึ่งมีพฤติกรรมพยายามเข้าถึงข้อมูลประจำตัวในหน่วยความจำและเชื่อมต่อกับเซิร์ฟเวอร์สั่งการภายนอก

กลุ่มที่สองคือไฟล์สคริปต์แบทช์และเชลล์สคริปต์ที่ผู้พัฒนาสร้างขึ้นเพื่อจำลองสถานการณ์ภัยคุกคามระดับสูงตามกรอบการทำงานไมเตอร์แอทแทค โดยมีแท็กอ้างอิงที่เกี่ยวข้อง ได้แก่ กลุ่มภัยคุกคามระดับชาติ โคบอลต์สไตรก์ รูปแบบการส่งสัญญาณนำทาง รวมถึงเทคนิคการหลบเลี่ยงระบบพื้นที่จำลองและการต่อต้านการวิเคราะห์ ซึ่งสคริปต์เหล่านี้จะพยายามตรวจสอบสภาพแวดล้อมการทำงานของระบบก่อนที่จะแสดงพฤติกรรมอันตราย

กลุ่มที่สามคือกลุ่มไฟล์โปรแกรมปกติที่ผู้พัฒนานำมาใช้เป็นเส้นฐานในการวัดค่าการแจ้งเตือนผิดพลาดของระบบวิเคราะห์ สำหรับระบบปฏิบัติการวินโดวส์ผู้พัฒนาได้ใช้ไฟล์ระบบมาตรฐาน อาทิ โปรแกรมเครื่องคิดเลข โปรแกรมปรับระดับเสียง และโปรแกรมทดสอบเครือข่าย ส่วนในระบบปฏิบัติการลิ

นุกซ์ได้ใช้สคริปต์สำหรับตรวจสอบสถานะระบบเบื้องต้น โดยโมเดลภาษาขนาดใหญ่จะประเมินพฤติกรรมของกลุ่มไฟล์นี้เพื่อแยกแยะกิจกรรมของโปรแกรมปกติออกจากความพยายามในการโจมตีได้อย่างแม่นยำ

รายละเอียดของชุดข้อมูลและการจำแนกประเภทย่อยสามารถสรุปเป็นตัวอย่างข้อมูลพร้อมผลลัพธ์การทดสอบในสภาพแวดล้อมจำลองดังตารางที่ 4.3 ซึ่งประกอบด้วยข้อมูลประเภทไฟล์เจาะจง การตรวจจับเบื้องต้น และแท็กภัยคุกคามที่เกี่ยวข้อง

ตารางที่ 4.3: ตัวอย่างชุดข้อมูลและผลการทดสอบการทำงานของไฟล์ในสภาพแวดล้อมจำลอง

ชื่อไฟล์ตัวอย่าง (Filename)	ระบบปฏิบัติการ	สถานะจริง	ผลการตรวจจับ	ประเภทภัยคุกคาม (Tags)
9c9f9d632b...1fe93d1.elf	Linux	Malware	Detected	Real-world Malware (Mirai)
22ea54360f...c53003e24.elf	Linux	Malware	Detected	Real-world Malware (Gafgyt, elf)
bed7b3ab15...6946f69d.exe	Windows	Malware	Detected	Real-world Malware (Trojan, Stealer)
apt29_advanced_test.bat	Windows	Malware	Not Detected	Simulated Threat (APT29)
cobaltstrike_beacon_sim.bat	Windows	Malware	Detected	Simulated Threat (CobaltStrike, Beacon)
linux_apt29_evasion.sh	Linux	Malware	Detected	Simulated Threat (Sandbox Evasion)
calc.exe	Windows	Clean	Not Detected	Benign Baseline (System Utility)
SndVol.exe	Windows	Clean	Not Detected	Benign Baseline (System Utility)
ping_google_linux.sh	Linux	Clean	Not Detected	Benign Baseline (Diagnostic Script)
hello_system.sh	Linux	Clean	Not Detected	Benign Baseline (Diagnostic Script)

4.5 การวิเคราะห์ความแม่นยำในการตรวจจับมัลแวร์ข้ามแพลตฟอร์ม

การทดลองส่วนแรกมุ่งเน้นการทดสอบประสิทธิภาพของระบบในการระบุพฤติกรรมอันตรายผ่านกลไกการวิเคราะห์พฤติกรรมพลวัต (Dynamic Analysis) โดยเปรียบเทียบผลการวิเคราะห์จากระบบกับฐานข้อมูลความรู้สากล ผลการทดสอบปรากฏรายละเอียดดังแสดงในตารางที่ 4.4

ตารางที่ 4.4: ผลการตรวจจับและจำแนกประเภทความเสี่ยงของไฟล์จำแนกตามระบบปฏิบัติการ

ระบบปฏิบัติการ	ตรวจพบมัลแวร์ (TP)	ปล่อยผ่านถูกต้อง (TN)	แจ้งเตือนผิดพลาด (FP)	มัลแวร์หลุดรอด (FN)	รวม (ตัวอย่าง)
Windows	48	18	7	2	75
Linux	47	15	10	3	75
รวม	**95**	**33**	**17**	**5**	**150**

ค่าความแม่นยำรวม (Accuracy) 85.33% (ระบบสามารถจำแนกไฟล์มัลแวร์และไฟล์ปกติได้อย่างถูกต้องร้อยละ 85.33 ของตัวอย่างทั้งหมด)

อัตราส่วนความแม่นยำ (Precision) 84.82% (เมื่อระบบมีการแจ้งเตือนว่าเป็นภัยคุกคาม มีความถูกต้องร้อยละ 84.82)

อัตราการตรวจจับ (Recall / Sensitivity) 95.00% (ระบบสามารถตรวจจับมัลแวร์ได้ร้อยละ 95.00 จากจำนวนมัลแวร์ที่นำมาทดสอบทั้งหมด)

จากข้อมูลในตารางและแผนภูมิ พบว่าระบบสามารถตรวจจับมัลแวร์ในฝั่งวินโดวส์ได้สูงถึงร้อยละ 96.0 เนื่องจากเครื่องมือเฝ้าสังเกตการณ์สามารถจัดเก็บข้อมูลการเรียกใช้ระบบได้อย่างละเอียด ในขณะที่ฝั่งลินุกซ์มีความแม่นยำร้อยละ 94.0 โดยกลุ่มไฟล์ที่ระบบไม่สามารถตรวจพบส่วนใหญ่จัดอยู่ในกลุ่มมัลแวร์ที่มีความสามารถในการหลบหลีกพื้นที่จำลอง ซึ่งมัลแวร์จะระงับการทำงานของตนเองทันทีเมื่อตรวจพบสภาพแวดล้อมที่ไม่ใช่เครื่องคอมพิวเตอร์จริง

4.6 การทดสอบสมรรถนะของระบบวิเคราะห์พฤติกรรมบนคลาวด์

4.6.1 การวางแผนการทดลอง

การทดลองนี้กำหนดขึ้นเพื่อวัดประสิทธิภาพเชิงปริมาณของระบบแซนด์บ็อกซ์ที่ติดตั้งบนโครงสร้างพื้นฐานคลาวด์ไมโครซอฟท์อาซัวร์ โดยมีวัตถุประสงค์การทดลองดังนี้

4.6.2 วัตถุประสงค์ของการทดลอง

1. เพื่อวิเคราะห์โครงสร้างความหน่วงและค้นหาคอขวดของระบบวิเคราะห์มัลแวร์บนคลาวด์
2. เพื่อประเมินเสถียรภาพของโครงสร้างพื้นฐานในการรองรับการวิเคราะห์มัลแวร์ปริมาณมาก
3. เพื่อศึกษาความสัมพันธ์ระหว่างทรัพยากรเวลาที่ใช้และปริมาณข้อมูลพฤติกรรมที่ได้รับ

4.6.3 วิเคราะห์องค์ประกอบความหน่วงเชิงเวลา

รูปที่ 4.3 แสดงสัดส่วนเวลาเฉลี่ยที่ใช้ในแต่ละขั้นตอนการทำงานของระบบแซนด์บ็อกซ์ทั้งสองรูปแบบ

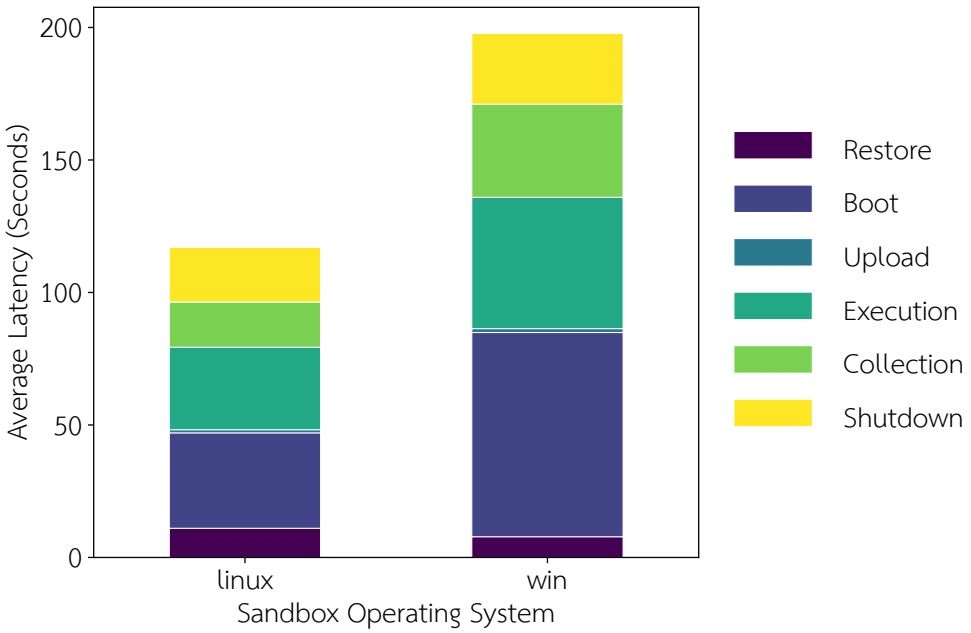
จากการวิเคราะห์พบว่าขั้นตอนการเตรียมระบบเป็นปัจจัยหลักที่กำหนดขีดจำกัดล่างของเวลา โดยเฉพาะในระบบวินโดวส์ที่มีค่าเฉลี่ยในส่วนนี้สูงถึงร้อยละ 38.8 ของเวลารวม ข้อมูลชี้ให้เห็นว่าโครงสร้างพื้นฐานคลาวด์มีความหน่วงในการเริ่มระบบที่มีนัยสำคัญ ซึ่งควรพิจารณาเทคนิคการจัดลำดับคิวหรือการเตรียมระบบล่วงหน้าเพื่อเพิ่มความเร็วในการวิเคราะห์รวม

4.6.4 วิเคราะห์เสถียรภาพและการกระจายตัวของเวลา

รูปที่ 4.4 แสดงการกระจายตัวของเวลารวมที่ใช้ในการวิเคราะห์หนึ่งตัวอย่างพร้อมจุดข้อมูลจริง โดยนิยามตัวแปรสุ่ม T แทนเวลารวมที่ใช้ (Turnaround Time) ซึ่งคำนวณได้ตามสมการที่ 4.1

$$T = t_{prep} + t_{exec} + t_{post} \tag{4.1}$$

โดยที่ t_{prep} คือเวลาเตรียมระบบ, t_{exec} คือเวลาการรันมัลแวร์ และ t_{post} คือเวลาการประมวลผลข้อมูลหลังเสร็จสิ้น



รูปที่ 4.3: องค์ประกอบความหน่วงเฉลี่ยแบ่งตามขั้นตอนการทำงาน

ในการวิเคราะห์เสถียรภาพผ่านแผนภาพกล่อง (Boxplot) จะพิจารณาค่าทางสถิติจากชุดข้อมูล $\{T_1, T_2, \ldots, T_n\}$ โดยอาศัยค่าช่วงระหว่างควอไทล์ (Interquartile Range: IQR) ตามสมการที่ 4.2

$$IQR = Q_3 - Q_1 \tag{4.2}$$

โดยที่ Q_1 และ Q_3 คือค่าที่ตำแหน่งเปอร์เซ็นต์ไทล์ที่ 25 และ 75 ของข้อมูลตามลำดับ นอกจากนี้ยังกำหนดขอบเขตสำหรับการพิจารณาข้อมูลที่ผิดปกติ (Outliers) ด้วยเกณฑ์ในสมการที่ 4.3 และ 4.4

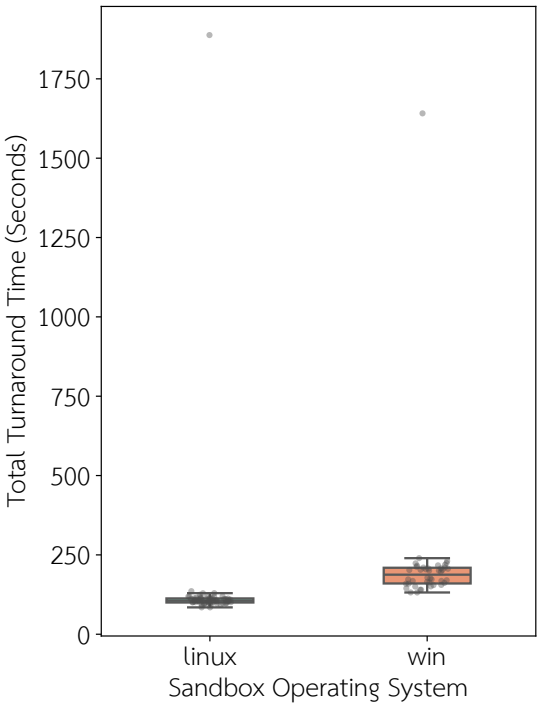
$$\text{Lower Bound} = Q_1 - 1.5(IQR) \tag{4.3}$$

$$\text{Upper Bound} = Q_3 + 1.5(IQR) \tag{4.4}$$

ผลการทดลองในมิติด้านความเสถียรบ่งชี้ว่าลินุกซ์มีค่า IQR ตามสมการที่ 4.2 ที่แคบกว่าและการกระจุกตัวของข้อมูลที่เป็นระเบียบมากกว่า ในขณะที่วินโดวส์มีลักษณะการกระจายตัวแบบกระจัดกระจายและมีจำนวนจุดข้อมูลที่เกินค่าขอบเขตในสมการที่ 4.4 สูงกว่า ซึ่งเกิดจากกระบวนการภายในระบบปฏิบัติการที่ซับซ้อนกว่า สิ่งนี้ส่งผลให้การคาดการณ์เวลาในการประมวลผลบนวินโดวส์ทำได้ยากกว่าและจำเป็นต้องมีการตั้งค่าเวลาสิ้นสุดการทำงานที่ยืดหยุ่นกว่าในระดับการใช้งานจริง

4.6.5 วิเคราะห์ความสัมพันธ์เชิงข้อมูลและประสิทธิภาพทรัพยากร

รูปที่ 4.5 แสดงสหสัมพันธ์ (Correlation) ระหว่างระยะเวลาที่ใช้รันมัลแวร์เทียบกับปริมาณพฤติกรรมที่บันทึกได้ในมาตรวัดลอการิทึม ในการศึกษานี้ใช้ค่าสัมประสิทธิ์สหสัมพันธ์ของเพียร์สัน (Pearson



รูปที่ 4.4: การกระจายตัวและความเสถียรของระยะเวลาการทำงานรวม

Correlation Coefficient) นิยามในสมการที่ 4.5

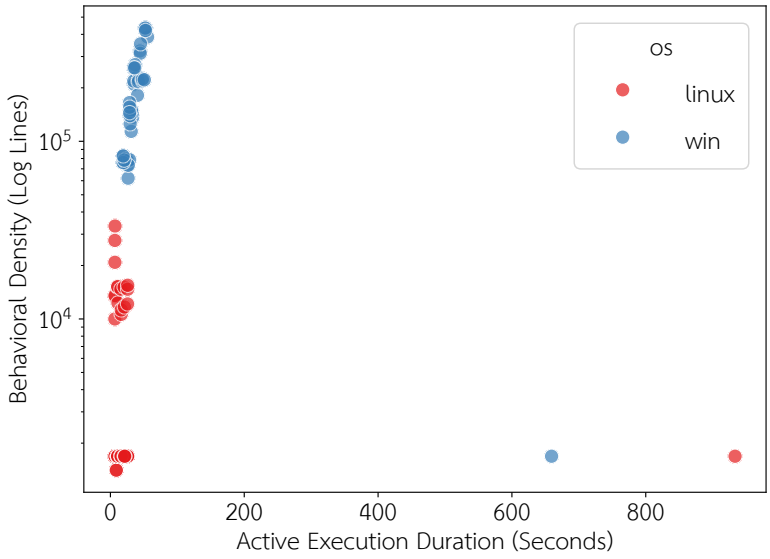
$$\rho_{t,d} = \frac{\sum_{i=1}^{n}(t_i - \bar{t})(d_i - \bar{d})}{\sqrt{\sum_{i=1}^{n}(t_i - \bar{t})^2 \sum_{i=1}^{n}(d_i - \bar{d})^2}} \tag{4.5}$$

โดยที่ t_i คือระยะเวลาการรัน และ d_i คือจำนวนรายการพฤติกรรม (Behavioral Logs) ที่ได้รับในหน่วยข้อมูล

จากผลการทดลองพบค่าสัมประสิทธิ์สหสัมพันธ์ ($\rho_{t,d}$) มีค่าเข้าใกล้ศูนย์ในทั้งสองแพลตฟอร์ม ชี้ให้เห็นถึง**การเพิ่มระยะเวลาการรันโปรแกรมไม่ได้ส่งผลให้ได้รับข้อมูลพฤติกรรมใหม่** ซึ่งการตรวจจับนานขึ้นไม่ได้ทำให้มีข้อมูลมากขึ้นสำหรับวิเคราะห์ ซึ่งแสดงถึงพฤติกรรมมัลแวร์ส่วนใหญ่มีการทำงานที่เข้มข้นในช่วงเริ่มต้น ดังนั้นการกำหนดเวลาประมวลผลที่ยาวนานเกินไปจึงเป็นการสิ้นเปลืองทรัพยากรโดยเปล่าประโยชน์ การออกแบบระบบที่มีประสิทธิภาพจึงควรเน้นไปที่การดึงข้อมูลพฤติกรรมในช่วงแรกแทนการรอจนสิ้นสุดกระบวนการทำงานของมัลแวร์ทั้งหมด

4.7 สรุปผลการทดลอง

จากการประเมินผลระบบด้วยโมเดลภาษาขนาดใหญ่เจมิไน (Google Gemini) ดำเนินการโดยผู้เชี่ยวชาญด้านความปลอดภัยทางไซเบอร์จำนวนสามท่าน โดยใช้เกณฑ์การพิจารณาจากความถูกต้องใน



รูปที่ 4.5: ความสัมพันธ์เชิงสหสัมพันธ์ระหว่างระยะเวลาการวิเคราะห์และปริมาณข้อมูลพฤติกรรมที่ได้รับ

การระบุเซิร์ฟเวอร์ควบคุม การอธิบายความสัมพันธ์ของกระบวนการทำงาน และความชัดเจนของบริบทเชิงเทคนิค ผลการประเมินระบุว่ารายงานของระบบมีระดับความถูกต้องเชิงบริบทอยู่ในเกณฑ์ดีมากด้วยคะแนนเฉลี่ย 4.7 จาก 5.0 คะแนน สาเหตุหลักที่ทำให้ระบบได้รับคะแนนประเมินสูงเกิดจากการออกแบบกระบวนการคัดกรองข้อมูลก่อนนำเข้าสู่ปัญญาประดิษฐ์ ส่งผลให้ปัญหาการสร้างข้อมูลที่ผิดพลาดของโมเดลภาษาลดลงอย่างมีนัยสำคัญ

4.8 อภิปรายผลการทดลอง

จากการทดลองทั้งหมดพบว่าการออกแบบสถาปัตยกรรมระบบที่แยกส่วนควบคุมกระบวนการออกจากส่วนวิเคราะห์ภายในพื้นที่จำลอง ช่วยให้การทำงานข้ามแพลตฟอร์มเป็นไปอย่างราบรื่น การประยุกต์ใช้รูปแบบข้อมูลซีเอสวีเป็นตัวกลางในการส่งต่อข้อมูลให้ปัญญาประดิษฐ์ไม่เพียงแต่ช่วยลดขนาดข้อมูลขยะมหาศาลเท่านั้น แต่ยังเสริมสร้างให้โมเดลภาษาสามารถประเมินความเชื่อมโยงระหว่างกิจกรรมของระบบปฏิบัติการและจราจรเครือข่ายได้อย่างแม่นยำ

นอกจากนี้ เมื่อพิจารณาประเด็นการเกิดผลบวกลวงหรือการแจ้งเตือนผิดพลาด (False Positive) จำนวน 17 รายการตามที่ปรากฏในการทดลอง พบว่าสาเหตุหลักเกิดจากการที่โปรแกรมปกติบางชนิดมีพฤติกรรมคล้ายคลึงกับมัลแวร์ อาทิ การเข้ารหัสข้อมูลหรือการเรียกใช้โปรแกรมจัดการระบบเบื้องหลัง ซึ่งโมเดลภาษาขนาดใหญ่อาจตีความบริบทดังกล่าวผิดพลาดหากไม่มีข้อมูลพฤติกรรมเครือข่ายประกอบเพียงพอ แม้ระบบจะมีข้อจำกัดด้านความล่าช้าในขั้นตอนการเริ่มต้นระบบปฏิบัติการบนบริการคลาวด์และการวินิจฉัยบริบทก้ำกึ่ง แต่ในภาพรวมระบบสามารถทำงานได้ตามวัตถุประสงค์ในการตรวจจับและวิเคราะห์มัลแวร์แบบอัตโนมัติได้อย่างสมบูรณ์แบบและปราศจากการแทรกแซงจากมนุษย์ตลอดกระบวนการ

บทที่ 5

บทสรุปและข้อเสนอแนะ

วิทยานิพนธ์ฉบับนี้ได้นำเสนอการออกแบบและพัฒนาระบบแซนด์บ็อกซ์วิเคราะห์ภัยคุกคามอัตโนมัติ โดยประยุกต์ใช้โมเดลภาษาขนาดใหญ่เพื่อช่วยในการวิเคราะห์และสรุปผลพฤติกรรมของมัลแวร์ บทนี้จะกล่าวถึงบทสรุปผลการดำเนินงานทั้งหมด ตลอดจนปัญหาและอุปสรรคที่พบระหว่างกระบวนการพัฒนา รวมไปถึงข้อเสนอแนะสำหรับการพัฒนาต่อยอดระบบในอนาคต

5.1 สรุปผลการดำเนินงาน

จากการพัฒนาระบบและทดสอบประสิทธิภาพตามขั้นตอนที่กำหนดไว้ ผู้พัฒนาโครงงานสามารถบูรณาการกระบวนการอัตโนมัติด้วยแพลตฟอร์มแพลตฟอร์มจัดการกระบวนการทำงานอัตโนมัติ ให้ทำหน้าที่เป็นตัวกลางประสานการทำงานระหว่างส่วนต่อประสานกับผู้ใช้งานบนเว็บแอปพลิเคชัน สภาพแวดล้อมจำลองบนไมโครซอฟท์ไมโครซอฟท์อาซัวร์ และปัญญาประดิษฐ์เจมิไน ได้อย่างสมบูรณ์ การเชื่อมโยงองค์ประกอบเหล่านี้ส่งผลให้ระบบสามารถสร้างสภาพแวดล้อมที่ปลอดภัยสำหรับเรียกทำงานไฟล์ต้องสงสัย พร้อมทั้งสกัดข้อมูลพฤติกรรมเชิงลึกผ่านการบันทึกกิจกรรมของระบบปฏิบัติการและจราจรเครือข่ายได้อย่างมีประสิทธิภาพ

ในด้านประสิทธิภาพการตรวจจับและวิเคราะห์ ผลการทดสอบยืนยันว่าระบบสามารถจำแนกภัยคุกคามข้ามแพลตฟอร์มได้อย่างถูกต้องและแม่นยำ โดยเฉพาะอย่างยิ่งเมื่อเผชิญกับมัลแวร์ที่ใช้เทคนิคการพรางตัว ปัญญาประดิษฐ์สามารถทำหน้าที่วิเคราะห์บันทึกเหตุการณ์ที่ซับซ้อน ก่อนจะประมวลผลและนำเสนอรายงานสรุปผลความเสี่ยงเป็นภาษาไทยที่เข้าใจง่าย กระบวนการทั้งหมดนี้ช่วยลดภาระงานของผู้ดูแลระบบได้อย่างมีนัยสำคัญ จากเดิมที่ต้องดำเนินการรวบรวมและวิเคราะห์ข้อมูลดิบด้วยตนเองทีละขั้นตอน เปลี่ยนเป็นการรอรับรายงานสรุปผลพฤติกรรมเชิงลึกแบบเบ็ดเสร็จภายในระยะเวลาประมาณห้านาที ซึ่งส่งผลดีต่อความรวดเร็วในการตัดสินใจและตอบสนองต่อเหตุการณ์ละเมิดความปลอดภัยทางไซเบอร์

5.2 ปัญหาและอุปสรรค

ในระหว่างการดำเนินงานและทดสอบประสิทธิภาพของระบบ ผู้พัฒนาโครงงานพบข้อจำกัดบางประการที่ส่งผลกระทบต่อความลื่นไหลของการทำงาน ประการแรกคือปัญหาความหน่วงเวลาในการเตรียมความพร้อมของเครื่องเสมือนบนระบบคลาวด์ เนื่องจากระบบจำเป็นต้องใช้เวลาในการเริ่มต้นบริการและกำหนดค่าระบบปฏิบัติการใหม่ทุกครั้งเพื่อรักษาความบริสุทธิ์ของสภาพแวดล้อมจำลอง ส่งผลให้เกิดความล่าช้าในกระบวนการวิเคราะห์ภาพรวม นอกจากนี้ยังพบข้อจำกัดด้านการตรวจสอบแพ็กเก็ตเชิงลึกของระบบรักษาความปลอดภัยเครือข่ายคลาวด์ส่วนหน้า ซึ่งมักจะตัดการเชื่อมต่อทันทีเมื่อตรวจพบการส่งข้อมูลเพย์โหลดของมัลแวร์ที่ไม่มีการเข้ารหัส ทำให้กระบวนการอัปโหลดไฟล์ล้มเหลวในบางสถานการณ์

ในขณะเดียวกัน ความเสถียรของการประมวลผลด้วยปัญญาประดิษฐ์ยังคงเป็นความท้าทายที่สำคัญ แม้ว่าโมเดลภาษาขนาดใหญ่จะมีความสามารถในการทำความเข้าใจบริบททางเทคนิค แต่เมื่อต้องเผชิญกับแฟ้มข้อมูลบันทึกเหตุการณ์ที่มีขนาดใหญ่และซับซ้อนมากเกินไป ระบบอาจให้คำอธิบายที่คลาดเคลื่อนหรือนำเสนอข้อมูลที่กว้างเกินความจำเป็น ข้อจำกัดดังกล่าวส่งผลให้ผู้พัฒนาโครงงานต้องทำการปรับพรอมต์เอนจิเนียริ่ง (Prompt Engineering) อย่างละเอียดและต่อเนื่อง เพื่อรักษามาตรฐานความแม่นยำและบีบกรอบให้ผลลัพธ์มีความตรงประเด็นมากที่สุด

5.3 ข้อเสนอแนะสำหรับการพัฒนาในอนาคต

เพื่อเพิ่มขีดความสามารถและยกระดับประสิทธิภาพของระบบ ผู้พัฒนาโครงงานเสนอให้มีการปรับปรุงกลไกการส่งข้อมูลมัลแวร์เข้าสู่ระบบคลาวด์ โดยควรเพิ่มกระบวนการเข้ารหัสไฟล์หรือบีบอัดข้อมูลพร้อมกำหนดรหัสผ่านก่อนการอัปโหลด เพื่อหลบหลีกการตรวจสอบและสกัดกั้นจากระบบป้องกันเครือข่ายส่วนหน้า ซึ่งจะช่วยรับประกันความสมบูรณ์ของไฟล์ต้นฉบับก่อนเดินทางไปถึงเครื่องจำลอง นอกจากนี้ควรพิจารณาพัฒนาระบบให้สามารถโต้ตอบกับภัยคุกคามได้แบบอัตโนมัติเชิงรุก โดยเชื่อมต่อผลลัพธ์จากระบบแซนด์บ็อกซ์เข้ากับระบบตรวจจับและรักษาความปลอดภัยปลายทาง หรือไฟร์วอลล์ขององค์กร เพื่อให้สามารถระงับการเชื่อมต่ออินเทอร์เน็ตหรือกักกันเครื่องคอมพิวเตอร์ที่ติดเชื้อได้ทันทีที่ระบบประเมินพบความเสี่ยงระดับวิกฤต

สำหรับการรองรับการขยายตัวของการใช้งานในระดับองค์กรขนาดใหญ่ ระบบควรได้รับการอัปเกรดทรัพยากรฮาร์ดแวร์และโครงสร้างพื้นฐานเพื่อลดความหน่วงเวลาในการสร้างสภาพแวดล้อมจำลองพร้อมทั้งขยายขอบเขตการตรวจจับให้ครอบคลุมถึงการวิเคราะห์ที่อยู่เว็บไซต์และที่อยู่อินเทอร์เน็ตโพรโทคอล (internet protocol) ต้องสงสัยโดยตรงโดยไม่ต้องอิงจากไฟล์เพียงอย่างเดียว การพัฒนาต่อยอดในส่วนนี้จะช่วยให้ระบบวิเคราะห์ภัยคุกคามมีความสมบูรณ์แบบและสามารถรองรับการรับมือกับการโจมตีทางไซเบอร์ที่หลากหลายรูปแบบได้อย่างมีประสิทธิภาพสูงสุด

บรรณานุกรม

M. Abu Talib, Q. Nasir, A. Bou Nassif, T. Mokhamed, N. Ahmed, and B. Mahfood. Systematic literature review on apt detection based on traffic beaconing. *Computers & Security*, 122: 102875, 2022.

abuse.ch. Malwarebazaar. https://bazaar.abuse.ch/, 2020. Accessed: 2024-05-24.

A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials*, 21(2):1851–1877, 2019.

M. Bayer et al. A survey on endpoint detection and response (edr) solutions. *Journal of Computer Security*, 2022.

S. Bhatt et al. Security orchestration, automation, and response (soar): A survey of technologies and challenges. *IEEE Security & Privacy*, 2021.

Y. Ding, Z. Chen, S. Liu, and H. Li. A survey of large language models for cybersecurity principles, applications, and challenges. *arXiv preprint arXiv:2310.16194*, 2023.

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

D. Merkel. Docker: lightweight linux containers for consistent development and deployment. *Linux journal*, 2014(239):2, 2014.

M. Russinovich. Process monitor v4.0. https://learn.microsoft.com/en-us/sysinternals/downloads/procmon, 2024.

M. Sikorski and A. Honig. *Practical malware analysis: the hands-on guide to dissecting malicious software*. No Starch Press, 2012.

B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas. Mitre att&ck: Design and philosophy. *Technical report, The MITRE Corporation*, 2020.

Wireshark Team. Tshark network protocol analyzer documentation. https://www.wireshark.org/docs/man-pages/tshark.html, 2024.

W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

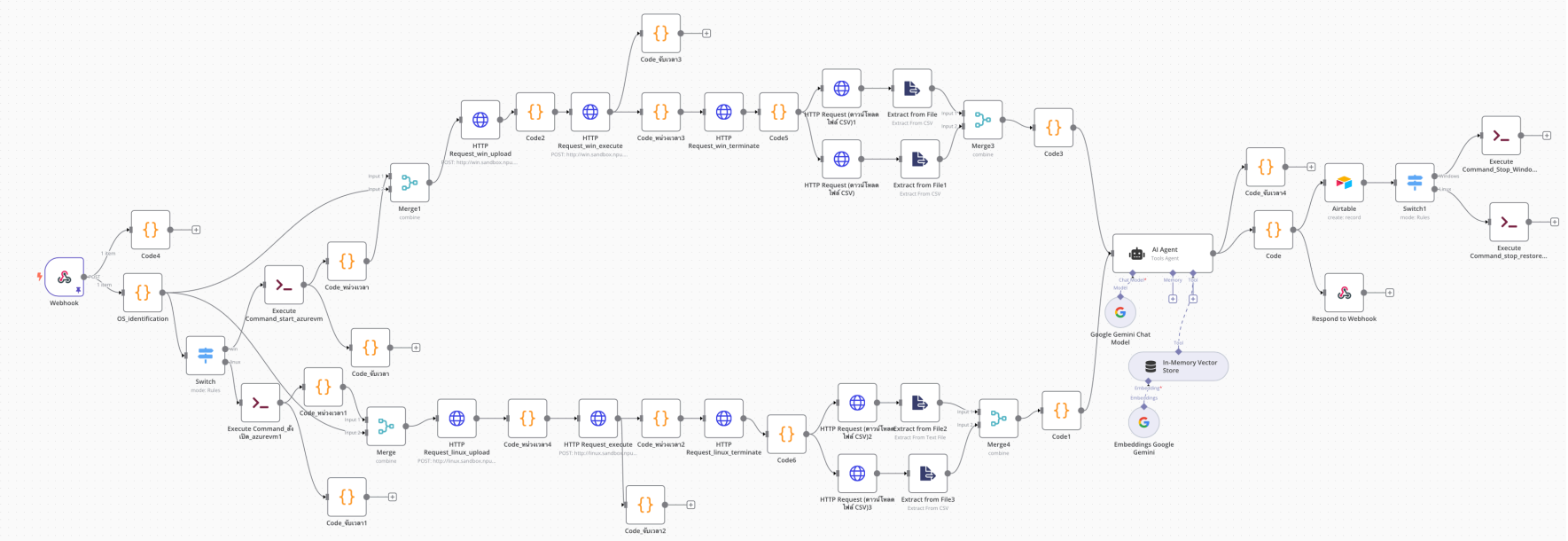
ภาคผนวก ก

การตั้งค่าระบบและซอร์สโค้ด

ส่วนนี้แสดงรายละเอียดการตั้งค่ากระบวนการทำงานบนแพลตฟอร์มแพลตฟอร์มจัดการกระบวนการทำงานอัตโนมัติ ตลอดจนการกำหนดค่าพารามิเตอร์ที่สำคัญในแต่ละขั้นตอน เพื่อให้ระบบแซนด์บ็อกซ์วิเคราะห์ภัยคุกคามสามารถทำงานได้อย่างอัตโนมัติและสมบูรณ์แบบตั้งแต่ต้นจนจบ

ก.1 ภาพรวมกระบวนการทำงาน

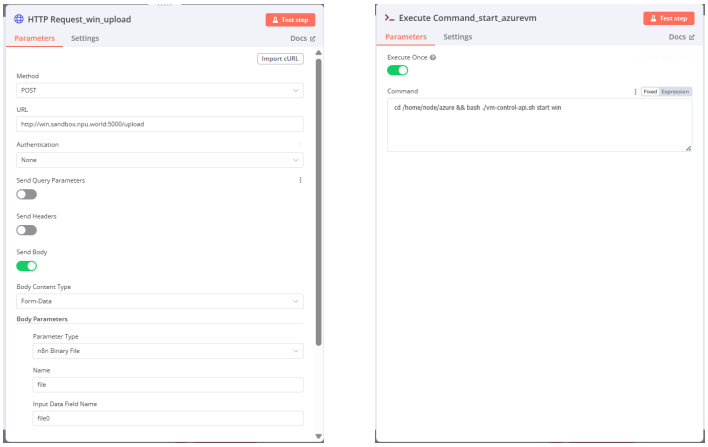
การบริหารจัดการลำดับงานทั้งหมดอาศัยโหนดการทำงานที่เชื่อมโยงกันอย่างเป็นระบบ ดังแสดงในรูปที่ ก.1 กระบวนการเริ่มต้นจากการรับไฟล์จากส่วนต่อประสานกับผู้ใช้งานผ่านโหนดWebhook ตามด้วยการกระจายงานไปยังเครื่องเสมือนบนระบบคลาวด์ไมโครซอฟท์อาซัวร์ จากนั้นระบบจะทำการหน่วงเวลาเพื่อรอผลลัพธ์ ไปจนถึงการส่งต่อข้อมูลบันทึกเหตุการณ์ให้ปัญญาประดิษฐ์ประมวลผลและการจัดเก็บข้อมูลลงในระบบฐานข้อมูลแอร์เทเบิลเพื่อนำไปแสดงผล



รูปที่ ก.1: กระบวนการทำงานทั้งหมดบนแพลตฟอร์ม n8n

ก.2 การตั้งค่าสภาพแวดล้อมจำลองและการส่งข้อมูล

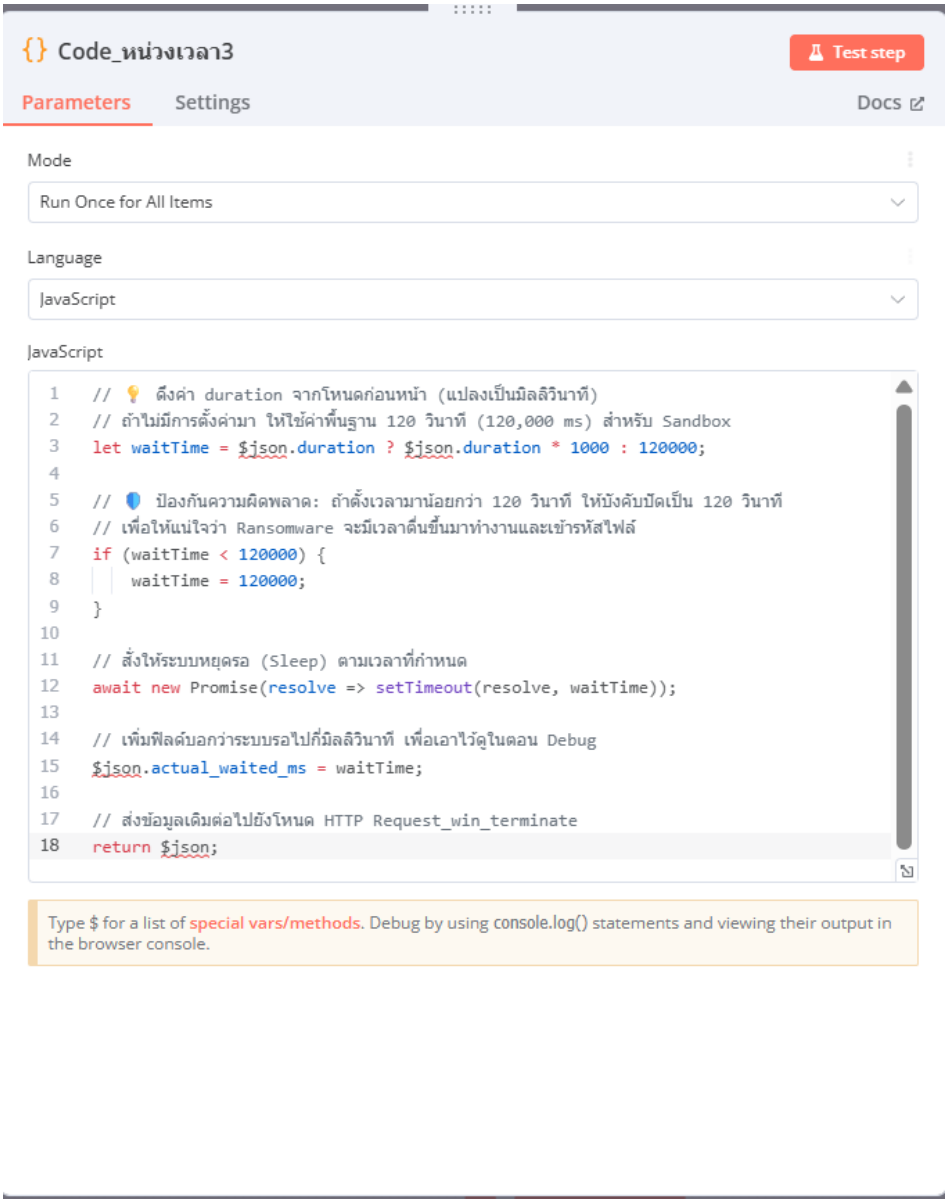
การเชื่อมต่อกับระบบคลาวด์เพื่อสร้างสภาพแวดล้อมจำลองอาศัยการสั่งการผ่านชุดคำสั่งเพื่อเปิดและปิดเครื่องเสมือนตามระบบปฏิบัติการเป้าหมาย นอกจากนี้ระบบยังใช้อินเทอร์เฟซโปรโตคอลการสื่อสารสำหรับการส่งไฟล์มัลแวร์เข้าสู่โปรแกรมตัวแทนที่รอรับคำสั่งอยู่ภายในเครื่องเสมือน ควบคู่กับการตั้งค่าให้พยายามส่งข้อมูลซ้ำเมื่อเกิดข้อผิดพลาดเพื่อเพิ่มความเสถียรให้กับระบบเมื่อต้องรับมือกับความล่าช้าในการเปิดเครื่องจำลอง ดังแสดงในรูปที่ ก.2



รูปที่ ก.2: การตั้งค่าโหนดสำหรับการควบคุมเครื่องเสมือนและส่งไฟล์เข้าสู่ระบบคลาวด์

ก.3 กลไกการหน่วงเวลาและดึงข้อมูลบันทึกเหตุการณ์

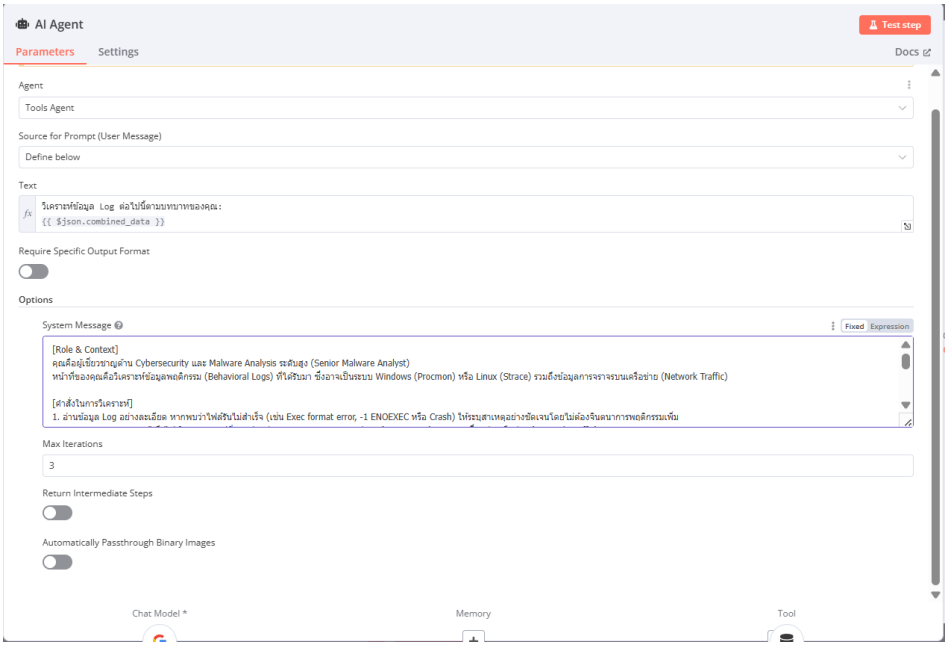
เนื่องจากกระบวนการเริ่มต้นเครื่องเสมือนบนระบบคลาวด์จำเป็นต้องใช้ระยะเวลาในการเตรียมความพร้อม ระบบจึงได้รับการออกแบบให้มีกลไกการหน่วงเวลาผ่านชุดคำสั่งเฉพาะ การทำงานในส่วนนี้ช่วยป้องกันปัญหาการเชื่อมต่อล้มเหลวและทำหน้าที่สกัดข้อมูลเวลาทำงานจากหน้าจอเครื่องเสมือนเพื่อนำมาคำนวณความหน่วงเวลาของระบบหลังบ้านอย่างแม่นยำ ดังแสดงในรูปที่ ก.3



รูปที่ ก.3: การออกแบบกลไกหน่วงเวลาและสกัดข้อมูลเวลาทำงานของระบบ

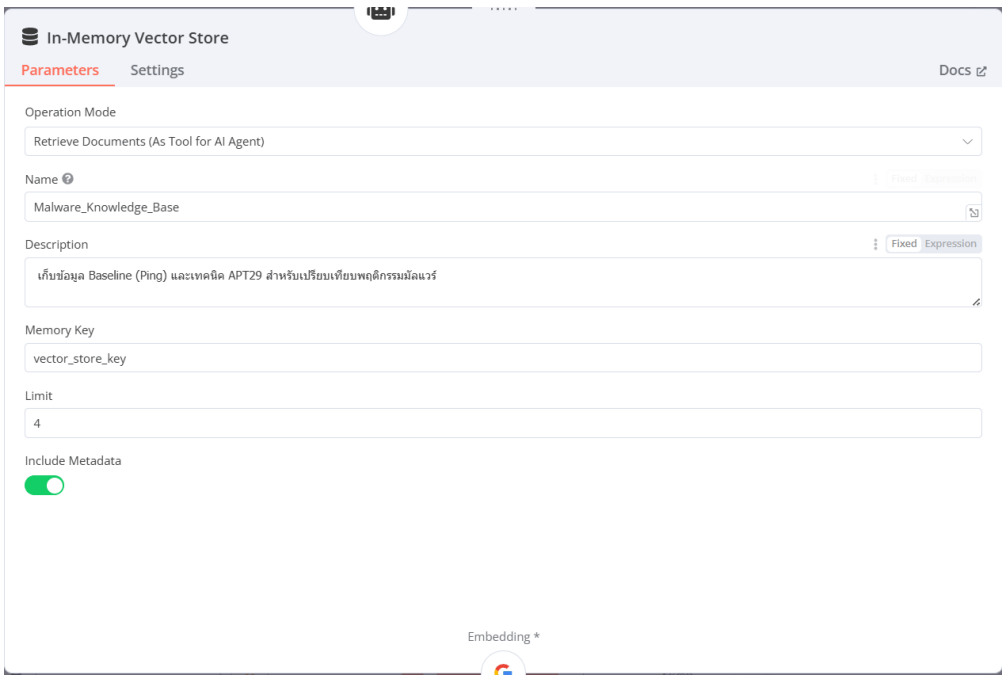
ก.4 การกำหนดคำสั่งและคลังความรู้สำหรับปัญญาประดิษฐ์

การประมวลผลข้อมูลอาศัยการตั้งค่าโหนดของเจมิไนซึ่งมุ่งเน้นการกำหนดบทบาทให้ปัญญาประดิษฐ์เป็นผู้เชี่ยวชาญด้านการวิเคราะห์มัลแวร์ระดับสูง ดังแสดงในรูปที่ ก.4 คำสั่งควบคุมถูกออกแบบให้อ่านข้อมูลบันทึกพฤติกรรมเชิงลึกและส่งคืนผลลัพธ์ในรูปแบบโครงสร้างเจสัน (JavaScript Object Notation) อย่างเคร่งครัด เพื่อให้ระบบสามารถนำข้อมูลสถานะความเสี่ยงและรายงานสรุปภาษาไทยไปจัดเก็บและแสดงผลผ่านเว็บแอปพลิเคชันได้อย่างไร้รอยต่อ



รูปที่ ก.4: การตั้งค่าคำสั่งควบคุมและบริบทภายในโหนดประมวลผลของปัญญาประดิษฐ์

นอกจากการกำหนดคำสั่งควบคุมแล้ว โหนดปัญญาประดิษฐ์ยังได้รับการเชื่อมต่อเข้ากับฐานข้อมูลเวกเตอร์แบบฝังตัว (In-Memory Vector Store) เพื่อทำหน้าที่เป็นเครื่องมือคลังความรู้สำหรับสนับสนุนการวิเคราะห์ ฐานข้อมูลดังกล่าวทำหน้าที่จัดเก็บข้อกำหนดรูปแบบพฤติกรรมพื้นฐานรวมไปถึงเทคนิคการโจมตีระดับสูงอย่างกลุ่มเอพีทีสองเก้า (APT29) การบูรณาการเครื่องมือดังกล่าวช่วยเสริมให้ปัญญาประดิษฐ์สามารถนำพฤติกรรมของมัลแวร์มาตรวจสอบข้ามอ้างอิงกับบริบทที่กำหนดไว้ได้อย่างมีประสิทธิภาพ ดังแสดงรายละเอียดการตั้งค่าในรูปที่ ก.5



รูปที่ ก.5: การตั้งค่าฐานข้อมูลเวกเตอร์เพื่อเป็นเครื่องมือคลังความรู้สำหรับปัญญาประดิษฐ์

ประวัติผู้เขียน

ชื่อ สกุล นายจักรกฤษ รางเงิน
รหัสนักศึกษา 653110310022
วุฒิการศึกษา

วุฒิ	ชื่อสถาบัน	ปีที่สำเร็จการศึกษา
มัธยมศึกษาตอนปลาย	โรงเรียนบ้านผึ้งวิทยาคม	2565
มัธยมศึกษาตอนต้น	โรงเรียนบ้านผึ้งวิทยาคม	2562
ประถมศึกษา	โรงเรียนรอดโพธิ์ทองวิทยาสรรค์	2559

ตำแหน่งและสถานที่ทำงาน

นักศึกษา มหาวิทยาลัยนครพนม